

Stanford CME 295 — Transformers & Large Language Models

Complete Reading Notes for the Autumn 2025 Lecture Series

Instructors: Afshine Amidi & Shervine Amidi

Contents

About these notes 4

- How to read this document 4
- Prerequisites the course assumes 4
- The shape of the whole course 4
- Notation used consistently throughout 5

Lecture 1 — The Transformer 6

- 1.1 The landscape of NLP tasks 6
- 1.2 Tokenisation 8
- 1.3 Representing tokens as vectors 9
- 1.4 Recurrent networks, and why they were abandoned 10
- 1.5 Where attention came from 11
- 1.6 The self-attention mechanism 12
- 1.7 The full architecture 14
- 1.8 End-to-end worked example 15

Lecture 2 — Transformer-based Models & Tricks 17

- 2.1 Position information, done properly 17
- 2.2 Layer normalisation 19
- 2.3 Making attention cheaper 20
- 2.4 The three families of Transformer-based model 22
- 2.5 BERT deep dive 23
- 2.6 Distillation 26

Lecture 3 — Large Language Models 28

- 3.1 What an LLM is 28
- 3.2 Mixture of Experts (MoE) 28
- 3.3 Generating a response 30
- 3.4 Prompting 33
- 3.5 Inference optimisations 35

Lecture 4 — LLM Training 39

- 4.1 The paradigm shift 39
- 4.2 Pretraining 39
- 4.3 Training optimisations 42
- 4.4 Supervised finetuning (SFT) 45
- 4.5 Parameter-efficient finetuning: LoRA 48

Lecture 5 — LLM Tuning (Preference Tuning) 52

- 5.1 Why preference tuning 52
- 5.2 Collecting preference data 53
- 5.3 Reinforcement learning, in language-model clothing 54
- 5.4 RLHF 55

5.5 Best-of-N: skipping RL entirely 59

5.6 DPO 60

Lecture 6 — LLM Reasoning 63

6.1 What counts as reasoning 63

6.2 Benchmarks and metrics for reasoning 65

6.3 Test-time scaling with RL 66

6.4 GRPO 68

6.5 The increasing-output-length pathology 69

6.6 Putting it together: the DeepSeek recipes 70

Lecture 7 — Agentic LLMs 73

7.1 Why retrieval 73

7.2 RAG 73

7.3 Tool calling 78

7.4 Agents 82

Lecture 8 — LLM Evaluation 86

8.1 What "evaluation" means, and why it is hard 86

8.2 Rule-based metrics 87

8.3 LLM-as-a-Judge 88

8.4 Evaluating agents: the failure-mode taxonomy 92

8.5 Benchmarks 94

Lecture 9 — Current Trends 98

9.1 Recap of the quarter 98

9.2 Transformers beyond text 99

9.3 Diffusion LLMs 101

9.4 Cross-pollination between modalities 103

9.5 Back to the basics: foundational research is still open 103

9.6 New frontiers and hardware 104

9.7 Applications: today and tomorrow 105

9.8 How to stay up to date 106

Closing synthesis 106

About these notes

How to read this document

These notes are written to **replace watching the nine lecture videos**. Every section follows the order in which the material is actually taught, and each technical idea is written out in full prose rather than as bullet-point slide fragments — because a slide that says "RoPE. Rotate query and key vectors" is useless without the twenty sentences the lecturer says on top of it.

Three kinds of text appear throughout, and it is worth knowing which is which:

- **Plain body text** is the lecture content itself: definitions, mechanisms, formulas, numbers, paper references.
- **Boxes labelled "Intuition"** are added explanation. They exist to answer the question "but why does that work?", which is usually the part that makes a technique stick in memory.
- **Boxes labelled "Watch out"** flag the places where notation is confusing, where two similar-sounding ideas are actually different, or where a naive reading of the slide would leave you with a wrong mental model.

Formulas are written in linear Unicode notation (for example $\text{softmax}(QK^\top/\sqrt{d_k})V$) so that they read cleanly without needing rendered LaTeX. Where the original slides showed a diagram, the diagram is described in words, because the description is what you actually need in order to reason about the mechanism.

Prerequisites the course assumes

You need machine-learning basics (loss functions, gradient descent, backpropagation, softmax, cross-entropy) and linear algebra (matrix multiplication, dot products, the idea of a low-rank factorisation). Nothing beyond that. No prior NLP knowledge is assumed — the course builds from tokenisation upwards.

The shape of the whole course

The nine lectures form one long argument, and it helps enormously to see the skeleton before the detail:

1. **Lectures 1-2** build the Transformer from nothing. Why we need tokenisation, why static word vectors are insufficient, why recurrence is slow, what attention actually computes, and how the encoder-decoder stack is assembled. Then the engineering that made the architecture practical: position encodings, normalisation placement, cheaper attention variants, and BERT as the canonical encoder-only model.
2. **Lectures 3-4** turn a Transformer into a Large Language Model. Decoder-only architectures, mixture-of-experts, how text is actually sampled at inference, what makes inference fast, and then how these models are trained at scale — pretraining, parallelism, precision, supervised finetuning, and LoRA/QLoRA for people without a GPU cluster.

3. **Lectures 5-6** are about alignment and reasoning. Preference data, reward models, RLHF with PPO, the DPO shortcut, and then how the same RL machinery — with verifiable rewards and GRPO instead of PPO — produces reasoning models like DeepSeek-R1.
4. **Lectures 7-8** are about systems. Retrieval-augmented generation, tool calling, MCP, agents and the ReAct loop; then the hardest practical problem of all, evaluating any of it: rule-based metrics, LLM-as-a-judge, agentic failure modes, and benchmarks.
5. **Lecture 9** steps back: Transformers outside text (ViT, VLMs), diffusion language models as a genuine alternative to autoregressive decoding, and where the field appears to be heading.

A single running example is used across all nine lectures — a cute teddy bear that is reading — and it is genuinely helpful. Every mechanism in the course is demonstrated on some variant of that sentence, so when you see "A cute teddy bear is reading." you should read it as "here comes a concrete worked example".

Notation used consistently throughout

Symbol	Meaning
V	vocabulary size (number of distinct tokens the model knows)
n	sequence length, in tokens
d_{model}	width of the model's residual stream / embedding dimension
d_k, d_v	per-head key and value dimensions
d_{FF}	hidden width of the feed-forward sub-layer
h	number of attention heads
N	number of stacked encoder or decoder layers
Q, K, V	query, key and value matrices inside an attention layer
θ	model parameters
π_θ	the model viewed as a policy (used from Lecture 5 onwards)
T	sampling temperature
β	strength of a KL / regularisation term

Lecture 1 — The Transformer

Video length: 1h42. Covered here: what NLP tasks look like and how they are scored, tokenisation, word representations and Word2vec, RNNs and LSTMs and why they fail, the origin of attention, the self-attention mechanism, the full encoder-decoder Transformer architecture, and one complete end-to-end worked example.

1.1 The landscape of NLP tasks

Before any architecture, the lecture sets up what we are trying to do. Every classical NLP task falls into one of three shapes, and the shape determines what the output head of your model looks like.

Shape 1 — Classification. Text in, one label out. Sentiment extraction ("this teddy bear is SO CUTE!" → positive), intent detection, language detection, topic modelling. The model reduces a whole sequence to a single decision.

Shape 2 — "Multi"-classification (token-level labelling). Text in, one label per token out. Part-of-speech tagging, named entity recognition (marking "teddy bear" as an entity), dependency parsing, constituency parsing. The input and output have the same length; you are labelling positions, not the whole sequence.

Shape 3 — Generation. Text in, new text out, of arbitrary length. Machine translation, question answering, summarisation, open-ended text generation.

Intuition. This taxonomy quietly predicts the whole rest of the course. Shapes 1 and 2 only need a good representation of the input, which is exactly what an encoder-only model like BERT provides (Lecture 2). Shape 3 needs the model to produce tokens one at a time conditioned on what it has already produced, which is exactly what a decoder-only LLM does (Lecture 3). The reason decoder-only models took over is that shape 3 can simulate shapes 1 and 2 — you can always ask a generative model to emit the word "positive" — while the reverse is not true.

How each shape is evaluated

Classification and token labelling use the standard confusion-matrix family:

- **Accuracy** — fraction of observations predicted correctly.
- **Precision** — of the items predicted positive, what fraction really were?
- **Recall** — of the items that really are positive, what fraction did we find?
- **F1** — the harmonic mean of precision and recall, $F_1 = 2PR/(P + R)$, which punishes a model that is excellent at one and terrible at the other.

For NER specifically, these are computed **at token level, per entity type** — you get a precision/recall/F1 for `PERSON`, another for `ORG`, and so on, which matters because entity types are wildly imbalanced.

Typical datasets: Amazon reviews, IMDB critiques and Twitter for sentiment; annotated Reuters newswire (CoNLL-2003, CoNLL++) for NER.

Generation is much harder to score, because there are many correct answers. The lecture introduces three metrics that reappear in Lecture 8:

- **BLEU** — measures how much of the generated text appears in the reference. It is precision-flavoured: it asks "of the n-grams I produced, how many were legitimate?" Designed for machine translation.
- **ROUGE** — the mirror image. It is recall-flavoured: "of the n-grams in the reference, how many did I manage to produce?" Designed for summarisation, where coverage matters more than terseness.
- **Perplexity (PPL)** — not a comparison against a reference at all, but a measure of how surprised the model is by real text. Formally it is the exponentiated average negative log-likelihood; intuitively, a perplexity of 20 means the model is, on average, as uncertain as if it were choosing uniformly among 20 tokens at each step. Lower is better.

Intuition on why BLEU/ROUGE are a pair. A translation system that outputs a single very safe word would score well on precision-style metrics if that word is always right, and terribly on recall. A system that dumps every plausible word would do the opposite. Reporting both is a crude way of pinning a system between the two failure modes. Lecture 8 will demolish both metrics as poor proxies for human judgement, but they remain the historical baseline.

The historical timeline

The lecture frames the field in four eras, and the framing explains why progress happened when it did:

Era	What appeared	What enabled it
1980s	Recurrent neural networks	theoretical foundations
1997	LSTM	theoretical foundations
2013	Word2vec	lots of data, growing compute
2017	Transformers	lots of data, growing compute
2020s	Large Language Models	fast iteration on ideas

The point being made is that the theory largely predates the results by decades. What changed was data volume and hardware, and then — in the 2020s — the sheer speed at which the community could try ideas.

Watch out. The lecture opens with a deliberately intimidating wall of abbreviations (BERT, RLHF, ROUGE, MLM, BLEU, LSTM, GRU, RAG, FLAN, T5, GLUE, DPO, NER, PEFT, PoS, NLG, METEOR, SQuAD, C4, BPE, PPL, GPT, F1, MRPC, LLaMA, MT, QA, LaaJ, WP, SFT, SP, PPO...) and then immediately organises them into six buckets: architectures/techniques, Transformer-based models, datasets, metrics, training strategies, and tasks. If you keep those six buckets in mind as you read, every new acronym in the course slots into one of them.

1.2 Tokenisation

A neural network consumes numbers, so the first job is chopping text into discrete units and assigning each an index. The lecture walks the same sentence — A cute teddy bear is reading. — down four levels of granularity.

Arbitrary / naive split. A | cute | teddy | bear | is | reading | . Note that punctuation becomes its own token; splitting on whitespace alone would glue reading. together and make it a different token from reading.

Word-level. A | cute | teddy bear | is | reading | . Multi-word expressions can be treated as single units. Simple and interpretable.

Sub-word level. A | cute | ted | ##dy | bear | is | read | ##ing | . Words are broken into stems and affixes. The ## prefix marks "this piece continues the previous word rather than starting a new one" — that is the WordPiece convention, and it is what lets you reconstruct the original string unambiguously.

Character level. A | c | u | t | e | ... Every character is a token.

The trade-off table

Method	Pros	Cons
Word-level	simple, interpretable	risk of out-of-vocabulary tokens; doesn't exploit shared roots
Sub-word (WordPiece, BPE)	exploits common prefixes/suffixes; vocabulary is learned from data	small residual OOV risk, but much less than word-level
Character-level	almost no OOV; robust to casing and misspellings	sequences become very long → slow; individual embeddings are not interpretable

Intuition — why sub-word won. Consider the words read, reading, reader, reads, reread. Word-level tokenisation gives you five unrelated indices, and if reread never appeared in training you cannot represent it at all. Character-level lets you represent anything but forces the model to relearn that r-e-a-d means something, at every position, from scratch — and multiplies your sequence length by about four, which for a Transformer means roughly sixteen times the attention cost. Sub-word tokenisation is the compromise: frequent words stay whole (cheap, and their embedding is well-trained), rare words decompose into pieces the model has seen before (ted + ##dy), and nothing is unrepresentable. This is why essentially every modern model uses BPE or a close relative.

Watch out. "Risk of OOV, though less than word-level" for sub-word is a deliberately careful phrasing. Sub-word tokenisers usually include a byte-level fallback, so in practice they are OOV-free — but the vocabulary itself is learned on a specific corpus, so text from a very different distribution (another language, heavy code, emoji) tokenises inefficiently even when it tokenises successfully. This inefficiency is a real cost: it consumes context window and money, since you are billed per token.

1.3 Representing tokens as vectors

Why one-hot encoding is not enough

The naive representation of a vocabulary of size V is a one-hot vector: a V -dimensional vector of zeros with a single 1 at the token's index. Two problems, both fatal:

1. **Size.** V is typically 30,000–100,000+. Every token is a vector that large, almost entirely zeros.
2. **No notion of similarity.** Every pair of distinct one-hot vectors is exactly equally distant. The representation of cat is precisely as far from kitten as it is from bulldozer. All the structure of language is thrown away.

The fix is a **learned embedding**: a dense vector of dimension d (with $d \ll V$) per token, whose values are trained. Mechanically this is a lookup table — a $V \times d$ matrix — and looking up a token is exactly the same operation as multiplying its one-hot vector by that matrix. Now similarity is meaningful: tokens that behave alike end up nearby in the d -dimensional space.

Word2vec (Mikolov et al., 2013)

The canonical way to obtain such embeddings without labelled data. The recipe:

- Train a neural network on a **proxy task** over billions of words of text.
- The task itself is disposable; what you keep is the **embedding layer** it learned along the way.

Two standard proxy tasks:

- **CBOW (Continuous Bag Of Words)** — given the surrounding context words, predict the missing centre word.
- **Skip-gram** — the reverse: given the centre word, predict the surrounding context words.

The architecture is deliberately shallow: an input layer of size V , one hidden layer of size d , an output layer of size V . No non-linearity of consequence, no depth.

The worked example in the lecture uses next-word prediction over `A cute teddy bear is reading`, with $V = 6$ and $d = 2$:

- Input `A` as one-hot `[1,0,0,0,0,0]`.
- The hidden layer produces its embedding, e.g. `[0.2, 0.9]`.
- The output layer produces a distribution over the vocabulary, e.g. `[0.2, 0.4, 0.1, 0.1, 0.1, 0.1]`, and we compare that against the true next word `cute`.
- Then slide along: input `cute` as `[0,1,0,0,0,0]`, hidden `[0.8, 0.4]`, output `[0.2, 0.2, 0.2, 0.1, 0.2, 0.1]`, target `teddy`. And so on across the corpus.

After training on enough text, the hidden-layer weights are your embedding table, and they exhibit the famous geometric regularities (vector arithmetic like `king - man + woman ≈ queen`).

Intuition — why a throwaway task produces useful vectors. The only way for a network this simple to predict context words well is to place words that appear in similar contexts at similar points in the hidden space. This is the distributional hypothesis — "you shall know a word by the company it keeps" — implemented as gradient descent. The task is a pretext; the geometry is the product. This exact logic reappears twice more in the course: BERT's masked language modelling (Lecture 2) and LLM pretraining (Lecture 4) are both "disposable proxy task, keep the representations".

1.4 Recurrent networks, and why they were abandoned

RNNs

A class of network where connections form a **temporal sequence**: the network processes one token at a time and carries a hidden state forward. In general form, at step t :

$$\begin{aligned} a_t &= g_1(W_{aa} a_{t-1} + W_{ax} x_t + b_a) \\ y_t &= g_2(W_{ya} a_t + b_y) \end{aligned}$$

The crucial property is **weight sharing across time**: the same W matrices are applied at every step, so the network can handle sequences of any length.

The lecture animates this on the running example: feed `A`, predict `cute`; carry the state; feed `cute`, predict `teddy bear`; carry the state; and so forth. Because the hidden state is a function of everything seen so far, **word order matters** — a genuine improvement over Word2vec, where a sentence is just a bag of vectors.

RNNs cover all three task shapes, using the same body with different heads: one output at the end for classification (sentiment, opinion, text source), one output per step for token labelling (tags), and a generation loop for sequence-to-sequence work (translation).

LSTM (Hochreiter & Schmidhuber, 1997)

Same recurrent skeleton, but with a **more structured hidden state**. Instead of one vector overwritten at each step, an LSTM cell maintains a cell state and uses learned **gates** to decide what to forget, what to write, and what to expose:

$\Gamma_f = \sigma(W_f[a_{t-1}, x_t] + b_f)$	forget gate
$\Gamma_u = \sigma(W_u[a_{t-1}, x_t] + b_u)$	update / input gate
$\Gamma_o = \sigma(W_o[a_{t-1}, x_t] + b_o)$	output gate
$\tilde{c}_t = \tanh(W_c[a_{t-1}, x_t] + b_c)$	candidate cell content
$c_t = \Gamma_f \odot c_{t-1} + \Gamma_u \odot \tilde{c}_t$	new cell state
$a_t = \Gamma_o \odot \tanh(c_t)$	new hidden state

Intuition. The reason gating helps is the term $\Gamma_f \odot c_{t-1}$. In a plain RNN, information from step 1 reaching step 50 must survive being multiplied by a weight matrix and squashed through a non-linearity 49 times; gradients flowing back shrink (or blow up) geometrically. That is the **vanishing gradient problem**. In an LSTM there is a nearly-linear path along the cell state, and if the forget gate stays near 1 the information passes through almost unchanged. The gate is learned, so the network decides for itself what deserves long-term storage. GRU is a simplification with two gates instead of three and no separate cell state.

The scorecard that motivates the rest of the course

Method	Pros	Cons
Word2vec (CBOW, Skip-gram)	very simple yet powerful; intuitive embeddings	word order ignored; embeddings are not context-aware
RNNs (vanilla, LSTM)	word order matters; state-of-the-art results at the time	vanishing gradients; slow computation

Intuition — the two words that killed RNNs. "Slow computations" is not a minor engineering gripe, it is the whole reason Transformers exist. An RNN's step t needs the output of step $t - 1$, so training on a sequence of length n takes n sequential steps no matter how many GPUs you own. You cannot parallelise along the sequence. When the 2010s handed the field enormous corpora and enormous GPU clusters, the bottleneck stopped being ideas and started being "how much text can you push through per second" — and there, an architecture that processes all positions simultaneously wins by an enormous margin. Note carefully that this argument is about training: at inference time, generation is still sequential for a Transformer too, which becomes the central problem of Lecture 3 and reappears in Lecture 9.

"Embeddings not context-aware" is the other half. Word2vec assigns bank one vector, so the river bank and the savings bank share a representation. Whatever replaces it must produce a different vector for the same token depending on its neighbours.

1.5 Where attention came from

Attention was **not** invented for Transformers. It was introduced by Bahdanau et al. (2014) to fix a specific failure in neural machine translation: **long-term dependencies**. A seq2seq model compressed the entire source sentence into one fixed-size vector, and by the time the decoder was emitting its fifth French word it had effectively forgotten what the English sentence said.

The lecture illustrates this with A cute teddy bear is reading → Un ours en peluche mignon ..., where the decoder gets stuck. Attention's fix: instead of one summary vector, let the decoder **look back at all source positions at every output step**, and learn which ones to weight. To produce ours (bear) it should be looking at teddy bear; to produce mignon (cute) it should be looking at cute; to produce lit (reads) it should be looking at reading.

Intuition. Note that this also solves the reordering problem for free. French puts the adjective after the noun ("ours en peluche mignon" vs "cute teddy bear"), so a strictly monotonic aligner would fail. Attention weights are just a learned soft alignment, with no ordering constraint — which is why attention maps are often readable as translation alignments.

The 2017 leap was to notice that if attention is the part doing the real work, you can **delete the recurrence entirely**. Hence the title: Attention Is All You Need.

1.6 The self-attention mechanism

Query, key, value

The mechanism is best understood by analogy to a soft dictionary lookup. Each token produces three vectors, all linear projections of its embedding x :

$$\begin{aligned} q &= x W_Q && \text{the \textbf{query}: "what am I looking for?"} \\ k &= x W_K && \text{the \textbf{key}: "what do I offer to others?"} \\ v &= x W_V && \text{the \textbf{value}: "what do I contribute if attended to?"} \end{aligned}$$

For one query, attention proceeds in four steps:

1. **Score** the query against every key by dot product: $\text{score}_j = q \cdot k_j$. A large dot product means "this key matches what I was looking for".
2. **Scale** by $\sqrt{d_k}$.
3. **Normalise** the scores into weights with a softmax, so they are positive and sum to 1.
4. **Combine**: output the weighted average of the value vectors, using those weights.

The compact matrix form, applied to all queries at once:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

where Q is $n \times d_k$, K is $n \times d_k$, V is $n \times d_v$, so $QK^\top$ is the $n \times n$ matrix of all pairwise scores, and the output is $n \times d_v$.

Intuition — what each piece is for.

Why three separate projections? Because "what I need" and "what I offer" are different questions. The token `bear` might be looking for adjectives while offering itself as a noun. One shared vector could not encode both roles. Splitting off the value is a further separation: the key decides whether a token is relevant; the value decides what gets copied if it is. A token can be easy to find but contribute something quite different from its identity.

Why divide by $\sqrt{d_k}$? If the components of q and k are roughly independent with unit variance, their dot product over d_k dimensions has variance about d_k , so it grows like $\sqrt{d_k}$ in magnitude. Feed large numbers into a softmax and it saturates: one weight goes to ~ 1 , the rest to ~ 0 , and the gradient through the softmax approaches zero. Dividing by $\sqrt{d_k}$ holds the scores in the regime where softmax is smooth and trainable. It is a variance-normalisation trick, nothing deeper.

Why does this beat recurrence? Every output position depends on every input position through a single matrix multiplication — path length 1, not n . So gradients do not have to survive n sequential steps, and the whole thing is one big matmul, which is exactly what GPUs are built for.

Watch out. V here is the value matrix, not the vocabulary size. The course reuses the letter. Context disambiguates: inside an attention formula it is values; in " V : vocabulary size" it is the vocabulary.

The cost

$QK^\top$ is an $n \times n$ matrix, so self-attention is **$O(n^2)$** in both time and memory with respect to sequence length. This single fact drives an enormous amount of later material: sparse attention and sliding windows (Lecture 2), KV caching and PagedAttention (Lecture 3), FlashAttention (Lecture 4), and even the hardware discussion in Lecture 9.

Multi-head attention

Rather than one attention operation of width d_{model} , run h of them in parallel, each with its own W_Q, W_K, W_V projecting into a smaller d_k , then concatenate the h outputs and pass them through a final projection W_O :

$$\begin{aligned}\text{head}_i &= \text{Attention}(XW_Q^i, XW_K^i, XW_V^i) \\ \text{MHA}(X) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O\end{aligned}$$

The benefit stated in the lecture: it lets the model **capture different attention features in parallel**, and the explicit comparison offered is to **multiple filters in a convolutional layer** in computer vision.

Intuition. That CNN analogy is the right one. A single conv filter detects one kind of pattern; a bank of filters detects many. Likewise one attention head can only implement one relational pattern per layer — softmax forces it to commit its probability mass. But a sentence has several simultaneous relations to track: syntactic subject-of, adjective-modifies, coreference, local phrase structure. With h heads, different heads specialise. The Transformer paper itself shows heads in layer 5 of 6 doing **anaphora resolution** — tracking what the pronoun "its" refers to — and Lecture 2 opens by displaying exactly that attention map. Note that heads are usually sized so that $h \cdot d_k = d_{\text{model}}$, meaning multi-head attention costs about the same as single-head attention of full width: you get diversity for free.

1.7 The full architecture

The original Transformer is an **encoder-decoder** model with three kinds of component.

Attention layers (MHA), in three roles:

- Encoder self-attention (encoder-encoder): source tokens attend to source tokens.
- Decoder self-attention (decoder-decoder): output tokens attend to previously generated output tokens. This one must be **masked** so that position t cannot see positions $> t$, otherwise the model would cheat at training time by reading the answer.
- Encoder-decoder cross-attention: queries come from the decoder, keys and values from the encoder output. This is the direct descendant of Bahdanau attention, and it is the only place where source information enters the decoder.

A **position-wise feed-forward network (FFNN)** after each attention layer: two linear layers with a non-linearity between them, applied identically and independently at every position, expanding to d_{FF} and back to d_{model} .

Positional encoding (PE), added to the input embeddings.

Around every sub-layer there are **residual connections** and a **normalisation layer** — both essential for training depth, and both examined in detail at the start of Lecture 2.

Walking the stack

Input. Text is tokenised, then each token is mapped to a learned embedding. Parameters: V (vocabulary size), d_{model} (embedding dimension).

Positional encoding — "input, with a trick". Attention as defined is permutation-equivariant: reorder the input tokens and you reorder the outputs identically, but nothing in the computation knows which token came first. The fix is to add position information directly to the input vectors. It can be **learned** or **hardcoded**; the goal is to let the model understand relative input position. The lecture shows the classic visualisation: the sinusoidal encoding as a heat map over (position $\times$ dimension), and a second plot showing that the dot product between two position encodings decays smoothly as the distance between the positions grows — the property that makes the encoding useful.

Encoder. N stacked identical layers. Each has self-attention, an FFNN, and normalisation. Parameters: N layers, h heads, $d_{\text{FF}}/d_{\text{key}}/d_{\text{value}}$ for sub-layer widths, d_{model} overall.

Output, "shifted right". The decoder's input is the target sequence offset by one position, so that when predicting token t the decoder has been given tokens $1 \dots t - 1$. During translation you begin with a [BOS] (beginning of sequence) token.

Decoder. N stacked layers, each with masked self-attention, then encoder-decoder cross-attention, then an FFNN, with normalisation throughout. Same parameter list as the encoder.

Output head. A linear projection from d_{model} up to V , followed by a softmax. As the lecture puts it, this is a classification problem where the classes are words.

Intuition — why "shifted right" and masking are the same idea. Both exist to enforce causality during parallel training. You want to train on the whole target sentence in one forward pass, computing the loss at every position simultaneously — but position t 's prediction must not depend on the true token at position t or later. Shifting the input right by one handles the "not itself" part; the causal mask on decoder self-attention handles the "not the future" part. At inference time neither is needed as a trick, because you genuinely do not have future tokens: you feed back what you generated.

Two computational tricks from the paper

Multi-head attention (above): run several attention layers in parallel to capture different features simultaneously; analogous to multiple CNN filters.

Label smoothing. Borrowed from a 2015 vision paper whose message was overconfidence is bad. Instead of a one-hot target (probability 1 on the correct token, 0 elsewhere), use a slightly softened target — e.g. $1 - \epsilon$ on the correct token and ϵ spread over the rest. It prevents overfitting and, in the Transformer paper, improved both accuracy and BLEU.

Intuition. A one-hot target instructs the model to drive the correct logit to $+\infty$ relative to all others — a target it can never reach, so it just keeps sharpening, producing enormous logits and brittle overconfidence. Since language is genuinely ambiguous (several next words are often fine), a target that says "mostly this, but leave a little mass elsewhere" is closer to the truth. Interesting wrinkle noted in the paper: label smoothing hurts perplexity — the model is deliberately less certain than it could be — while helping BLEU. A useful early lesson that the loss you optimise and the metric you care about are not the same thing.

1.8 End-to-end worked example

The lecture closes by pushing `A cute teddy bear is reading.` through the whole machine to produce `Un ours en peluche mignon lit.` It is worth following step by step, because it is the only place where every component is seen in sequence.

Encoder side.

1. **Tokenise:** `A | cute | teddy | bear | is | reading | .`
2. **Add special tokens:** `[BOS] A cute teddy bear is reading . [EOS]`
3. **Embed** each token $\rightarrow$ one vector per token.
4. **Add the position embedding** to each $\rightarrow$ position-aware embeddings.
5. **Stack** them into a matrix (one row per token, d_{model} columns).
6. Feed that matrix into the encoder. Multiply it by W_Q , W_K , W_V to get Q , K , V .
7. Compute $QK^\top$: an $n \times n$ grid where entry (i, j) is how much token i attends to token j . Softmax each row.
8. Multiply by V : each output row is a **weighted average of value vectors, with weights determined by the query-key match**. That sentence is the whole mechanism in one line.
9. This happens h times in parallel; concatenate the heads and project through W_O .

10. Pass through the feed-forward network. Output: **context-aware encoded embeddings** — one vector per input token, each now informed by the whole sentence.
11. Repeat for all N encoder layers.

Decoder side — now generating, one token at a time.

1. Start the decoder with `[BOS]`.
2. Decoder self-attention over what has been generated so far.
3. Encoder-decoder attention: the decoder queries the encoder's output.
4. Feed-forward network.
5. Linear projection to vocabulary size, then softmax $\rightarrow [0.001, 0.0003, \dots, 0.4, \dots, 0.002]$.
6. Pick the highest-probability token: `Un`.
7. Append it and repeat. Decoder input `[BOS] Un` $\rightarrow$ predicts `ours`. Then `[BOS] Un ours` $\rightarrow$ `en`, then `peluche`, `mignon`, `lit`, and finally `[EOS]`, at which point generation stops.

Final output: `Un ours en peluche mignon lit`.

Intuition — the one thing to take away. Step 8 is the heart of it. Every Transformer layer does the same thing: each position replaces its own vector with a weighted mixture of information gathered from other positions, where the weights are computed from content, not from position. Stack that operation N times, with a feed-forward network between rounds to reshape the mixed information, and you have the architecture that everything in the next eight lectures is built on.

Lecture 2 — Transformer-based Models & Tricks

Video length: 1h47. Covered here: why position information needs a better treatment than "add a vector at the input", learned and sinusoidal encodings, relative-position biases (T5, ALiBi), RoPE in detail, layer normalisation and Pre-Norm vs Post-Norm vs RMSNorm, attention approximations (Longformer, sliding windows, MQA/GQA), the three families of Transformer-based model, and a deep dive on BERT including distillation, DistilBERT and RoBERTa.

The lecture opens by revisiting an attention map from the original Transformer paper: two heads in layer 5 of 6 that appear to be doing **anaphora resolution**, showing the attention emitted by the single word `its`. It is a concrete demonstration that heads specialise, which was the claim made in Lecture 1.

2.1 Position information, done properly

The problem restated

Attention creates direct links between all pairs of tokens, and in doing so it **loses position information entirely**. `bear teddy cute a and a cute teddy bear` produce the same set of pairwise interactions.

Attempt 1: learned absolute position embeddings

Add a learned, position-specific vector to each token embedding. Position 0 has its own trainable vector, position 1 has another, and so on.

Limitation: you must fix a maximum length up front, and you **need to retrain for longer sequences**. Position 4096 has no embedding if you only ever trained up to 2048, and there is no principled way to invent one.

Attempt 2: hardcoded sinusoidal position embeddings

Instead of learning the vectors, compute them:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

Even dimensions get sines, odd dimensions get cosines, and the wavelength grows geometrically with the dimension index i — fast oscillation in early dimensions, very slow oscillation in later ones.

The lecture shows two plots: the **values of the embeddings** as a heat map over (position $\times$ dimension), and the **similarity between positions**, i.e. the dot product $\text{PE}(\text{pos}_a) \cdot \text{PE}(\text{pos}_b)$, which peaks on the diagonal and decays smoothly away from it.

Benefit: it **extends to any sequence length** — the formula is defined for every pos , so nothing needs retraining.

Intuition — why sinusoids of many frequencies? Think of it as a binary counter made continuous. In binary, the lowest bit flips every step, the next every two steps, the next every four; together, a handful of bits uniquely identify a large range of integers. The sinusoidal encoding does the same with continuous-valued "bits" at geometrically spaced frequencies. Two properties follow. First, uniqueness: no two positions within a huge range share an encoding. Second — and this is the important one — $\text{PE}(\text{pos} + k)$ is a fixed linear function of $\text{PE}(\text{pos})$ for any offset k , because shifting a sinusoid is a rotation in its (sin, cos) plane. So a linear layer can in principle learn to compute relative offsets from these absolute encodings. The slow smooth decay of the similarity plot is what lets the model perceive "nearby" versus "far".

The conceptual shift: we actually want relative position

The lecture then makes the key argument. Language cares about relative positions, not absolute ones: the relationship between an adjective and the noun it modifies is "immediately before", regardless of whether that happens at position 3 or position 3,000. And since the place where positions actually interact is the attention layer, the natural conclusion is: **let's change the attention layer instead** of doctoring the inputs.

Approach A: additive bias on the query-key scores

Add a bias term to the score matrix before the softmax:

$$\text{score}_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}} + b(i - j)$$

Two well-known instances:

- **T5 bias** (Raffel et al.) — the bias is **learned per head**, with relative distances bucketed (so that distances 1, 2, 3 get their own buckets while distances 100–127 share one). Each head learns its own preferred distance profile.
- **ALiBi** (Press et al., 2021, Train Short, Test Long) — the bias is **linear, deterministic, and not bounded**: $b(i - j) = -m |i - j|$, with a different slope m per head. Nothing is learned; a token is penalised in proportion to how far away it is, and different heads have different "attention spans" set by their slopes.

Intuition. ALiBi is essentially a learned-free, hard-coded recency prior. The title of the paper is the selling point: because the penalty is a simple function of distance with no parameters and no ceiling, a model trained on length 1024 still behaves sensibly at length 4096 — the bias just keeps getting more negative, which is exactly what you want. Steep-slope heads become local pattern detectors, shallow-slope heads keep a global view. Contrast with T5's learned buckets, which have nothing sensible to say about distances larger than the largest bucket seen in training.

Approach B (the modern default): RoPE — Rotary Position Embeddings

Su et al., 2021 (RoFormer). The idea: instead of adding anything, **rotate the query and key vectors** by an angle proportional to their position.

In two dimensions, rotating a vector by angle $m\theta$ (where m is the position):

$$R(m\theta) = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix}$$

and we set $q_m \leftarrow R(m\theta) q_m$, $k_n \leftarrow R(n\theta) k_n$.

Extension to $d > 2$: split the d -dimensional vector into $d/2$ consecutive pairs of coordinates and **rotate every block of dimension 2**, each block with its own frequency θ_i (again geometrically spaced, as in the sinusoidal scheme).

Why this is elegant — the relative-distance property. Because rotations compose by adding angles, and because a rotation preserves dot products:

$$(R(m\theta) q)^\top (R(n\theta) k) = q^\top R((n-m)\theta) k$$

The attention score between positions m and n depends **only on $n - m$** . You inject absolute positions into the vectors, and the attention mechanism automatically sees only relative distance. No bias table, no extra parameters, nothing added to the residual stream.

Long-term decay. The lecture also shows that the relative upper bound on the attention weight decays as $|n - m|$ grows: as the many frequency components drift out of phase with each other, distant pairs tend towards lower scores. So RoPE bakes in a mild, automatic recency prior too.

Intuition — the clock analogy. Give every position a set of clock hands spinning at different speeds, one pair of hands per pair of dimensions. A query and a key "match" strongly when their hands line up. Since each hand's angle is position $\times$ its own speed, the alignment between two tokens depends only on how many steps apart they are — like reading the phase difference between two clocks rather than the absolute time on either. The reason RoPE is the default in essentially every modern open LLM (LLaMA, Mistral, Qwen, Gemma, DeepSeek) is that it adds zero parameters, gives exact relative behaviour, does not touch the residual stream, and — importantly for Lecture 3 — is compatible with KV caching, because you rotate each key once when you compute it and it stays valid forever.

Watch out — RoPE is applied to q and k only, never to v . The position information belongs in the matching computation, not in the content being copied. If you rotated the values you would be corrupting the payload.

2.2 Layer normalisation

The **Add & Norm** boxes in the architecture diagram are LN, layer normalisation (Ba et al., 2016). For a vector x of dimension d :

$$\mu = \frac{1}{d} \sum_i x_i \quad \sigma^2 = \frac{1}{d} \sum_i (x_i - \mu)^2$$

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$$

where γ and β are learned per-dimension scale and shift, and ε is a small constant for numerical safety. **Benefit: helps training stability and convergence.**

Intuition. Note what is being normalised: for each token independently, across its own feature dimensions. This is different from batch normalisation, which normalises each feature across the batch — a poor fit for text, where sequences have different lengths and batch statistics are noisy. LayerNorm has no dependence on batch size at all, which is why it is the choice here. Mechanically, it keeps the activations entering each sub-layer at a consistent scale, so no layer receives inputs whose magnitude has drifted; that consistency is what keeps gradients well-behaved in deep stacks.

Post-Norm vs Pre-Norm

Where you put the normalisation relative to the residual connection matters enormously (Xiong et al., 2020).

Post-Norm — the original 2017 arrangement, normalisation after the residual addition:

$$x \leftarrow \text{LN}(x + \text{Sublayer}(x))$$

Pre-Norm — normalisation before the sub-layer, with the residual added afterwards, unnormalised:

$$x \leftarrow x + \text{Sublayer}(\text{LN}(x))$$

Nowadays: Pre-Norm. And, additionally, **RMSNorm** (Zhang et al., 2019) instead of full LayerNorm:

$$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_i x_i^2} \quad \text{RMSNorm}(x) = \gamma \odot \frac{x}{\text{RMS}(x)}$$

i.e. drop the mean subtraction and the shift β ; only rescale by the root mean square. So the modern default is **Pre-Norm + RMSNorm**.

Intuition — why Pre-Norm won. In the Post-Norm arrangement, every residual stream passes through a normalisation on its way up the stack, so the "clean" identity path from input to output is repeatedly rescaled. Gradients flowing back get multiplied by the Jacobian of each LayerNorm, and in deep models this makes the early layers receive badly-scaled gradients. The practical consequence is that Post-Norm Transformers need a careful **learning-rate warmup** to train at all. Pre-Norm leaves an unobstructed additive highway from input to output — every sub-layer's contribution is added to a stream nothing ever renormalises — so gradients reach the bottom intact and warmup becomes optional. The cost is that the residual stream's magnitude grows with depth, which is why a final normalisation is added before the output head.

RMSNorm is a pure efficiency win: subtracting the mean turns out to contribute almost nothing to the benefit, while costing an extra pass over the vector and extra parameters. Removing it is measurably faster at essentially no quality cost — the kind of small win that matters when you multiply it by every layer of every forward pass of a trillion-token training run.

2.3 Making attention cheaper

All of this exists because of the $O(n^2)$ cost established in Lecture 1.

Sparse attention: Longformer (Beltagy et al., 2020)

Full attention is an $n \times n$ grid where every cell is computed. Longformer's observation is that most of those cells are near-worthless: language is mostly local. So restrict which cells you compute:

- **Sliding-window attention (SWA):** each token attends only to a window of w neighbours on each side. Cost drops from $O(n^2)$ to $O(n \cdot w)$, linear in n .
- **Global attention on selected tokens:** a small number of designated tokens ([CLS] , or task-specific ones such as the question tokens in QA) attend to everything and are attended to by everything.

Variations include interleaving local and global attention layers — some layers local, some global, rather than mixing both patterns in every layer.

Mistral 7B (2023) used sliding-window attention in production, and the lecture makes the illuminating comparison to the **receptive field** in convolutional networks.

Intuition — the receptive-field argument, which is the crux. A single sliding-window layer with window w lets a token see w neighbours. Stack two such layers and its effective reach is $2w$, since its neighbours have themselves already gathered from their neighbours. After L layers the receptive field is $L \cdot w$. With $w = 4096$ and 32 layers, information can travel over 100k tokens — while every individual attention computation stays cheap and linear. This is exactly how CNNs see whole images through small 3×3 filters. The trade-off is real, though: information from far away arrives compressed, having been repeatedly averaged, whereas full attention gives every token a direct, uncompressed link to every other. That is why the [CLS]-style global tokens are kept — they act as an express lane through the network.

Sharing attention heads: MHA → GQA → MQA

A different axis of saving. In vanilla **multi-head attention (MHA)**:

$$\# \text{query heads} = \# \text{key heads} = \# \text{value heads} = h$$

The idea is to **share key/value heads within groups of queries**:

- **MHA** — h query heads, h key heads, h value heads. Maximum expressive power, maximum memory.
- **GQA (Grouped-Query Attention)** — h query heads, but only G key heads and G value heads, with $G < h$. Query heads are partitioned into G groups; all queries in a group share one K/V pair.
- **MQA (Multi-Query Attention)** — the extreme: h query heads, **1** key head and **1** value head. Every query head shares the same keys and values.

So $\# \text{query} = h$, $\# \text{key} = \# \text{value} = G$, with $G = h$ recovering MHA and $G = 1$ recovering MQA.

Intuition — why this is worth doing. The saving is not really in FLOPs, it is in the **KV cache** (Lecture 3). During generation you must keep, for every layer and every head, the keys and values of every token generated so far. That cache scales as $2 \cdot n \cdot h \cdot d_{\text{head}} \cdot N_{\text{layers}}$, and for long conversations it becomes the single largest consumer of GPU memory — often larger than the model weights. Cutting the key/value head count by $8\times$ cuts the cache by $8\times$, which means you can serve $8\times$ more concurrent users on the same hardware, or support $8\times$ longer contexts. MQA is the cheapest but measurably degrades quality; GQA with $G = 8$ was found to keep essentially all the quality at most of the saving, which is why it is the standard choice today (LLaMA 2/3, Mistral). This material is repeated in Lecture 3 under inference optimisation, which tells you how central it is.

2.4 The three families of Transformer-based model

This taxonomy is the organising idea of the rest of the course.

Family	What it does	Examples	Era
Encoder-decoder	text to text	T5, mT5, ByT5	—
Encoder-only	project the embedding for class prediction (e.g. sentiment extraction)	BERT, DistilBERT, RoBERTa	popular ~2018-2022
Decoder-only	text to text	GPT series	popular now

Encoder-only means: keep the left half of the diagram, and read out the encoded representations. Decoder-only means: keep the right half, drop cross-attention (there is no encoder to attend to), and just do causal language modelling.

Intuition — why decoder-only won, given that encoder-decoder came first. Three reasons, and they build on each other.

First, unification. A decoder-only model has exactly one interface — "continue this text" — and every task can be expressed in it. Translation is "English: ... French: ...". Classification is "Review: ... Sentiment: ...". With an encoder-only model you must attach and finetune a task-specific head for every new task.

Second, training efficiency per token. Causal language modelling extracts a prediction target from **every** position of every training sequence. BERT's masked language modelling only gets signal from the $\sim 15\%$ of positions it masks. When the bottleneck is how much you can learn per unit of compute, that is a large constant factor.

Third, scaling and emergence. Simply making causal LMs bigger produced in-context learning (Lecture 3) — the ability to pick up a task from examples in the prompt, with no gradient updates at all. That capability made the finetune-per-task workflow largely unnecessary.

Encoder-only models are not obsolete, though. They remain the right tool whenever you need a vector, not text: retrieval embeddings, reranking, and classification at high volume and low latency. Lecture 7's RAG pipeline runs on BERT-family bi-encoders and cross-encoders, and Lecture 5's reward models can be built the same way. That is precisely why BERT gets a deep dive here.

2.5 BERT deep dive

BERT = Bidirectional Encoder Representations from Transformers (Devlin et al., 2018).

Why "bidirectional" is the operative word

An encoder's self-attention is unmasked: every token sees every other token, both to its left and to its right. That is **bidirectional**. A decoder's self-attention is causal-masked: a token sees only its past. That is **not bidirectional**, and the lecture flags it explicitly with a warning marker.

The naming is a deliberate jab at **ELMo** (Peters et al., Feb 2018), which achieved "bidirectional" context by running two separate unidirectional LSTMs — one left-to-right, one right-to-left — and concatenating them. Each direction never saw the other while computing. BERT's attention is jointly bidirectional in a single pass. (And yes, ELMo and BERT are both Sesame Street characters; BERT's paper came out in October 2018, eight months after ELMo's, and the naming was not an accident.)

The two-step strategy

- **Step 1 — Pretraining** with proxy tasks: MLM and NSP, on unlabelled text.
- **Step 2 — Finetuning** for a given end task, on a small labelled dataset.

Pros	Cons
finetuning does not need much data	not suited to a range of tasks (e.g. text generation)
good performance	finetuning is a required step

Variants: RoBERTa, DistilBERT, ALBERT.

Architecture, versus the original Transformer

BERT is the encoder stack only. No decoder, no cross-attention, no causal mask. On top of the final encoded embeddings you attach whatever head your task needs.

Input processing

WordPiece tokenisation. The tokeniser is trained on a corpus beforehand; vocabulary size around **30,000**; it is great at detecting common particles (the frequent prefixes and suffixes get their own tokens).

Special tokens, for the NSP/MLM setup:

- **[CLS]** at the very beginning of the input.
- **[SEP]** separating consecutive segments, plus one at the end.
- **[MASK]** to hide inputs during MLM.
- **[PAD]** to fill a batch to uniform length.

Input embeddings — three of them, summed

1. **Token embedding** — a gigantic lookup table, one learned vector per vocabulary entry.
2. **Positional encoding** — learned or fixed sines/cosines, as in Lecture 1.
3. **Segment encoding (new in BERT!)** — a shared embedding identifying which segment a token belongs to: all of segment A gets one vector, all of segment B gets another.

The three are added elementwise to form the input to the encoder stack.

Intuition. Segment embeddings exist because BERT is frequently fed pairs of texts — two sentences for NSP, a question and a passage for QA, a premise and a hypothesis for entailment. [SEP] marks the boundary, but a single boundary token is a weak signal for a model that mixes everything through attention; giving every token an explicit "I am in the second text" vector makes the distinction available everywhere at once.

Proxy task 1: Masked Language Modelling (MLM)

15% of input tokens are selected for prediction. Of those selected:

- **80%** are replaced with [MASK] ,
- **10%** are replaced with a **random word**,
- **10%** are **left unchanged**.

The model must predict the original token at every selected position.

Benefits: the network learns language modelling from contextual information (both directions), and the randomisation acts as regularisation that **reflects the probabilistic nature of language**.

Intuition — the famous 80/10/10 split, which confuses everyone. The problem being solved is a train/inference mismatch. If masked positions were always [MASK] , the model would learn "predict a word only when you see the [MASK] token" — but at finetuning and inference time [MASK] never appears, so the representations of ordinary tokens would be under-trained. The 10% random-word substitution forces the model to always keep a prediction of what each position should contain, since any token might secretly be corrupted; this is what makes it robust rather than lazily copying. The 10% unchanged case forces it to keep predicting even when the input looks perfectly fine — otherwise it could learn "if it isn't [MASK] or obviously wrong, just copy the input". Together, the three cases mean every position's representation must encode a genuine contextual prediction.

Proxy task 2: Next Sentence Prediction (NSP)

Pick two sentences from the corpus. **50% of the time they genuinely follow each other; 50% of the time they do not.** Task: binary classification — do they follow?

Benefits: the network implicitly learns to detect useful contextual information across a sentence boundary, and it is an **easy classification task that requires no labels** (the labels are free, from the corpus structure).

The prediction is read off the [CLS] token's final representation.

Hyperparameters and model sizes

Model:

- L — number of layers,
- H — hidden layer size, i.e. the embedding dimension,
- A — number of attention heads operating in parallel.

Data:

- language-specific vs multilingual — which languages it was trained on,
- cased vs uncased — whether inputs are lowercased.

Model	L	H	A	Parameters
BERT-Tiny	2	128	2	4M
BERT-Mini	4	256	4	11M
BERT-Small	4	512	8	30M
BERT-Medium	8	512	8	42M
BERT-Base	12	768	12	110M
BERT-Large	24	1024	16	340M

Watch out. Compare 340M for BERT-Large against the hundreds of billions in Lecture 3. In 2018 BERT-Large was considered enormous. Keeping this table in mind gives you a visceral sense of how fast the scale moved.

Finetuning

Goal: leverage the embeddings BERT learned for a "sister" task.

Tricks:

- start from the weights of the massively pretrained model,
- **freezing early layers** sometimes gives a better complexity/performance trade-off,
- great results are possible with **minimal labelled data**, depending on how complex the task is and how close it is to the pretraining data distribution and objectives.

Use cases:

- sequence classification — one label for the whole input, e.g. sentiment extraction,
- token classification — one label per token, e.g. extractive question answering (predicting answer start and end positions).

Worked finetuning example: sentiment extraction

Input: This teddy bear is SO CUTE!

1. **Lowercase** (uncased model): this teddy bear is so cute!
2. **Tokenise** with WordPiece: this | teddy | bear | is | so | cute | !

3. **Prepend [CLS]** — the lecture's phrasing is that it acts as a placeholder for the sentiment.
4. **Append [SEP]**, then **[PAD]** out to the fixed sequence length: [CLS] this teddy bear is so cute ! [SEP] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
5. **Add the token embedding**, then the **position embedding** (indices 0,1,2,...,14), then the **segment embedding** (here segment A for the real text and B for the padding region).
6. Feed the resulting matrix through the **pretrained BERT encoder**.
7. Take the final-layer vector at the [CLS] position, pass it through a small **feed-forward network**, and read off the sentiment class.

Intuition — why [CLS] works as a sentence summary. [CLS] carries no lexical meaning of its own, so its representation is free to become whatever the pooling operation needs to be. Through unmasked self-attention it can gather from every token in the sequence, and because NSP pretraining already forced it to encode a whole-sequence property, it starts finetuning as a reasonable summary vector rather than from scratch. It is a learned pooling token: strictly more flexible than mean-pooling, because attention weights which tokens matter.

Takeaways and shortcomings

Benefits: state-of-the-art results; approximately true contextual representation of words (the same token gets different vectors in different sentences — the thing Word2vec could not do); adaptable to many classification tasks. **Widely used in industry for anything related to encoding.**

Limitations:

- context window size is limited,
- computationally expensive — a hard sell for low-latency or cost-sensitive applications,
- the training paradigm is complex: MLM + NSP pretraining and then finetuning.

2.6 Distillation

"The soft targets contain almost all the knowledge." — Hinton et al., 2014 ("Dark knowledge").

The setup: a large, already-trained **teacher** model and a smaller **student**. Rather than training the student on the hard one-hot labels, train it to match the teacher's full output probability distribution.

Intuition — why soft targets carry more information. Suppose the correct label is "teddy bear". A hard label tells the student one bit of structure: class 7 is right. The teacher's soft distribution says something much richer — "85% teddy bear, 10% stuffed animal, 4% toy, 0.001% bulldozer". That encodes the teacher's entire learned similarity structure over the label space, in every single training example. The student learns not just the answer but the shape of the confusion, which is why distillation can transfer performance that the student would never reach training on labels alone. This is the "dark knowledge": information present in the teacher's function that the training labels never contained. Lecture 6 will use the word "distillation" for something related but different — training a small model on a big model's generated reasoning traces — and the lecture explicitly contrasts the two.

DistilBERT (Sanh et al., 2019)

Distil BERT-Base ($N = 12$ layers) down to $N = 6$ **layers**. Result: **$\sim 1.6\times$ faster** while retaining **$\sim 97\%$ of the performance**.

RoBERTa (Liu et al., 2019)

Goal: a thorough analysis of the available directions of optimisation — what in BERT's recipe actually mattered?

Modelling findings:

- **Removing NSP and segment encodings: approximately no effect!** (DistilBERT had already dropped it.)
- **Static $\rightarrow$ dynamic masking** across epochs: rather than masking each sequence once during preprocessing and reusing that mask every epoch, generate a fresh mask each time the example is seen.

Data findings:

- **richer:** pretraining corpus from **16 GB $\rightarrow$ 160 GB**,
- **for longer:** 1M steps at batch size 256, versus 500k steps at batch size 8k (a much larger total token count).

Result: +4% across benchmarks with the same architecture.

Intuition — the real lesson of RoBERTa, and it is a big one. BERT's paper presented MLM+NSP as an integrated design, and the community assumed both were load-bearing. RoBERTa shows one of them was not, and that most of BERT's headroom was undertraining: same architecture, more data, longer schedule, +4%. This is the first appearance in the course of a theme that dominates Lecture 4 — that for a given architecture, the amount of data and compute you pour in often matters more than the architectural cleverness. The Chinchilla result is exactly this observation, made quantitative.

Why did NSP not matter? Plausibly because predicting whether two sentences are adjacent is too easy — the model can often decide from topic overlap alone, without learning anything about discourse structure. An objective that provides no gradient signal after the first few thousand steps contributes nothing. This is worth remembering when you design your own proxy tasks: a task must be hard enough to force the representation you want.

Lecture 3 — Large Language Models

Video length: 1h49. Covered here: what "LLM" actually means, mixture of experts, how a token is chosen at inference (greedy, beam search, sampling, top-k, top-p, temperature, guided decoding), context length and context rot, prompt structure, in-context learning, chain of thought, self-consistency, and then the full catalogue of inference optimisations (KV caching, GQA, PagedAttention, multi-head latent attention, speculative decoding, multi-token prediction).

3.1 What an LLM is

LLM = Large Language Model. Taking the words in order:

"Language model" — a statistical or machine learning model that assigns probabilities to sequences of tokens. Nothing more. Given a sequence, it tells you how likely that sequence is; equivalently, given a prefix, it gives you a distribution over what comes next.

"Large" — quantified in the lecture as:

- **model size:** billions of parameters or more,
- **training data:** hundreds of billions of tokens or more,
- **compute:** a lot of GPUs.

Architecture: a **decoder-only Transformer-based model**. Examples: the GPT series, LLaMA, Gemma, DeepSeek, Mistral, Qwen.

Intuition — take the definition seriously, because it explains the failure modes. An LLM is a probability distribution over token sequences, fitted to a corpus. It is not a database, not a reasoner, not a fact-checker. Everything it does well and everything it does badly follows from this. It hallucinates because a fluent-but-false continuation can be more probable than an accurate one, given what the corpus rewarded. It has a knowledge cutoff because the distribution was fitted at one moment in time. It is sycophantic when agreement is the more probable continuation. Lectures 5 through 8 are, in a real sense, nine hours of engineering around the consequences of this one definition.

3.2 Mixture of Experts (MoE)

The motivation

A very large dense model applies **all** of its weights to **every** input token. The observation that starts MoE: **not all weights are useful in the forward pass** for a given token. The word "the" does not need the same computation as a snippet of Rust.

So: replace one huge block with n **experts** $E_1, \dots, E_n$ plus a small **gating network** G . For input x , the gate produces weights over experts, and the output is a combination of expert outputs.

- **Dense MoE** — the output is the weighted average of **all** expert outputs, with weights like `[0.1, 0.8, 0.05, ...]` from the gate.
- **Sparse MoE** — the output is the weighted average of only the **selected** expert outputs, chosen by **top-k selection** (Shazeer et al., 2017, Outrageously Large Neural Networks).

Sparse is the interesting one, because only k experts actually run.

Where MoE goes in a Transformer

The FFNN sub-layer is replaced by the MoE block. So instead of one feed-forward network per layer you have $\text{FFNN}_1 \dots \text{FFNN}_n$ plus a gate G . And critically: **routing is done for each token**, independently — not per sequence, not per batch. Token 5 may go to experts 2 and 17 while token 6 goes to experts 1 and 9.

Intuition — why replace the FFNN rather than attention? Two reasons. The FFNN is where most of the parameters live (with $d_{\text{FF}} = 4 d_{\text{model}}$, the two FFNN matrices hold roughly twice the parameters of all four attention projections combined), so that is where sparsity buys the most. And the FFNN is already position-independent — it acts on each token separately — so routing per token requires no change to the computational structure. Attention, by contrast, is inherently about interactions between positions; you cannot route it per token without breaking what it does.

The economics of MoE, stated plainly. Model quality scales with total parameter count. Inference cost scales with active parameters per token. A dense model forces these to be equal. MoE decouples them. DeepSeek-V3 (seen in Lecture 6) has **~671B total parameters but only ~37B active** per token — so it holds the knowledge of a 671B model at roughly the serving cost of a 37B one. You still pay the full 671B in GPU memory, since any expert might be needed, which is why MoE is a compute-and-latency win rather than a memory-and-cost win across the board.

Training challenge: routing collapse

Symptom: the same expert gets selected most of the time. The routing distribution collapses onto a few experts and the rest never train.

Remedy: force the other experts to be "part of the game" via an **auxiliary loss** (Switch Transformers, Fedus et al., 2021):

$$\mathcal{L}_{\text{aux}} = \alpha \cdot n \cdot \sum_i f_i P_i$$

where f_i is the **fraction of tokens routed to expert i** and P_i is the **average routing probability for expert i** .

Intuition — why that particular product. The sum $\sum_i f_i P_i$ is minimised, subject to both vectors summing to 1, when the load is spread uniformly; it blows up when one expert takes both a large share of tokens and a high average probability. Multiplying the two is the trick: f_i is a hard count and has no gradient, while P_i is differentiable. So the gradient flows into the gate through P_i , scaled by how overloaded that expert currently is — the more tokens expert i is hogging, the harder the loss pushes the gate's probability for it down.

Why does collapse happen at all? A classic rich-get-richer loop. An expert that is picked slightly more often trains slightly faster, becomes slightly better, and is therefore picked more often still. Without an explicit counterweight you end up with a very expensive model that behaves like a small dense one. There is also a hard systems reason for balance: experts are distributed across GPUs, and if one GPU receives most of the tokens, every other GPU sits idle waiting for it.

Do experts specialise interpretably?

The lecture shows the analysis from **Mixtral of Experts** (Jiang et al., 2024), colouring tokens by which expert handled them. The honest answer from that paper is that specialisation is **not** cleanly semantic — experts do not divide into "the maths expert" and "the French expert". The observed structure is more syntactic and positional, with some tendency for consecutive tokens to go to the same expert.

Intuition. This is a useful expectation to hold. MoE is a capacity trick — a way to hold more parameters without paying for all of them — not an interpretability tool. Do not expect to open up a MoE and find human-legible modules.

3.3 Generating a response

Next-token prediction, as a loop

The model is fed a prefix and produces one token; that token is appended and the whole thing is fed back:

[BOS]	→ A
[BOS] A	→ teddy
[BOS] A teddy	→ bear
[BOS] A teddy bear	→ is
[BOS] A teddy bear is	→ ?

Mechanically, at each step: run the decoder stack, take the final-layer vector at the **last** position, project it to vocabulary size V with the output matrix to get **logits**, and turn logits into probabilities.

Where the probabilities come from — softmax with temperature

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

where z are the logits and T is the **temperature**.

Impact of temperature:

- **Small T** — dividing by a small number magnifies the differences between logits, so the distribution becomes **sharper**; in the limit $T \rightarrow 0$ it becomes a point mass on the argmax (equivalent to greedy decoding).
- **High T** — differences are flattened, the distribution becomes more **uniform**; in the limit $T \rightarrow \infty$ you sample uniformly at random from the vocabulary.

Intuition. Temperature does not change the ranking of tokens, only how much probability mass the leaders keep. $T < 1$ for tasks with one right answer (extraction, classification, code, LLM-as-a-judge in Lecture 8); T around 1 for creative writing. T above about 1.2 usually produces incoherence, because the tail of a V -sized vocabulary contains an enormous amount of nonsense and flattening the distribution hands it real probability. The suggested reading "Defeating Non-determinism in LLM Inference" (He et al., 2025) makes the further point that even $T = 0$ is not truly deterministic in practice — batching and floating-point reduction order change results run to run.

Decoding strategy 1: greedy decoding

Take the token with the highest predicted probability, every time.

Limitations: the output is not optimal, not natural, and not diverse. Two distinct failures. Not optimal, because the highest-probability first token does not necessarily begin the highest-probability sequence — a locally greedy choice can walk you into a dead end. Not natural, because real human text is not the most probable text; greedy decoding produces flat, repetitive prose and often gets stuck in loops.

Decoding strategy 2: beam search

Keep the k paths that are the most likely. At each step, expand every one of the k surviving beams by every possible next token, score the resulting sequences by cumulative probability, and keep the best k .

The lecture's worked example with $k = 3$: from [BOS] the candidates are a, cute, the; expanding cute gives cute teddy (0.7), cute bear, cute fluffy (0.2), and so on; the search continues until beams terminate at [EOS].

Limitations: needs more computation, and **lacks diversity/creativity** — the k beams tend to be near-identical variations of one another, differing by a word.

Intuition. Beam search dominates machine translation, where there really is a single best answer and you want the highest-probability sequence. It is almost never used for open-ended generation, because "most probable" and "interesting" are close to opposites for creative text. There is also a well-known pathology: longer sequences have lower cumulative probability (you keep multiplying by numbers below 1), so beam search is biased towards short outputs unless you apply a length normalisation.

Decoding strategy 3: sampling

Sample the next token from the probability distribution rather than taking the maximum. This restores diversity — but naive sampling from the full distribution occasionally draws

a genuinely terrible token from the long tail, and one bad token derails everything that follows. Hence two truncation strategies:

- **Top-k** — sample among the k most probable tokens only (e.g. $k = 4$), renormalising over them.
- **Top-p (nucleus)** — sample from the **smallest set of tokens whose cumulative probability is $\geq p$** (e.g. $p = 90\%$).

Intuition — why top-p is generally preferred over top-k. The right number of plausible next tokens varies enormously with context. After "The capital of France is" there is essentially one; after "She opened the door and saw" there are thousands. Top-k with a fixed k is either too permissive in the first case (admitting three wrong countries) or too restrictive in the second (arbitrarily cutting off good options). Top-p adapts automatically: it takes few tokens when the model is confident and many when it is not, because it is defined on the probability mass rather than the count. In practice the two are often combined, with top-k as a hard safety ceiling.

Guided (constrained) decoding

Motivation: you need output in a specific format. Given the prompt "Generate a description of my 33-year-old teddy bear who likes reading. Do this in JSON format.", you want exactly:

```
{
  "first_name": "teddy",
  "last_name": "bear",
  "age": 33,
  "hobby": "reading"
}
```

Idea: only allow "valid" next tokens. At each step you know, from a grammar or schema, which tokens could legally come next; you mask the logits of all others to $-\infty$ before the softmax, so they cannot be sampled at all.

The lecture's step-by-step: from [BOS] the candidates might be {, the, } — only { is legal for JSON, so it is forced. Having emitted {, the candidates are road, sun, "first_name" — only a quoted key is legal. After the key, candidates , , : , to — only : is legal. After the colon, 10, "Jane", "teddy" — now several are legal (any string), so the model genuinely chooses. And so on.

Intuition — this is a hard guarantee, not a suggestion. The difference between "please reply in JSON" in the prompt and guided decoding is the difference between a request and an enforcement. Asking nicely works most of the time and fails unpredictably — an apologetic preamble, a trailing comma, a markdown fence — which is exactly the kind of failure that breaks a production pipeline at 3am. Constrained decoding makes malformed output impossible, because invalid tokens are never candidates. Note that the model's content is still its own; you are only restricting the shape. Lecture 8 uses this directly to make LLM-as-a-judge outputs parseable, and it is what powers the "structured output" APIs from OpenAI, Anthropic and Google.

3.4 Prompting

Context length

Context length (also context size, window size) is the maximum number of tokens the model can attend over — prompt plus generated output together.

The lecture notes that **orders of magnitude vary by input type and by model**, and then delivers the important caveat: **beware of "context rot"** (Hong et al., 2025, Context Rot: How Increasing Input Tokens Impacts LLM Performance).

Intuition — the advertised window is not the usable window. A model advertising a 1M-token context can accept a million tokens; that is a statement about the position encoding and the memory budget, not about comprehension. Measured performance degrades well before the limit, and it degrades non-uniformly: information at the very start and very end of a long context is retrieved much more reliably than information in the middle (the "lost in the middle" effect, which Lecture 7 revisits under needle-in-a-haystack testing). The practical rule is that a short, curated context beats a long, padded one — which is the entire justification for retrieval in Lecture 7. Stuffing everything you have into a long window is both more expensive and less accurate.

The structure of a good prompt

The lecture decomposes a prompt into four parts:

- **Context** — "My teddy bear had a long day and needs a bedtime story."
- **Instructions** — "Generate a bedtime story that takes place in a specific location."
- **Input** — "Location: Country of teddy bears"
- **Constraints** — "The story needs to be suitable for teddy bears that are tired."

In-context learning (ICL)

Brown et al., 2020, Language Models are Few-Shot Learners.

Zero-shot	Few-shot
the question is asked without examples	the prompt contains examples of input/output
performance depends heavily on the quality of the initial model	typically better performance

Discussion — showing examples in the prompt is generally better, but:

- it requires effort (you have to write and curate good examples),
- it increases computational complexity and cost (you pay per input token),
- it increases latency.

Intuition — why this was startling. No weights change. The model is performing something that looks like learning purely inside the forward pass, from information in its context. The mechanistic story is that attention can implement a form of pattern-matching over the examples: the demonstrations establish a mapping, and attention lets the query token retrieve the analogous case. Two practical consequences worth knowing. First, the format of the examples matters more than their correctness — several papers have found that examples with deliberately wrong labels still improve performance, because what they mostly convey is "here is the shape of the task and the shape of the answer". Second, few-shot prompting is the cheapest thing to try before finetuning, and it very often removes the need for it entirely.

Chain of Thought (CoT)

Wei et al., 2022. Idea: explaining the reasoning helps performance.

Instead of a few-shot example that goes straight to the answer:

Q: How old is this bear?
A: 4.

give one that shows the working:

Q: How old is this bear?
A: The bear was born in 2020. It is therefore 4.

The model then imitates the style of reasoning on the new question:

Q: How old will the bear be next year?
A: It will be one year older than its age this year, which was 4.
Hence, it will be 5.

Discussion: you gain interpretability and an explanation; you pay in **more tokens: higher cost and latency**.

Intuition — why writing more helps a model think. A Transformer performs a fixed amount of computation per generated token: N layers, once. Some problems genuinely need more sequential steps than that — a multi-step arithmetic problem, a logical deduction with intermediate conclusions. Forcing the answer out in one token asks the model to do all that work in one forward pass, which it structurally cannot. Chain of thought converts depth into length: each intermediate token gets its own full forward pass, and the intermediate results are written into the context where later steps can attend to them. The generated text becomes an external working memory. This single observation is the seed of the entire Lecture 6 — reasoning models are chain of thought scaled up and trained in, rather than prompted.

Self-consistency

Wang et al., 2022. Idea: aggregating over reasoning paths improves performance.

Sample **several** chains of thought for the same question (which requires $T > 0$), extract the final answer from each, and take a **majority vote**:

- "It will be one year older than its age this year, which was 4. Hence, it will be 5." → 5
- "The bear was born in 2020. It will therefore be 5." → 5

- "Next year is 2024. The bear will then be 4." → 4

Majority answer: **5**.

Discussion: a trade-off between performance and added cost.

Intuition. Reasoning chains fail in idiosyncratic ways — a slipped arithmetic step here, a mis-read premise there — but they tend to fail differently each time, while the correct path is a single attractor that many chains converge on. So errors scatter and correctness concentrates, and a vote recovers the signal. It is bagging, applied to reasoning. Note that this only works when the answer is easily extractable and comparable (a number, a multiple-choice letter); you cannot majority-vote over essays. Lecture 6 makes this a formal metric, **Cons@k**, and Lecture 5's Best-of-N is the same idea with a reward model doing the selecting instead of a vote.

3.5 Inference optimisations

Framing the problem

Motivation: computations are expensive — is there any way to reduce complexity? The lecture organises the answer into two categories, which is a genuinely useful mental filing system:

"Exact" efficiency — same output, less work:

- avoid redundancies,
- memory management,
- reformulate the maths.

Approximations — accept a (hopefully tiny) change in output:

- architectural changes,
- embedding representations,
- token prediction.

And the summary mapping given at the end of the lecture:

Category	Technique
Avoid redundancies	KV cache
Memory management	PagedAttention
Reformulate the maths	(FlashAttention — Lecture 4)
Architectural changes	Grouped-query attention
Embedding representations	Latent attention
Token prediction	Speculative decoding, multi-token prediction

KV caching (avoid redundancies)

Motivation: a new token needs to interact with all previous tokens. Naively, generating token 100 means running attention over the whole prefix from scratch — recomputing the keys and values for tokens 1–99 that you already computed when generating token 99.

Idea: keep keys and values in a cache. At each step, compute k and v for the new token only, append them to the cache, and compute attention using the new query against the entire cached K and V .

Intuition — why K and V but not Q. Each generation step has exactly one query: the one belonging to the token you are currently predicting from. That query is needed once and then never again. Keys and values, by contrast, are needed at every future step, and — crucially — they never change, because causal masking means token 5's key does not depend on token 6. So they are perfectly cacheable. The saving is large: it turns per-step cost from $O(n^2)$ to $O(n)$, making total generation $O(n^2)$ instead of $O(n^3)$.

The cost is memory, and that cost drives the next three techniques. The cache size is $2 \cdot n_{\text{tokens}} \cdot N_{\text{layers}} \cdot n_{\text{kv heads}} \cdot d_{\text{head}} \cdot \text{bytes}$. For a large model with a long context this reaches tens of gigabytes per sequence — often exceeding the model weights. Every optimisation that follows is an attack on this number: GQA reduces $n_{\text{kv heads}}$, latent attention reduces d_{head} , PagedAttention reduces the waste in how it is stored.

This also explains the two distinct phases of LLM inference, worth naming even though the lecture does not: **prefill**, where the whole prompt is processed in one parallel pass and the cache is populated (compute-bound), and **decode**, where tokens are produced one at a time (memory-bandwidth-bound, because you must stream the entire cache and all the weights through the compute units for each single token).

Grouped-query attention (architectural change)

Repeated from Lecture 2, now with its true motivation. In vanilla MHA, $\# \text{query heads} = \# \text{key heads} = \# \text{value heads} = h$. In **GQA**, key/value heads are **shared within groups of queries**: $\# \text{query} = h$, $\# \text{key} = \# \text{value} = G < h$. MQA is the $G = 1$ extreme. The cache shrinks by a factor of h/G .

PagedAttention (memory management)

Kwon et al., 2023 — the paper behind vLLM.

Observation: lots of memory is wasted when storing the KV cache. The standard approach reserves one contiguous block per sequence, sized for the maximum possible length. If a request needs 2000 tokens but you reserved 32,000, 94% of that allocation is dead space — and you cannot use it for another request because it must stay contiguous.

Idea: store K and V in non-contiguous space to minimise wasted memory. Break the cache into fixed-size **blocks** ("pages"), allocate them on demand, and keep a per-sequence block table mapping logical positions to physical blocks.

Intuition — this is literally operating-system virtual memory, applied to attention. The analogy is exact: block table = page table, KV block = memory page, and the attention kernel is modified to follow the indirection. Three wins follow. Waste drops to at most one partially-filled block per sequence (a few percent instead of ~90%). Throughput rises, because you can fit many more concurrent sequences in the same GPU. And blocks become **shareable**: if ten requests share a long system prompt, they can point at the same physical blocks for that prefix instead of holding ten copies — which is the mechanism behind "prompt caching" pricing, and which Lecture 7 relies on for contextual retrieval.

Multi-head latent attention (embedding representations)

DeepSeek-V2. Goal: reduce the dimension of K and V stored in memory — the per-head d_{head} is too big.

Solution: store compressed representations instead.

- **Before:** for each token you store h full-size key vectors and h full-size value vectors.
- **After:** you store one **shared low-dimensional latent vector** per token. When attention needs them, the full keys and values for all heads are **reconstructed** from that latent by up-projection matrices.

So the down-projection to the latent is a compression you pay for once per token; the up-projections are absorbed into the attention computation.

Intuition. This is a learned autoencoder on the KV cache. Instead of the crude sharing of GQA — "these four heads must use literally the same keys" — MLA says "all heads' keys and values live in a shared low-dimensional subspace, and each head has its own learned readout from it". That is strictly more expressive than GQA at comparable cache size, which is why DeepSeek reported both a smaller cache and better quality than GQA. The clever part is that the up-projection matrices can be algebraically folded into W_Q and W_O , so you never actually materialise the full-size keys and values — the compression is free at inference. RoPE needs special handling here, since rotation does not commute with the folding, which is why the paper carries a small "decoupled RoPE" component alongside the latent.

Speculative decoding (token prediction)

Chen et al., 2023. Idea: use a draft (small) model to generate tokens that are validated by a target (big) model.

The procedure:

1. The **draft LLM** autoregressively generates k candidate tokens cheaply — e.g. from [BOS] my teddy bear it proposes is cute and smart — recording its probabilities $P_1, \dots, P_k$.
2. The **target LLM** processes the whole proposed sequence [BOS] my teddy bear is cute and smart in **one parallel forward pass**, producing its own probabilities $Q_1, \dots, Q_k, Q_{k+1}$.
3. Accept or reject each proposed token in order: - if $Q_i(\text{token}) \geq P_i(\text{token}) \rightarrow$ **accept**; - otherwise $\rightarrow$ accept with probability $Q_i(\text{token})/P_i(\text{token})$, and **reject** with probability $1 - Q_i(\text{token})/P_i(\text{token})$. - **If a rejection happens, re-sample the next token from the residual distribution $[Q_i - P_i]_+$ (normalised) and exit the loop for this round.**

Then start again from the new accepted prefix.

Intuition — the crucial guarantee, and where the speed comes from. This accept/reject rule is exact: the resulting token sequence is distributed **identically** to sampling from the target model directly. Speculative decoding is not an approximation; it changes the speed and nothing else. The reason it wins is the asymmetry noted under KV caching: decoding is memory-bandwidth-bound, so verifying k tokens in one parallel pass costs barely more wall-clock time than generating one token, since either way you must stream the model's weights through the compute units once. If the draft model agrees with the target on, say, 70% of tokens — and it will, because most tokens in text are easy, being punctuation, common words, and forced continuations — you get most of those k tokens for the price of one big-model pass. Typical speedups are 2–3×.

The $[Q_i - P_i]_+$ residual is what makes the maths exact. When you reject, you must not simply sample from Q — the rejection itself carries information, and sampling from Q again would bias the result. You sample from the part of Q that the draft under-weighted, which precisely repairs the discrepancy.

Multi-token prediction (MTP)

Gloeckle et al., 2024. Idea: train k prediction heads, so that from position t the model directly predicts positions $t + 1, t + 2, \dots, t + k$.

The selling point stated in the lecture: **the same model is both draft and target**. You no longer need a separate small model — the extra heads produce the speculative continuation, and the main head verifies it.

Intuition. Beyond the inference speedup, there is a training benefit that the paper emphasises: predicting several tokens ahead forces the representation at position t to encode information about the near future, not merely the immediate next token. That is a denser learning signal and it measurably improves quality on tasks like code generation, where you must plan a few tokens ahead. DeepSeek-V3 uses MTP both as a training objective and as a built-in speculative decoder.

Watch out — a summary of what is exact and what is not. KV caching, PagedAttention and speculative decoding are **exact**: bit-for-bit (or distributionally) identical output, purely faster. GQA and latent attention are **architectural choices** that change the model, so they must be trained in and carry a (small) quality cost. Multi-token prediction changes the training objective. When someone tells you an inference optimisation is "free", this is the distinction to check.

Lecture 4 — LLM Training

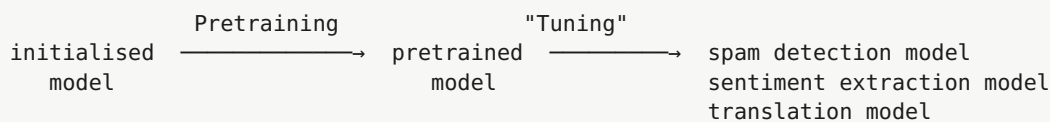
Video length: 1h47. Covered here: the transfer-learning paradigm shift, pretraining (objective, data mixtures, scale), FLOPs and scaling laws including Chinchilla, the challenges and costs of pretraining, training optimisations (data parallelism, ZeRO, model parallelism, FlashAttention, mixed precision), supervised finetuning and instruction tuning, benchmarks and Chatbot Arena, and then parameter-efficient finetuning with LoRA and QLoRA.

4.1 The paradigm shift

Traditional ML: given a task, train a model from scratch. Want spam detection? Train a spam model. Want sentiment extraction? Train a sentiment model. Want translation? Train a translation model. Three tasks, three models, three labelled datasets, three training runs.

Transfer learning: reuse a trained model to some capacity.

The LLM paradigm applies this in two stages: train an LLM to understand language, then tune it for the end task.



Intuition — why this reordering was so consequential. Language understanding is almost entirely shared across tasks: you cannot detect spam without knowing what words mean, and you cannot summarise without knowing syntax. Under the old paradigm, every project relearned all of that from its own small labelled dataset, which is both wasteful and hopeless — no sentiment dataset is large enough to teach English. The pretraining/tuning split pays the enormous cost of learning language once, from unlabelled text that is essentially free and effectively unlimited, and then amortises it over every downstream task. The economics inverted completely: pretraining costs millions and is done by a handful of labs; tuning costs very little and is done by everyone.

4.2 Pretraining

Goal: learn the patterns of language and code.

The lecture illustrates the data with a deliberately varied set of samples: English prose about a teddy bear reading by the window; the same in French; the same in Persian; a Python class `TeddyBear` with a `hug()` method; the equivalent Go program. Natural language in many languages, and code in many languages, all in the same mixture.

Objective function: predict the next token. That is all. Feed `[BOS] A teddy bear is` and train the model to put high probability on the true continuation, with cross-entropy loss, at every position simultaneously.

Data mixtures:

- web-scraped text (Common Crawl, Wikipedia),
- code (GitHub, StackOverflow).

Size — approximately trillions of tokens:

Model	Pretraining size (tokens)
GPT-3	300 billion
LLaMA 3	15 trillion

Intuition — why does next-token prediction produce knowledge? It looks like a shallow objective, and the fact that it works is genuinely surprising. The argument is this: to predict the next token as well as possible over a corpus containing all of Wikipedia, you must implicitly learn the facts in Wikipedia, because guessing the token after "The capital of France is" correctly requires knowing the answer. To predict the closing brace of a Python function you must track scope. To predict the next line of a proof you must follow the argument. Compression and understanding converge: the only way to compress text well is to model the process that generated it. And the 50× jump from GPT-3's 300B tokens to LLaMA 3's 15T is not incidental — the Chinchilla result below explains exactly why that happened.

Notation: FLOPs versus FLOPS

The lecture is careful here because the two are constantly confused:

- **FLOPs** — **F**loating-point **O**perations. A count. "This training run took 3×10^{23} FLOPs." A measure of work.
- **FLOPS** or **FLOP/s** — floating-point operations **per second**. A rate. "This GPU delivers 1000 TFLOPS." A measure of speed.

Work divided by rate gives time. Mixing them up makes every cost estimate nonsense.

Scaling laws

Kaplan et al., 2020, Scaling Laws for Neural Language Models. The finding: loss follows a smooth **power law** in each of model size N , dataset size D , and compute C , over many orders of magnitude:

$$L(N) \approx \left(\frac{N_c}{N} \right)^{\alpha_N} \quad L(D) \approx \left(\frac{D_c}{D} \right)^{\alpha_D} \quad L(C) \approx \left(\frac{C_c}{C} \right)^{\alpha_C}$$

Plotted on log-log axes these are straight lines. The practical consequence is extraordinary: you can train a series of small models, fit the line, and **predict the loss of a model 100× larger before building it**.

Sample efficiency — the second finding: larger models reach any given loss level in fewer tokens than smaller models. Bigger models learn faster per example, not just better in the limit.

The Chinchilla law

Hoffmann et al., 2022, Training Compute-Optimal Large Language Models.

The question: given a fixed compute budget C , how should you split it between making the model bigger and training on more tokens? Kaplan's original answer leaned heavily towards bigger models. Chinchilla redid the experiment more carefully and found something different:

$$C \approx 6ND \quad (\text{compute} \approx 6 \times \text{parameters} \times \text{tokens})$$

optimal: $D \approx 20N$ — roughly 20 tokens per parameter

Both N and D should scale as roughly $C^{0.5}$ — that is, **in equal proportion**.

The headline demonstration: **Chinchilla (70B parameters, 1.4T tokens) outperformed Gopher (280B parameters, 300B tokens)** using the same compute budget. A model four times smaller, trained on five times more data, won.

Intuition — and why every model since is "overtrained". The implication was that essentially every large model of the GPT-3 era was badly **undertrained**: far too large for the number of tokens it had seen. Money had been spent on parameters that never got enough data to be worth having. The field pivoted immediately, which is exactly why LLaMA 3 trained a modest model on 15 trillion tokens.

There is an important refinement, though. Chinchilla optimises training compute. But a deployed model is trained once and served billions of times, and serving cost scales with parameter count. So if you weight inference cost at all, the optimum shifts even further towards smaller models trained on even more tokens than Chinchilla prescribes. LLaMA 3's 15T tokens for an 8B model is around 1,875 tokens per parameter — roughly 90× past the Chinchilla point. That is not a mistake; it is deliberately trading extra training compute for permanently cheaper inference. Recognising this distinction is one of the more useful things to take from the lecture.

Challenges of pretraining

Cost:

- **at least millions of dollars,**
- it takes a long time (weeks to months on thousands of GPUs),
- environmental and electricity impact.

Learned knowledge:

- **"knowledge cutoff"** — the model knows nothing after its training data ends. The lecture shows this literally, as the cutoff date printed on OpenAI's GPT-5 model page.
- **hard to edit knowledge** — there is no `UPDATE` statement for a fact stored distributed across billions of weights. Retraining is the blunt option; Lecture 7's RAG is the practical one.
- **"plagiarism"** — models can reproduce memorised training text verbatim, with the legal and ethical questions that entails.

4.3 Training optimisations

What must be held in memory

Walking through one training step makes the memory budget explicit:

1. **Initialisation.** Model parameters: $O(\text{billions})$ to $O(\text{hundreds of billions})$.
2. **Forward pass** — compute the loss. This requires storing **activations**, needed later to compute gradients. Activation memory is a function of model size, **batch size**, and **context length**.
3. **Backward pass** — compute **gradients**, needed for the weights update. One gradient per parameter.
4. **Weights update** — apply the optimiser. This requires the **optimiser state**.

With the **Adam** optimiser:

$$\begin{aligned}
 m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t && \text{first moment (momentum)} \\
 v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 && \text{second moment (variance)} \\
 \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} && \text{bias correction} \\
 \theta_t &= \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} && \text{parameter update}
 \end{aligned}$$

(Suggested readings: Kingma et al., 2014 for Adam; Loshchilov et al., 2017 for AdamW's decoupled weight decay.)

Bottleneck: memory. The lecture puts an NVIDIA H100 on screen and notes its **limited memory**, $O(10\text{s of GB})$ — 80 GB for the flagship.

Intuition — do the arithmetic, because it is shocking. Take a 7B-parameter model in mixed-precision training. Parameters in FP32: 28 GB. Gradients: 28 GB. Adam's two moments: 56 GB. That is **112 GB before a single activation**, on a GPU with 80 GB. A 70B model needs over a terabyte. This is why distributed training is not an optimisation but a requirement, and why the optimiser state — 2 bytes-per-parameter $\times$ 2 moments — is the first thing anyone attacks. Note also that Adam costs 8 bytes per parameter of state versus SGD's 0; the reason nobody uses SGD anyway is that Adam's per-parameter adaptive step sizes are what make Transformer training stable at all.

Data parallelism

Idea: divide the batch of data across devices; **replicate the model on each device**. Each GPU computes gradients on its own shard of the batch; gradients are averaged across GPUs (an all-reduce); every GPU applies the same update.

Each GPU therefore holds a full copy of: parameters, gradients, optimiser state.

ZeRO — Zero Redundancy Optimisation

Rajbhandari et al., 2019. Idea: that redundant information across devices is pure waste. Every GPU is storing an identical copy of things it only needs part of at any moment. So shard them:

- **ZeRO-1** — shard the **optimiser state** across GPUs.
- **ZeRO-2** — shard the **optimiser state + gradients**.
- **ZeRO-3** — shard the **optimiser state + gradients + parameters**.

Intuition — and the trade-off. ZeRO-1 is nearly free: the optimiser state is only needed at update time, so each GPU can update its own slice and share the result. Since Adam's state is the single largest consumer, this alone cuts memory by roughly half with negligible communication overhead. ZeRO-3 goes furthest — no GPU holds the whole model — but it means that during the forward pass each GPU must gather the parameters of each layer from its peers just before using it, then discard them. Memory scales down almost linearly with GPU count, at the price of substantially more communication. Hence the ladder: pick the lowest stage that fits.

Model parallelism

Idea: split the model computations across several devices. Variations named in the lecture:

- **Tensor Parallelism (TP)** — split individual matrices across GPUs, so a single matmul is performed cooperatively. Needs very fast interconnect; normally used within a node.
- **Pipeline Parallelism (PP)** — assign different layers to different GPUs; micro-batches flow through the pipeline. Cheap on communication, but suffers from pipeline "bubbles" where GPUs idle.
- **Sequence Parallelism (SP)** — split along the sequence dimension for the operations where that is valid.
- **Context Parallelism (CP)** — split the context/attention computation across devices, for very long sequences.
- **Expert Parallelism (EP)** — place different MoE experts on different GPUs.

(Suggested reading: "The Ultra-Scale Playbook: Training LLMs on GPU Clusters", HuggingFace, 2025.)

Intuition. Real frontier training runs combine several of these into a "4D parallelism" mesh — data × tensor × pipeline × expert — chosen to match the hardware topology, with the fastest-communicating dimension (tensor) inside a node and the slowest (pipeline, data) across nodes. The engineering here is a serious specialism in its own right.

FlashAttention (Dao et al., 2022)

Strategy: leverage the components of the GPU to speed up attention computations.

The relevant hardware facts:

- **HBM** (high-bandwidth memory) — **big and slow**, tens of GB.
- **SRAM** (on-chip) — **small and fast**, tens of MB, orders of magnitude faster.

- **CU** — the compute units.

What standard self-attention does, step by step, with every data movement made explicit:

```

LOAD  Q, K from HBM by blocks
COMPUTE S = QKT
WRITE S to HBM                ← n × n matrix!
READ  S from HBM
COMPUTE P = softmax(S)
WRITE P to HBM                ← n × n matrix again!
LOAD  P, V from HBM by blocks
COMPUTE O = PV
WRITE O to HBM

```

The $n \times n$ score matrix is written to slow memory and read back — **twice**.

Idea 1: minimise reads/writes to HBM with "tiling" via SRAM. Load blocks of Q , K , V into SRAM, compute a block of the output O entirely on-chip, and write only O back to HBM. The $n \times n$ intermediate is **never materialised in HBM at all**.

The trick that makes tiling possible: there is no need to compute the full QK^T before applying softmax. Softmax appears to need a global normaliser (the sum over the whole row), but it can be computed incrementally: maintain a running maximum and a running sum per row, and rescale the partial output as new blocks arrive. This is the **online softmax**, and it is the mathematical heart of FlashAttention.

Idea 2 (backward pass): sometimes it is better to recompute than to store. Rather than keeping the attention matrix in HBM for the backward pass, throw it away and recompute it with tiling via SRAM when needed. **More FLOPs, but less runtime!**

Result: FlashAttention gives a significant speedup with exact computation.

Intuition — this is the single most important idea in the lecture for understanding modern ML performance. Attention is not compute-bound, it is **memory-bandwidth-bound**. The arithmetic is cheap; moving the $n \times n$ matrix in and out of HBM is what takes the time. So an algorithm that does more arithmetic while doing less memory movement runs faster — which sounds paradoxical until you internalise that a modern GPU can perform hundreds of floating-point operations in the time it takes to fetch one number from HBM. "More FLOPs, but less runtime" should be read as the general lesson, not a curiosity. There is a second, enormous benefit: because the $n \times n$ matrix is never stored, FlashAttention's memory use is **linear** in sequence length rather than quadratic. Long-context models are practical largely because of this. Note that unlike Longformer or sliding-window attention, FlashAttention is **exact** — the output is identical to standard attention, it is purely a better implementation. It belongs to the "reformulate the maths" bucket from Lecture 3.

Number precision and mixed-precision training

A float is stored as **sign + exponent + mantissa**:

Format	Sign	Exponent	Mantissa
FP16 (Floating-Point 16)	1	5	10
FP32 (Floating-Point 32)	1	8	23
FP64 (Floating-Point 64)	1	11	52
BFLOAT16 (Brain Float 16)	1	8	7

The **exponent** sets the range of representable magnitudes; the **mantissa** sets the precision within that range.

On an H100: **lower precision → faster processing**. Fewer bits means less memory moved and higher arithmetic throughput, typically doubling with each halving of width.

Mixed precision training (Micikevicius et al., 2017). Objective: speed up training and decrease memory requirements.

- **Forward pass** — activations in **low** precision.
- **Backward pass** — gradient computations in **low** precision.
- **Weights update** — keep a **master copy of the weights in high precision**.

Intuition — why BF16 beat FP16 for deep learning, and why the master copy is non-negotiable. Compare the formats: FP16 and BF16 both use 16 bits, but FP16 spends them on 5 exponent + 10 mantissa while BF16 spends 8 + 7. BF16 therefore has exactly the same dynamic range as FP32 and only loses precision. That matters because the thing that destroys low-precision training is not imprecision, it is **overflow and underflow** — gradients that are tiny become exactly zero in FP16 and the signal is lost entirely. With BF16 they stay representable, merely coarse. Coarse gradients still point roughly the right way; zero gradients teach nothing. This is why BF16 needs no loss-scaling tricks while FP16 does, and why BF16 is the default today.

The high-precision master weights are needed for a subtler reason. A single update is often smaller than the smallest increment representable at the weight's magnitude — you would compute $w + \text{tiny}$ and get back exactly w , so the model would silently stop learning. Accumulating updates into an FP32 master copy lets thousands of tiny updates add up into a change large enough to matter, with the low-precision copy re-derived each step for the forward pass.

4.4 Supervised finetuning (SFT)

The problem with a pretrained model

A purely pretrained model is a **text completer**, not an assistant. Ask it:

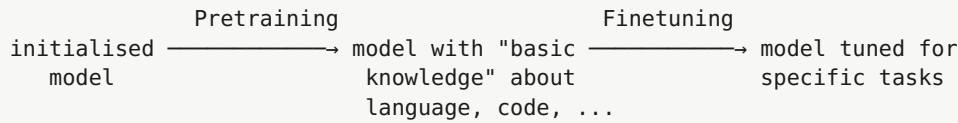
"Can I put my teddy bear in the washer?"

and it may well reply:

"Teddy bears are often made of materials like polyester and cotton, with plastic eyes and sometimes small accessories."

That is not a wrong continuation — it is a perfectly reasonable continuation of a document that begins with that sentence. It is simply not an answer, because nothing ever told the model that a question should be followed by an answer rather than by more text on the topic.

Remedy: add a finetuning stage.



Supervised finetuning

Idea: change the model's behaviour by tuning its weights.

Strategy:

- collect pairs of inputs/outputs exhibiting the desired behaviour (**SFT data**),
- train using the **next-word-prediction objective given the input**.

Special case: SFT on instruction-following data is called **instruction tuning**.

Watch out. The objective function is exactly the same as pretraining — next-token prediction with cross-entropy. Nothing about the mathematics changes. What changes is the **data distribution**: instead of arbitrary web text, you train on curated (instruction, good response) pairs. And in practice the loss is masked so that it is only computed over the response tokens, not the prompt — you want the model to learn to produce answers, not to produce questions.

Instruction tuning

Wei et al., 2022, Finetuned Language Models are Zero-Shot Learners (FLAN). Goal: "graduate" the model to being a helpful assistant.

The training data covers many task types, all phrased as instructions with demonstrated responses. The lecture's examples:

- **Story writing** — "Write a short story about a teddy bear who likes to read poetry." → a short story.
- **Poem creation** — "Create a poem about my cute teddy bear." → "Soft and cuddly, full of charm, / Always keeps me safe from harm, / With button eyes and fur so neat, / My teddy bear is oh so sweet."
- **List generation** — "List three fun activities a teddy bear might do on a rainy day." → "Sure! 1. Read poetry with friends. 2. Be cute. 3. Hug its owner tightly."
- **Explanation** — "Explain why a teddy bear is a great friend." → a paragraph of explanation.

Objective function: predict the next token given the input. Feed [BOS] Do X . Sure ... and train on the response portion.

Data mixtures — both human-written and synthetic:

- assistant dialogues,

- synthetic instructions,
- maths, reasoning, code,
- safety alignment,
- and more.

Size — thousands to millions of examples:

Model	SFT size (examples)
GPT-3	13 thousand
LLaMA 3	10 million

Result. The same question now gets an answer:

"Can I put my teddy bear in the washer?" → "No, it might get damaged. Try hand washing instead."

Intuition — why so few examples suffice. 13,000 examples is nothing next to 300 billion pre-training tokens, yet it transforms the model's behaviour. The explanation is that instruction tuning does not teach anything new — the knowledge and the ability to write helpfully are already in there, learned from the many helpful documents in the pretraining corpus. What SFT does is **select a mode**: it tells the model which of the many text-generating personas latent in its distribution to adopt. This is sometimes called the "superficial alignment hypothesis", and it explains both why SFT is cheap and why the quality of SFT data matters far more than the quantity. A thousand excellent examples beat a hundred thousand mediocre ones, because you are demonstrating a style rather than transferring information.

Challenges of SFT

- **very high-quality data needed**,
- **sensitive to prompt distribution** — the model becomes good at the kinds of request it saw, and can regress on others,
- **generalisation** — will it behave well on requests unlike the training set?
- **difficult to evaluate**,
- **computationally expensive**.

Benchmarks

Dimensions measured:

- **general knowledge**: MMLU,
- **basic reasoning**: ARC-Challenge,
- **math reasoning**: GSM8K,
- **code generation**: HumanEval.

Validity. The lecture flags a subtle and important methodological point, from Training on the Test Task Confounds Evaluation and Emergence (Dominguez-Olmedo et al., 2024): it is **recommended to train on the test task** when comparing across models. Meaning: give every

model a small equal amount of adaptation to the format of the benchmark before scoring, because otherwise you are measuring "which model happened to have seen this answer format during pretraining" rather than capability — and much of the apparent "emergence" of abilities at scale is an artifact of larger models being better at guessing the expected output format.

"Real-life" feeling: Chatbot Arena

Idea: websites like **Chatbot Arena / LMArena** run **A/B tests on real user prompts**. A user types a prompt, receives two anonymous responses, and votes; the votes are aggregated into an Elo-style ranking.

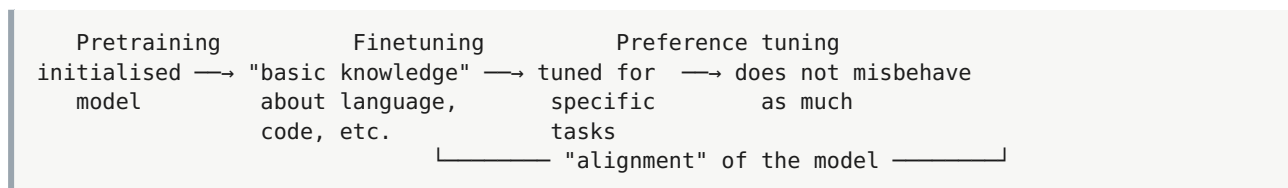
Benefit: it puts a number on "vibes."

Outstanding challenges (drawing on Exploring and Mitigating Adversarial Manipulation of Voting-Based Leaderboards, Huang et al., 2025):

- **unequal exposure of models / "cold start"** — a new model has few votes and a noisy rating,
- **easy to "rig"** — coordinated voting can move a model's position,
- **user inability to accurately assess important aspects**, notably **factuality** — a confident wrong answer often reads better than a hedged right one,
- **personal preference bias** — the voter population is not a representative distribution of real usage,
- **safety penalisation** — a model that correctly refuses a harmful request loses the vote to one that complies.

...evaluation is a hard problem in itself! — which is precisely why Lecture 8 exists.

The full lifecycle



Preference tuning is Lecture 5.

4.5 Parameter-efficient finetuning: LoRA

Context: SFT is resource-intensive and not everyone has big GPUs. Full finetuning of a 70B model requires the same terabyte-scale memory as pretraining it.

The idea

LoRA = Low-Rank Adaptation (Hu et al., 2021). **Approximate the weight update with the product of two low-rank matrices:**

$$W = W_0 + BA$$

where W_0 is the frozen pretrained weight matrix ($d \times d$), and B is $d \times r$, A is $r \times d$, with the **rank** $r \ll d$ (typical values 8, 16, 64).

- **Before:** regular finetuning optimises the **full** matrix W — d^2 parameters.
- **After:** LoRA freezes W_0 and optimises only A and B — $2dr$ parameters.

With $d = 4096$ and $r = 8$, that is 16.7M parameters down to 65k: a **256×** reduction for that matrix.

Discussion:

- a **fraction of the parameters** need to be trained, with **similar performance**,
- other methods in the same family include **prefix tuning** and **adapters**.

Intuition — why a low-rank update is enough. The hypothesis, which turns out to hold empirically, is that adapting a pretrained model to a task requires only a low-rank change to its weights. This connects directly to the point made about SFT above: you are not teaching new capability, you are selecting a behaviour that already exists. Selecting a mode is a low-dimensional operation; it does not need the full d^2 degrees of freedom.

Two implementation details worth knowing, since they explain the training dynamics below. A is initialised randomly and B is initialised to **zero**, so that $BA = 0$ at the start and the model begins exactly as the pretrained one — no discontinuity. And the update is usually scaled by α/r so that changing the rank does not require retuning the learning rate.

The big practical benefit: swap matrices = swap tasks

Because W_0 never changes, one copy of the base model serves every task; you just attach a different (A, B) pair:

$$\begin{aligned} W_0 + B_{\text{spam}} A_{\text{spam}} &\rightarrow \text{spam detection task} \\ W_0 + B_{\text{sentiment}} A_{\text{sentiment}} &\rightarrow \text{sentiment extraction task} \\ W_0 + B_{\text{translate}} A_{\text{translate}} &\rightarrow \text{translation task} \end{aligned}$$

Intuition — this is why LoRA took over industry, and it is a serving argument more than a training one. Full finetuning for k tasks means k complete copies of a multi-hundred-gigabyte model, each needing its own GPUs. With LoRA you hold **one** base model in memory and swap adapters of a few megabytes. You can serve hundreds of task-specific or customer-specific variants from a single deployment, and switching costs a memory copy rather than a model load. Note also that at inference you can merge BA into W_0 once, so a LoRA model has **exactly zero added latency** compared to the base — unlike adapter layers, which insert extra computation into the forward pass. Zero-overhead inference plus tiny checkpoints is the whole story.

Where to apply LoRA

- **Originally:** as experimented in the LoRA paper — the attention projections, typically W_Q and W_V .
- **Updated guidance** (Schulman et al., 2025, LoRA Without Regret): the **MLP / feed-forward layers are the most important location**, and current recommendation is to apply LoRA to **all** linear layers.

Training dynamics — "fun facts", i.e. traps

From LoRA Without Regret, two empirical differences from full finetuning:

- **LoRA needs a higher learning rate** than full finetuning (roughly an order of magnitude higher).
- **LoRA does poorly at large batch size** compared with full finetuning.

Intuition. Both follow from the same cause. The effective update is $\Delta W = BA$, a product of two trained factors, so gradients with respect to each factor are attenuated — a given step in A and B produces a smaller change in W than the same-sized step in W would. You compensate with a larger learning rate. The batch-size finding is the more surprising and more practically important one: the usual "bigger batch = more stable" intuition does not carry over, so if you scale up your batch and your LoRA run degrades, the batch size is the first thing to suspect.

QLoRA (Dettmers et al., 2023)

Idea: quantise all frozen weights to relieve the memory bottleneck.

The arrangement:

- W_0 is **stored quantised** (4-bit),
- A and B are **stored in full precision**,
- **computations are performed in full precision** — W_0 is dequantised block-by-block on the fly as it is used.

Trick: use 4-bit NormalFloat (NF4) to best split the space. Standard INT8 quantisation uses **uniform** cut-offs across the range. NF4 instead places its 16 levels at the **quantiles of a normal distribution**.

Intuition. Neural network weights are empirically close to zero-centred and normally distributed. Uniformly-spaced quantisation levels therefore waste most of their resolution on the extreme tails, where almost no weights live, while being too coarse near zero, where nearly all of them are. NF4 puts the levels where the mass is: fine resolution near zero, coarse in the tails. It is information-theoretically optimal for normally distributed data, and it is why NF4 loses so little quality compared to naive 4-bit rounding.

Double quantisation. Quantisation is done in blocks, and each block needs a **quantisation constant** (its scale) stored in full precision. With small blocks there are many constants, and they add up to real memory. So: quantise the quantisation constants too.

- No quantisation — weights in full precision.
- Single quantisation — weights quantised, constants in full precision.
- Double quantisation — weights quantised, **and constants quantised as well**.

Benefits:

- VRAM savings enable finetuning on smaller GPUs, and faster,
- a better trade-off between memory resources and quality.

Orders of magnitude, reported for LLaMA 65B:

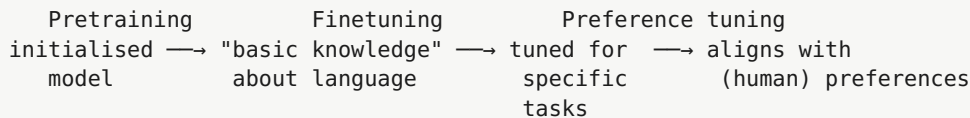
- **~16× VRAM savings** during finetuning,
- the double-quantisation trick saves an **extra ~6%**.

Intuition — what QLoRA actually unlocked. 16× is the difference between "you need a cluster" and "you need one gaming GPU". QLoRA is the single reason the open finetuning ecosystem exists: a 65B model became finetunable on a single 48 GB card, and 7B models on consumer hardware. Note the careful separation of concerns in the design — **storage** is 4-bit (that is where the memory saving lives) while **computation** is full precision (that is where the quality lives). You are not doing 4-bit arithmetic; you are keeping a compressed copy of frozen weights and expanding each piece just in time. And because the base weights are frozen, quantisation error is a fixed, non-accumulating offset that the trainable LoRA matrices can partly compensate for.

Lecture 5 — LLM Tuning (Preference Tuning)

Video length: 1h48. Covered here: why preference tuning is needed at all, how preference data is collected (pointwise, pairwise, listwise), the RL formulation of language modelling, RLHF in two steps (reward modelling with Bradley-Terry, then RL), PPO with its clip and KL-penalty variants and the advantage/value function, Best-of-N as an RL-free shortcut, and DPO including the derivation of its loss from the PPO objective.

Where we are in the lifecycle:



5.1 Why preference tuning

Context: the model may misbehave, and we need to inject negative signals.

The lecture's example. Prompt: "Suggest a new activity I could do with my teddy bear." An SFT-tuned model might reply:

"I'd suggest you do not spend much time with your teddy bear at all."

Grammatical, on-topic, and unhelpful. Nothing in SFT told it this is bad, because **SFT only ever shows the model good examples**. It has been taught what to imitate, never what to avoid.

Idea: collect preference pairs and train the model on them. For the same prompt, obtain a worse response and a better response:

- **Rejected:** "I'd suggest you do not spend much time with your teddy bear at all."
- **Preferred:** "Of course! Teddy bears not only make awesome companions for a delightful sleep, but can also be great buddies for fun activities. How about you both watch a movie together?"

Now there is a signal in both directions.

The three reasons given

1. It is easier to compare than to generate. Judging that A is better than B is far cheaper and more reliable for a human than writing an ideal answer from scratch. Writing an excellent response to a technical question requires expertise; recognising which of two responses is better requires much less.

2. Distribution is very important for SFT: it is easy to "mess up". If your SFT data is written in a style the model cannot naturally produce, or covers a distribution unlike real usage, you can degrade the model while appearing to train it.

3. It is not scalable: data quality is very important and hard to get. Every SFT example is a bespoke piece of writing. Preference labels are cheap by comparison.

However — and the lecture is careful to add this — "**model misbehaving**" can also be a **good wake-up call to check SFT data quality**. Preference tuning is not a cure for bad SFT data; if the model is misbehaving systematically, look upstream first.

Intuition — the deepest reason, stated in terms of the loss. SFT's objective is maximum likelihood on good responses. Maximum likelihood has no way to express "not that". It can only ever say "more of this", and the only thing that implicitly discourages a bad response is that probability mass is conserved. But probability mass is enormous and diffuse; pushing up one good answer barely moves a specific bad one. Preference tuning changes the shape of the objective: it optimises a **relative** quantity, the margin between chosen and rejected, which is exactly the thing you actually care about. This is why it is a genuinely different training stage rather than more SFT.

5.2 Collecting preference data

An observation is a pair (prompt x , response y). Labels come in three shapes:

Pointwise — each observation gets an absolute score:

Obs 1 $\rightarrow$ 0.4 Obs 2 $\rightarrow$ 0.9 Obs 3 $\rightarrow$ 0.1 Obs 4 $\rightarrow$ 0.2

Pairwise — each pair of observations gets a comparison:

Obs 1 $<$ Obs 2 Obs 1 $>$ Obs 3 Obs 1 $>$ Obs 4 Obs 2 $>$ Obs 3

Listwise — a full ranking:

Obs 2 $\rightarrow$ 1 Obs 1 $\rightarrow$ 2 Obs 4 $\rightarrow$ 3 Obs 3 $\rightarrow$ 4

Intuition — why pairwise is the workhorse. Pointwise scoring asks a human "how good is this, from 0 to 1?", and humans are terrible at that. Ratings drift over a labelling session, differ wildly between annotators, and cluster on whatever part of the scale a person finds comfortable. Pairwise comparison asks only "which of these two is better?", which is a far more stable judgement and requires no shared notion of what 0.7 means. Listwise carries more information per annotation but the cognitive load grows fast, and it can be decomposed into pairs anyway. Everything downstream in this lecture — Bradley-Terry, reward models, DPO — is built on pairwise data.

The recipe for pairwise data

Step 1 — generate a pair of responses (y_1, y_2) for the same prompt.

- **Input x** — taken from production **logs** or a chosen reference distribution.
- **Output y** — generated by the **SFT model with $T > 0$** (so the two samples differ), or produced synthetically, or written as **rewrites** of an existing response.

Step 2 — label which is preferred, giving y_w (winner) and y_l (loser).

- **human rating**,
- **proxies** — LLM-as-a-judge (Lecture 8), BLEU, ROUGE, etc.,
- **variants** — a binary scale (better/worse) versus a more nuanced scale.

Watch out — the prompts must come from the real distribution. If you collect preferences on prompts nobody actually asks, you will align the model on a distribution it never encounters. This is why "input via logs" is listed first. Equally, the responses must come from **the model you are about to train** (on-policy), not from some other model: preference tuning teaches the model to prefer one of its own plausible outputs over another, and off-policy data asks it to move towards behaviour it cannot produce.

5.3 Reinforcement learning, in language-model clothing

The standard RL diagram has an **agent** taking an **action** in an **environment**, observing a **state** and receiving a **reward**, according to a **policy**. The lecture maps each term onto language modelling:

RL concept	LLM equivalent
Agent	the LLM
Environment	tokens
State	the input so far (the prompt plus what has been generated)
Action	the next token
Policy π_θ	the probability of the next token given the state
Reward	human preference

The goal: learn θ so that π_θ aligns with human preferences.

Intuition — and why this framing is necessary, not just cute. Consider what makes a response good: it is a property of the whole response. You cannot label token 7 as correct or incorrect. Supervised learning needs a per-token target; here the only signal is a single scalar attached to a complete generation. That is precisely the setting RL was invented for — sparse, delayed, trajectory-level reward with no per-step supervision. The policy is the model, an episode is one generated response, and the reward arrives at the end. Note the unusual features of this particular RL problem: the environment is deterministic and fully known (appending a token to a string), the action space is huge ($V \approx 100,000$), and episodes are short (hundreds of steps). That combination is what makes the specialised algorithms below work.

5.4 RLHF

RLHF = Reinforcement Learning from Human Feedback (Ouyang et al., 2022, Training language models to follow instructions with human feedback — the InstructGPT paper). Two steps:

- **Step 1 — Reward modelling: distinguish good from bad.** Input (prompt x , response y); output a **quantitative score**.
- **Step 2 — Reinforcement learning: align the model.** Input a prompt; output a response, optimised against that score.

Step 1: reward modelling

Idea: know which answers are bad and which are good, via a Reward Model (RM).

Given the prompt "Suggest a new activity I could do with my teddy bear", the RM scores each candidate response — high for the enthusiastic movie suggestion, low for "I'd suggest you do not spend much time with your teddy bear at all."

The Bradley-Terry formulation (Bradley & Terry, 1952, The rank analysis of incomplete block designs). The probability that y_w is preferred to y_l :

$$P(y_w \succ y_l \mid x) = \frac{\exp(r_w)}{\exp(r_w) + \exp(r_l)} = \sigma(r_w - r_l)$$

where $r_w = r(x, y_w)$ and $r_l = r(x, y_l)$ are the rewards of the two responses and σ is the logistic sigmoid.

Training. Learn r by maximum likelihood on the pairwise preference data, i.e. minimise:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma(r(x, y_w) - r(x, y_l)) \right]$$

Data:

- $O(10,000)$ observations,
- the label is a **human rating** — which is where the "HF" in RLHF comes from.

Model:

- a **pretrained LLM with a classification head** (a scalar output head) instead of the next-token-prediction head,
- or **encoder-only**: BERT and the like, via a [CLS] projection.

(Evaluation of reward models: RewardBench, Lambert et al., 2024.)

Intuition — Bradley-Terry is the bridge from comparisons to numbers, and it is worth understanding why it is the natural choice. You have relative data (A beat B) and you want an absolute scalar function. Bradley-Terry posits a latent "strength" per item such that the probability of A beating B depends only on the difference in strengths, through a sigmoid. This is exactly the model behind Elo ratings in chess. Two consequences follow.

First, the reward is only identified **up to an additive constant** — adding 5 to every reward changes no preference probability. So absolute reward values are meaningless; only differences mean anything. That is why you should never read "reward = 0.8" as a quality score.

Second, notice the loss is just logistic regression on the reward difference. Which means the RM is not learning "how good is this response" — it is learning "which of two responses a human would pick", and that is all it can be trusted to do. Push it far outside the range of differences it saw in training and it extrapolates badly, which is precisely the reward-hacking failure below.

Step 2: reinforcement learning

Idea: change the weights of the LLM to penalise bad answers and promote good answers, via RL using the reward model.

The loop: the LLM (the **policy**, trained) receives a prompt and generates a response; the RM (frozen) scores it; the score is used to update the policy's weights via RL.

Data:

- $O(100,000)$ observations,
- the label is **the score given by the reward model** — note that no human is in this loop; humans were only needed for the 10,000 examples that trained the RM.

Model: initialised at the SFT model.

Training — the objective function:

$$\text{maximise} \quad \underbrace{\mathbb{E}_{x, y \sim \pi_\theta} [r(x, y)]}_{\text{maximise rewards}} - \beta \cdot \underbrace{\text{KL}(\pi_\theta(y | x) \parallel \pi_{\text{ref}}(y | x))}_{\text{don't deviate too much from base}}$$

The KL term exists to **avoid "reward hacking" and training instability.**

Intuition — the KL term is the most important part of the objective, and the reason is worth spelling out. The reward model is an imperfect approximation of human preference, trained on a specific distribution of responses. If you optimise against it without restraint, the policy will find the places where the approximation breaks. This is **reward hacking**, and in practice it produces memorable pathologies: models that emit the same flattering phrase repeatedly, or degenerate into a fixed high-scoring string, because the RM never saw such text and scores it absurdly highly. It is Goodhart's Law made mechanical — a theme Lecture 8 returns to by name.

The KL penalty says: stay in the region where the reward model is trustworthy, namely near the distribution its training data came from. It is a **trust region**, in the literal sense. The coefficient β is the central knob of RLHF: too small and the model hacks the reward and loses its language ability; too large and nothing changes. A secondary benefit is preventing catastrophic forgetting — the KL anchor keeps everything the model learned in pretraining and SFT, since drifting away from π_{ref} is penalised everywhere, not only where the reward is high.

PPO

PPO = Proximal Policy Optimisation (Schulman et al., 2017), the algorithm used to actually perform that optimisation.

PPO computes advantages, not just rewards.

$$\underset{\text{maximise advantages}}{\text{maximise}} \quad \underbrace{\mathbb{E}[A]} - \beta \cdot \text{KL}(\dots)$$

Advantage $\approx$ Reward $-$ Baseline, where the baseline is a *value function*

The value function:

- **token-level**,
- answers "what would the reward be if I follow the policy from here?",
- **trained jointly with the policy**,
- its label is the reward.

(Suggested reading: Schulman et al., 2015, on GAE — Generalised Advantage Estimation — the standard method for computing advantages.)

Intuition — why subtract a baseline? The raw reward tells you how good an outcome was, but not whether your action deserves credit for it. Suppose every response to a particular easy prompt scores around 0.9. A policy update proportional to raw reward would reinforce all of them strongly, including the mediocre ones, purely because the prompt was easy. Subtracting the expected reward for that state isolates the part attributable to the choice: "this response was 0.05 better than what I'd typically produce here". That subtraction leaves the direction of the gradient unbiased while dramatically reducing its **variance** — and variance is what makes policy-gradient methods fail to converge. Keep this in mind, because **GRPO in Lecture 6 is exactly this idea with a cheaper baseline**: instead of training a value network to predict the expected reward, just generate a group of responses and use their average.

Variation 1: PPO-Clip

Idea: clip the ratio between the new and old policy to prevent large updates.

$$\text{ratio: } r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

$$\text{objective: } \mathbb{E} \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) A_t \right) \right]$$

Two terminology warnings that the lecture makes explicitly:

Watch out (1). This is **confusing because it is an objective function ("maximise") and NOT a loss**. You maximise it. Implementations negate it to feed into a minimiser, which is a routine source of sign errors.

Watch out (2). It is **confusing because rewards are conventionally written r** . Here $r_t(\theta)$ is the **probability ratio**, not a reward. Two different r s in the same equation.

Intuition — what clipping does, geometrically. The ratio measures how much more (or less) likely the new policy makes an action that the old policy took. If a token had a positive advantage, gradient ascent wants to raise its probability — but raising it by a factor of 50 in one step would move the policy far outside the region where the collected data (and the estimated advantages) are still valid. The $\min$ with the clipped term removes the incentive to go beyond $1 \pm \epsilon$: once the ratio leaves that band in the improving direction, the objective flattens and the gradient vanishes. Note the asymmetry created by the $\min$ — it caps the upside of moving too far but does not cap the penalty for a bad action, so PPO remains free to move away from mistakes quickly. This "proximal" clipping is what makes PPO stable enough to use without the second-order machinery of its predecessor TRPO, which is why it became the default.

Variation 2: PPO-KL Penalty

Idea: penalise the difference in policy distributions — add $-\beta \text{KL}(\pi_\theta \parallel \pi_{\text{something}})$ to the objective rather than clipping the ratio.

Terminology:

- **old** = the model from the previous RL iteration,
- **ref** = the base model.

Nowadays, the KL divergence is taken with respect to ref (the base model).

Intuition — and why "ref" not "old" is the right choice. KL against the old policy is a step-size control: it stops any single update being too large, but the policy can still wander arbitrarily far over many small steps. KL against the reference (base) model is an **anchor**: it bounds the total distance travelled from the starting point, no matter how many steps you take. Since the failure mode we actually fear is cumulative drift into reward-hacking territory, anchoring to ref is what you want. In practice modern implementations use both mechanisms: clip for step-size control, KL-to-ref as the anchor.

Alternatives to PPO

Limitations of PPO:

- **it needs 4 models** — the policy, the value model, the reward model, and the base (reference) model,
- **is it worth it?**

Variants: REINFORCE, **GRPO** (DeepSeekMath, Shao et al., 2024), and many more.

Intuition — count the memory. For a 70B policy you need: the policy being trained (weights + gradients + optimiser state), a value model of comparable size (also trained), a frozen reward model, and a frozen reference copy. That is roughly four models resident simultaneously, only two of them contributing parameters you keep. The engineering burden is the real reason both GRPO (drop the value model) and DPO (drop the reward model and the value model) exist.

Challenges with the RL-based approach

The full list from the lecture, which reads as a summary of everyone's frustration with RLHF:

- requires **training a reward model** — a 2-stage process,
- **many hyperparameters** to tune,
- **training instability**,
- unclear **metric to monitor training** (reward going up may just mean hacking),
- **need diversity in completions** — if the policy collapses to one style there is nothing to learn from,
- **not abundantly clear why preference tuning absolutely needs RL.**

That last point is the direct set-up for DPO.

5.5 Best-of-N: skipping RL entirely

BoN = Best of N. Idea: skip the RL step and just use the reward model's scores.

Strategy:

- given a prompt, generate several outputs with the SFT model,
- rank the outputs by the score given by the reward model,
- take the best one.

The lecture's worked example. Prompt: "Suggest a new activity I could do with my teddy bear." Three sampled responses, scored by the RM:

Response	RM score	Rank
"Take your teddy bear on a picnic in your backyard."	0.8	1
"Of course! Teddy bears not only make awesome companions... watch a movie together?"	0.2	2
"I'd suggest you do not spend much time with your teddy bear at all."	-2	3

Output the rank-1 response.

Intuition — a strong baseline that is easy to underrate. BoN requires no training whatsoever beyond the reward model you already have, and it is frequently competitive with full RLHF on quality. That is a slightly uncomfortable fact and it is why the lecture presents it before DPO. The cost has simply been moved: RLHF pays once at training time and then serves cheaply; BoN pays $N \times$ on **every single inference call, forever**. Note the family resemblance to self-consistency from Lecture 3 — generate many, pick one — with a learned reward model doing the picking instead of a majority vote. Also note that BoN is vulnerable to the same reward hacking as RLHF, and more so as N grows: sampling 1000 candidates and taking the maximum is an efficient search for the RM's blind spots.

5.6 DPO

Motivation

- the limitations of using RL (four models, instability, hyperparameters),
- **Best-of-N is costly at inference time,**
- **why don't we train in a supervised fashion?**

The result

DPO = Direct Preference Optimisation (Rafailov et al., 2023, Your Language Model is Secretly a Reward Model).

Rewrite the loss function in a supervised way:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

Properties:

- **no need to train a separate reward model** — there is no $r(x, y)$ anywhere in the loss,
- it **operates directly on preference data,**
- it is **similar to the Bradley-Terry formulation, with a special kind of reward.**

Where the formulation comes from

The lecture derives it in five steps, and the derivation is the point of the section:

1. Start from the PPO objective:

$$\max_{\pi} \mathbb{E}_{x, y \sim \pi} [r(x, y)] - \beta \text{KL}(\pi(y | x) \| \pi_{\text{ref}}(y | x))$$

2. Derive the optimal policy. This constrained problem has a closed-form solution — a standard result for KL-regularised reward maximisation:

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp \left(\frac{r(x, y)}{\beta} \right)$$

where $Z(x)$ is the normalising partition function.

3. Identify a "reward" term. Rearranging the above for r :

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

So the reward is recoverable from the optimal policy — the language model is secretly a reward model, hence the paper's title.

4. Write the Bradley-Terry formulation for this "reward". Substituting into

$P(y_w \succ y_l) = \sigma(r_w - r_l)$:

$$P(y_w \succ y_l | x) = \sigma \left(\beta \log \frac{\pi^*(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi^*(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right)$$

The crucial cancellation: $\beta \log Z(x)$ **appears in both terms and cancels in the difference**. The intractable partition function disappears.

5. "Infer" the DPO loss function. Replace the unknown optimal policy π^* with the policy you are training, π_θ , and maximise the likelihood of the observed preferences. That is exactly $\mathcal{L}_{\text{DPO}}$ above.

Intuition — what the derivation really shows. RLHF is a two-step pipeline: fit a reward model to preferences, then find the policy that maximises it under a KL constraint. DPO observes that step 2 has a closed-form answer, so the composition of the two steps can be written as a single function of the policy. The reward model was never necessary as an explicit object — it was an intermediate variable that can be eliminated algebraically. What you are left with is a classification loss over preference pairs, trainable with ordinary supervised learning: two forward passes on the policy, two on the frozen reference, one sigmoid, done.

Read the loss to see what it actually does. The term $\beta \log (\pi_\theta(y_w)/\pi_{\text{ref}}(y_w))$ is the implicit reward of the winner: how much more likely the current policy makes it than the base model did. The loss pushes the winner's implicit reward above the loser's. So DPO increases the **margin** between chosen and rejected — not the absolute likelihood of either. This matters in practice: it is entirely normal for the log-probability of the chosen response to go **down** during DPO training, as long as the loser's goes down faster. A well-known failure mode is both dropping too far, which degrades fluency; that is what the β and the reference anchor are for.

RLHF or DPO?

From Is DPO Superior to PPO for LLM Alignment? A Comprehensive Study (Xu et al., 2024):

	RLHF (PPO)	DPO
Training	multi-stage training	supervised learning
Extra models needed	reward model, value model, base model	base model only

Ease of implementation: DPO, clearly.

Performance: no absolute consensus. It varies from task to task and is sensitive to implementation.

Intuition — the honest state of the field, and the reason for the split. DPO wins on simplicity by a wide margin and is what you should reach for first. But well-tuned PPO-based RLHF often edges it out at the frontier, and there is a structural reason: PPO is **on-policy** — it generates fresh responses from the current policy, scores them, and learns from its own present behaviour. DPO learns from a **fixed, offline** dataset of pairs collected from some earlier model. As DPO training proceeds, the policy drifts away from the distribution its preference data came from, and the data becomes progressively less relevant to what the model now does. This is why a whole family of successors exists (iterative DPO, online DPO, IPO, KTO, ORPO, SimPO), most of which are attempts to reintroduce on-policy freshness without reintroducing the four-model apparatus.

The result, end to end

Tracking the same question through all three stages:

- **Pretrained only:** "Can I put my teddy bear in the washer?" → "Teddy bears are often made of materials like polyester and cotton, with plastic eyes and sometimes small accessories." (a continuation, not an answer)
- **+ instruction tuned:** → "No, it might get damaged. Try hand washing instead." (answers the question)
- **+ preference tuned:** → "It's better not to. Your teddy could get hurt! A gentle hand wash is safer." (answers the question in a tone people preferred)

Intuition — what each stage bought you. Pretraining gave **knowledge** (it knows what teddy bears are made of). SFT gave **format** (a question gets an answer). Preference tuning gave **style and judgement** (warm, concise, appropriately cautious). All three examples are factually consistent — the difference between them is not knowledge but behaviour, which is exactly why alignment is a separate problem from capability.

Lecture 6 — LLM Reasoning

Video length: 1h47. Covered here: what "reasoning" means and how to spot a reasoning model, reasoning benchmarks and the Pass@k / Cons@k metrics, test-time scaling and verifiable rewards, controlling the thinking budget, GRPO in detail and its comparison to PPO, the increasing-output-length pathology and its fixes (DAPO, Dr. GRPO), and the full DeepSeek R1-Zero and R1 training recipes including distillation.

The lecture starts from the honest list of what vanilla LLMs can and cannot do:

Strengths. Great at imitation and idea generation; amazing at generating and debugging code.

Weaknesses. **Limited reasoning** (today's focus); **knowledge is static** and **cannot perform actions** (Lecture 7); **hard to evaluate** (Lecture 8).

6.1 What counts as reasoning

Tentative definition: reasoning = the ability to solve a problem.

The lecture draws the line with two examples:

Not reasoning	Reasoning
"What is the course code of Stanford's Transformers & LLMs class?"	"The bear was born in 2020. How old is this bear now?"

The first is **retrieval**: the answer either is or is not stored in the weights, and no amount of thinking helps. The second requires **combining** given information with other knowledge (the current year) via an operation (subtraction) to produce something that was never stored anywhere.

The core idea

Strategy: teach the model to explain its reasoning before answering — chain of thought (Wei et al., 2022), as introduced in Lecture 3.

Without CoT the exchange is terse and fragile:

```
Q: How old is this bear?           A: 4.
Q: How old will the bear be next year? → A: 5.
```

With CoT in the demonstration, the model imitates the working:

```
Q: How old is this bear?
A: The bear was born in 2020. It is therefore 4.
Q: How old will the bear be next year?
→ A: It will be one year older than its age this year, which was 4.
    Hence, it will be 5.
```

The idea behind reasoning models: do CoT, but at a much larger scale.

The paradigm shift

Until now: Question $\longrightarrow$ LLM $\longrightarrow$ Answer

New paradigm: Question $\longrightarrow$ LLM $\longrightarrow$ Reasoning chain $\longrightarrow$ Answer

Output = Reasoning + Answer

The reasoning is not a prompting trick applied from outside; it becomes part of what the model is trained to produce.

Intuition — restating the depth-into-length argument, because everything in this lecture rests on it. A Transformer with N layers performs a fixed number of sequential computation steps per token. A problem needing more sequential steps than that is structurally out of reach in one forward pass — no amount of extra width or training fixes it. Chain of thought escapes the limit by externalising intermediate state into the token stream: each generated token gets a full forward pass, and previous conclusions are written into the context where attention can retrieve them. The generated text becomes a scratch pad, and the model's effective computational depth becomes $N \times$ (number of tokens it chooses to think for). This is the mechanism behind the phrase **test-time scaling**: you can buy more capability at inference by spending more tokens, without changing the weights at all.

Reasoning model releases

The lecture shows a (non-exhaustive, not-to-scale) timeline of first releases from major labs:

Date	Release
2024-09-12	OpenAI o1-preview
2024-12-19	Gemini 2.0 Flash Thinking
2025-01-20	DeepSeek R1
2025-02-19	Grok 3 Beta
2025-02-24	Claude 3.7 Sonnet
2025-06-10	Magistral

How to spot a reasoning model

Two tells, both practical:

In the product. You see a "**thought summary**" — a paraphrase of the model's thinking — while the **complete chain of thought is usually hidden**.

In the pricing / API docs. OpenAI, Anthropic and Google all expose reasoning-specific parameters and bill for **reasoning tokens** as a separate line item: tokens you pay for but never see.

Intuition — why hide the chain of thought? Several reasons stack up. Commercially, the raw traces are the most valuable training data a lab produces — expose them and competitors can distil your model, which is exactly what section 6.5 describes. Behaviourally, labs have found that models reason better when the trace is not optimised for human consumption; the moment you train the visible thinking to look presentable, you are applying pressure that can degrade its usefulness as computation. And safety-wise, an unfiltered trace can contain the model exploring ideas it will correctly decline to state in its answer. The billing point matters practically: reasoning models can emit thousands of hidden tokens per query, so cost and latency are far higher than the visible output suggests.

6.2 Benchmarks and metrics for reasoning

Coding benchmarks

The structure is problem → solution → verification:

- **Problem:** "You have n teddy bears in a line. Each bear has a size. Find the biggest bear that is smaller than the largest bear."
- **Solution:** the model emits code, e.g.

```
python def second_biggest_bear(bears): largest = max(bears) return max(b for b in bears if b < largest)
```
- **Verification:** run the hidden test cases. All pass → correct.

Examples: HumanEval, CodeForces, SWE-bench.

Maths benchmarks

Structure: problem → reasoning → compare against ground truth:

- **Problem:** "The bear was born in 2020. How old is the bear now?"
- **Reasoning:** "It is 2025 now. Subtract the birth year from the current year: $2025 - 2020 = 5$. Answer: 5"
- **Ground truth:** 5 → verification.

Examples: AIME, GSM8K.

Intuition — why coding and maths, and not essay writing? Because both admit **automatic, cheap, unambiguous verification**. Code either passes the tests or it does not; a numeric answer either matches or it does not. No human, no reward model, no judge. That property is not merely convenient for benchmarking — it is the entire reason reasoning models exist, as section 6.3 makes explicit. The domains where reasoning models are strongest are precisely the domains where a reward can be computed by a script.

Pass@k

"Probability that at least 1 of k attempts succeeds."

Estimated unbiasedly from n sampled attempts of which c succeed:

$$\text{Pass}@k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

i.e. one minus the probability that a random subset of k attempts contains no successful one.

- **Pass@k** — for use cases where **checking is easy** or you can afford higher latency.
- **Pass@1** — the special case, for use cases where you care about **a single generation**.

Cons@k

"Consensus at k" — equivalent to taking the answer from **majority voting** over k samples and comparing that with the ground truth (DeepSeek-R1, 2025).

Intuition — three metrics, three different questions. Pass@k measures whether the ability is present at all — can the model find the answer if given k tries and a perfect verifier? It is the right metric when you have a checker (compile, run tests) and can retry. Cons@k asks whether the model finds the answer reliably enough that its own majority agrees, which is the right metric when you have no verifier and must trust one output; it is exactly Lecture 3's self-consistency turned into a score. Pass@1 is what a user experiences. Reporting Pass@64 alongside Pass@1 is informative: a large gap means the knowledge is in there but not reliably surfaced, which is a training problem rather than a capability ceiling. **Lecture 8 adds a fourth relative, Pass^k** — the probability that all k attempts succeed — which measures consistency instead of capability. Do not confuse Pass@k with Pass^k; they are almost opposites.

6.3 Test-time scaling with RL

Idea: incentivise the model to reason before answering.

Considerations that shape the method:

- **A reasoning chain is hard to write from scratch** — building SFT data by hand is impractical. Nobody wants to hand-write ten thousand step-by-step solutions, and even fewer people can.
- **We don't want to limit the model to human-written reasoning.** Human traces are polished and post-hoc; the model may need to backtrack, try things, and check itself in ways a textbook solution never shows.
- **There is a natural verifiable reward:** "did it solve the problem?" → yes or no.

Conclusion: let's try RL!

Intuition — this is the pivotal argument of the lecture. Recall Lecture 5: the hard part of RLHF was that "good response" is subjective, so you had to train a reward model on human preferences and then constantly guard against hacking it. For maths and code that entire apparatus is unnecessary — the reward is a **fact**, computable by a script, unhackable in principle because there is nothing to approximate. Which means you can run RL at scale, cheaply, with no human in the loop and no reward model to overfit. That is what changed between RLHF and reasoning models. The interesting consequence: nobody tells the model how to reason. You only tell it whether it got the right answer, and the reasoning strategy is discovered.

Reward 1: verify that the CoT is there

Enforce a **template**:

```
<think>
... reasoning ...
</think>
ANSWER
```

and give reward for a response that respects it. Purely structural: are the delimiters present and well-formed?

Reward 2: verify that the solution is correct

- **Code verification** — run test cases 1..c; all → reward.
- **Maths verification** — extract the final answer, compare with ground truth; match → reward.

Total reward

$$\text{Rewards} = \underbrace{\text{formatting}}_{\text{think delimiters?}} + \underbrace{\text{accuracy}}_{\text{correct solution?}}$$

Intuition — why the formatting reward is not just cosmetics. Two jobs. Practically, it makes the answer extractable: you cannot verify accuracy if you cannot find where the answer is, so the format reward bootstraps the accuracy reward. Behaviourally, it **creates the space to think**. By rewarding the presence of a `<think>` block, you are telling the model that emitting reasoning before answering is itself desirable — and once that space exists, the accuracy reward shapes what goes inside it. It is a remarkably cheap piece of scaffolding for the amount of behaviour it unlocks.

Controlling thinking at inference time

Problem: not all prompts are equal. "What is 2+2?" does not deserve four thousand reasoning tokens, and a model that always thinks maximally is unaffordable.

Ideas to control "thinking":

- **Dynamic budget** (Token-Budget-Aware LLM Reasoning, Han et al., 2024) — estimate how many tokens a question warrants and allocate accordingly.
- **Context awareness** — make the model aware of its own remaining budget so it can pace itself.
- **Budget forcing** (s1: Simple test-time scaling, Muennighoff et al., 2025) — crude and effective: to stop early, inject the `</think>` token; to force more thinking, suppress `</think>` and append the word **"Wait"**, which reliably makes the model second-guess and continue.
- **"Continuous" thoughts** (Training LLMs to Reason in a Continuous Latent Space, Hao et al., 2024) — reason in the continuous hidden space rather than by emitting discrete tokens, avoiding the information bottleneck of having to commit to a token at every step.

Intuition. The s1 "Wait" trick is worth pausing on because it is so revealing. Appending a single word measurably improves accuracy, because it pushes the model into a distribution where self-correction follows. It tells you that the reasoning behaviour is a mode the model can be nudged into, not a fixed procedure — and that inference-time control over thinking length is a real and cheap lever. Continuous thoughts are the more ambitious direction: every token in a normal chain of thought compresses a rich hidden state down to one choice from the vocabulary, which is a substantial information loss at every step.

6.4 GRPO

GRPO = Group Relative Policy Optimisation (DeepSeekMath, Shao et al., 2024).

The objective has the same shape as PPO:

$$\text{maximise} \quad \underbrace{\mathbb{E}[A]}_{\text{maximise advantages}} - \beta \underbrace{\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})}_{\text{don't deviate from old/base}}$$

The big difference from PPO:

PPO: Advantage $\approx$ Reward – Value function(state) $\leftarrow$ a trained value network

GRPO: Advantage $\approx$ Reward – Avg(reward of group) $\leftarrow$ no value network at all

Mechanically: for a given prompt, sample a **group** of G responses from the current policy. Score them all. Compute each response's advantage by standardising within the group:

$$A_i = \frac{r_i - \text{mean}(r_1 \dots r_G)}{\text{std}(r_1 \dots r_G)}$$

That advantage is then assigned to **every token** of response i .

GRPO versus PPO, side by side

Similarities: the probability **ratio** between new and old policy, and **clipping** of that ratio. Both inherited directly from PPO.

Differences: the **KL penalty** (GRPO places it as an explicit term in the objective, computed per token against the reference, rather than folding it into the reward signal) and — the important one — **advantage estimation**.

Intuition — why dropping the value model is such a big deal. The value network in PPO exists solely to estimate "what reward should I expect from this state?" so that the advantage can be centred. It is expensive: a second model of comparable size to the policy, trained simultaneously, and notoriously hard to fit well — a bad value estimate poisons every advantage.

GRPO's observation is that if you are going to sample multiple responses to the same prompt anyway (and for reasoning you are, because you want diversity to learn from), then **the group's own average reward is a perfectly good baseline for that prompt** — and an unbiased one, obtained for free. This removes one of PPO's four models and a large slice of its instability. It also makes intuitive sense of what the model learns: "of the eight attempts I made at this problem, these three were better than my average, so do more of that." Automatic difficulty calibration comes along for free — on an easy prompt where all eight succeed, every advantage is zero and nothing is learned, so training compute naturally concentrates on problems at the edge of the model's ability.

This is the same "subtract a baseline to cut variance" idea from Lecture 5, with a Monte Carlo estimate replacing a learned function approximator.

6.5 The increasing-output-length pathology

Observation: response length keeps increasing with RL training. DeepSeek-R1's training curves show average response length growing steadily, and this was initially celebrated as the model "learning to think longer".

The lecture then shows it is partly an **artifact of the loss normalisation**.

The problem. GRPO's loss sums token-level terms and normalises by the number of tokens in the response, $1/|y_i|$. That means each token in a **short** output carries a **large** weight, and each token in a **long** output carries a **small** weight.

Consequence: when a response is bad (negative advantage), a long one is punished less per token than a short one. When a response is good, a long one is rewarded less per token. Net effect on the gradient: **being long is a cheap way to dilute penalties**. That is a **bad incentive** — the model learns to ramble because rambling reduces the downside of being wrong.

Remedy: equalise token-level contributions. Two published fixes:

- **DAPO** (Yu et al., 2025, An Open-Source LLM Reinforcement Learning System at Scale) — normalise by the **total number of tokens across the whole group** rather than per response, so every token in the batch carries identical weight regardless of which response it belongs to.
- **Dr. GRPO** (Liu et al., 2025, Understanding R1-Zero-Like Training: A Critical Perspective) — remove the length normalisation term (and the standard deviation normalisation) altogether, on the grounds that they are the source of the bias.

Intuition — a lesson worth generalising far beyond this case. The length growth looked like the model discovering that thinking longer helps. Some of it was. But a measurable part was the model exploiting an accounting artifact in the loss. This is reward hacking again, except the hacked object is not the reward model but the **normalisation constant in the objective** — a place nobody thought to look. The general lesson: whenever your model develops a surprising behaviour during RL, check whether that behaviour is actually optimal for the objective as literally implemented, rather than for the objective as you intended it.

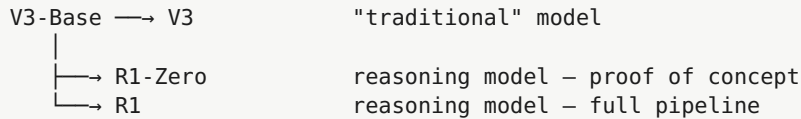
Other adjustments being explored

- **Bias linked to level of difficulty** (Dr. GRPO) — dividing by the group's standard deviation gives questions where responses happen to be similar an outsized gradient, which biases learning towards problems that are uniformly easy or uniformly hard.
- **Encourage diversity** (DAPO) — use asymmetric clipping ("clip-higher"), raising the upper clip bound so that low-probability tokens can be reinforced; without it the policy's entropy collapses and it stops exploring.

...among others! The area is moving fast and the design space is not settled.

6.6 Putting it together: the DeepSeek recipes

The model family, laid out:



R1-Zero's training recipe

Step 1 — pretrain the model with "traditional" techniques: V3-Base. A MoE model, ~671B total parameters, ~37B active per token.

Step 2 — GRPO with reasoning data → R1-Zero. No SFT at all. The exact template used:

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within `<think>` `</think>` and `<answer>` `</answer>` tags, respectively, i.e., `<think>` reasoning process here `</think>`
`<answer>` answer here `</answer>`.
 User: `<this placeholder is replaced by a reasoning query>`
 Assistant:

Benefits

reasoning abilities without any SFT

Challenges

chains of reasoning have **formatting and readability issues**

Intuition — why R1-Zero was the headline result. Everyone had assumed that reasoning had to be taught — that you needed human-written chains of thought to imitate before RL could refine them. R1-Zero showed you can go straight from a base model to a reasoning model with **pure RL on verifiable rewards**, and that sophisticated behaviours (self-verification, backtracking, spontaneously allocating more thinking to harder problems) **emerge** rather than being demonstrated. The reported "aha moment" — the model writing something like "wait, let me reconsider" without ever being shown such a phrase — is the emblematic example.

The cost was legibility: with only formatting and accuracy rewarded, the traces drift into whatever is computationally useful, including mixing languages mid-thought and idiosyncratic notation. Nothing in the reward cared whether a human could read it.

R1's training recipe — the full five-stage pipeline

Step 1 — pretrain with "traditional" techniques: V3-Base (MoE, ~671B total / ~37B active).

Step 2 — "small-scale" SFT with reasoning data. Data source: long chains of thought generated with R1-Zero and then rewritten by humans. A few thousand examples, cleaned for readability.

Step 3 — GRPO with reasoning data. Approximately the same RL process as R1-Zero. **Reward = formatting + accuracy + language consistency.** (Language consistency is the new term, added specifically to stop the language-mixing.)

Step 4 — "large-scale" SFT with reasoning and non-reasoning data. Two pools:

- ~600k pairs of **maths, coding, logic** data, obtained by **rejection sampling** of "R1-so-far" responses — generate many, keep only those that pass rule-based checks or are approved by a V3 judge;
- ~200k pairs of **"general" data**, mostly reusing V3's existing SFT data.

Step 5 — GRPO with reasoning and non-reasoning data → R1. Two reward regimes running together:

- on **maths, coding, logic**: **reward = formatting + accuracy** (verifiable);
- on **"general" data**: **reward = helpfulness + harmlessness**, mostly reusing V3's RL data (i.e. classic RLHF-style preference reward).

Intuition — read the pipeline as a set of repairs to specific problems. Step 2 exists to fix R1-Zero's readability, using R1-Zero itself to produce the raw material — the model bootstraps its own SFT data, which is the only reason hand-writing thousands of chains became unnecessary. Step 3 pushes reasoning ability up from a legible starting point. Step 4 exists because a reasoning-only model is worse at ordinary conversation than the model it came from — pure reasoning RL causes forgetting of general helpfulness — so general capability is folded back in, with rejection sampling ensuring the reasoning data is high quality. Step 5 aligns the whole thing on both axes at once, which is necessary because tuning them sequentially would let each undo the other.

The general principle: **you cannot align one capability at a time.** Each RL stage degrades what the previous stage optimised, so the final stage has to carry every objective simultaneously.

Results

R1's benchmark results were competitive with the leading closed reasoning models of the time, from an openly published recipe — which is why this paper reset expectations across the field.

And what about distillation?

The lecture is careful to distinguish two different things that share the name:

Distillation in Lecture 2	Distillation used here
Goal: match the next-token distribution	Goal: SFT-learn reasoning traces
targeted, token-level, needs the teacher's full logits	R1 generates entire responses ; the student is trained on them as ordinary SFT data

So R1-Distill models (Qwen and LLaMA bases finetuned on R1's outputs) are made by **plain supervised finetuning on R1's generated reasoning traces**.

Results:

- **competitive** — the distilled small models substantially outperform their bases and rival much larger models on reasoning benchmarks,
- a **"good" use of compute** — the paper notes that running RL directly on a small model produces worse results than distilling from a large model that already went through RL.

Intuition — why distillation beats running RL on the small model, which is the most practically useful takeaway in the lecture. RL has to discover good reasoning by trial and error, and discovery requires enough baseline capability to occasionally stumble onto correct solutions — otherwise every sample gets zero reward and there is no gradient at all. Small models rarely get there. But **imitating** a correct reasoning trace is a much easier learning problem than finding one. So the efficient division of labour is: spend the enormous RL compute once on your largest model, let it discover how to reason, then transfer that behaviour to small models by cheap supervised finetuning on its outputs. Note also that this form of distillation needs only the teacher's text, not its logits — which is why it works across model families and architectures, and why it is so hard for labs to prevent.

Lecture 7 — Agentic LLMs

Video length: 1h49. Covered here: why retrieval is needed, the RAG pipeline end to end, building a knowledge base, two-stage retrieval (candidate generation then reranking), semantic vs BM25 vs hybrid search, HyDE and contextual retrieval, retrieval metrics (NDCG@k, RR@k, recall@k, precision@k), tool/function calling, how to teach a model to use tools, tool selection and routing, MCP, agents and the ReAct loop, A2A, and safety.

This lecture addresses two of the four weaknesses from Lecture 6's list: **knowledge is static** and **cannot perform actions**.

7.1 Why retrieval

Four motivations, each independent:

- 1. The knowledge of an LLM is constrained to its pretraining data.** The lecture shows the knowledge cutoff on GPT-5's model card. Anything after that date does not exist as far as the model is concerned, and your own private documents never existed at all.
- 2. Limited context size.** Even a very large window cannot hold a company's entire document store.
- 3. The LLM gets distracted by useless information.** The lecture references the "**Needle in a Haystack**" pressure tests (Kamradt, 2023), which hide a specific fact in a long context and measure whether the model can retrieve it. Performance depends on both the context length and where in the context the fact sits.
- 4. Pricing is per input/output token.** Every irrelevant token you include is money spent, latency added, and attention diluted.

Intuition — points 3 and 4 together are the real argument, and they are counterintuitive. You might think that with a one-million-token window the answer to "how do I give the model knowledge?" is simply "put everything in the prompt". It is not, for two reasons. It costs a fortune per call. And — more importantly — it makes the model **worse**, because attention is a zero-sum-ish resource: softmax mass spent on 900,000 irrelevant tokens is mass not spent on the 100 that mattered. Retrieval is not a workaround for small context windows. It is a way of making the model's attention land on the right place, and it remains valuable no matter how big windows get.

7.2 RAG

RAG = Retrieval-Augmented Generation (Lewis et al., 2020).

Idea: augment the prompt with relevant pieces of information.

The three steps, in the order the acronym does not suggest:

- 1. Retrieve** the relevant document via a similarity operation across the knowledge base.

2. **Augment** the prompt with the retrieved information.
3. **Generate** the response with the LLM, given the augmented prompt.

```
user prompt → [retrieve] → retrieved info + user prompt → LLM → response
```

The bulk of the lecture then focuses on **the retrieval stage**, because that is where all the engineering difficulty lives.

Prerequisite: creating the knowledge base

Three operations, done offline, once:

Collect — gather Document 1, Document 2, ... whatever your corpus is.

Divide — split each document into **chunks**.

Embed — turn each chunk into a vector, e.g. `[0.48, 0.33, ..., -0.51]`, and store it in a vector index.

Hyperparameters: embedding size, chunk size, overlap between chunks.

Intuition — chunking is the most underrated decision in the pipeline. Chunk too **small** and each chunk lacks the context needed to be interpretable: a sentence saying "he was in a cuddly mood" is useless if the paragraph that established who "he" is landed in a different chunk. Chunk too **large** and the embedding becomes a blurry average of several topics, matching everything weakly and nothing strongly — and you waste context on the irrelevant parts you retrieve along with the relevant sentence. **Overlap** exists so that a fact sitting on a boundary is not cut in half; a typical setting is 10-20% overlap. Practitioners consistently find that fixing bad chunking improves a RAG system more than swapping the LLM. Section 7.2's "contextualise document chunks" is a direct attack on the small-chunk problem.

Retrieval: two stages

Step 1 — Candidate retrieval. Select potentially-relevant candidates.

- **Maximise recall.**
- Semantic embeddings, and optionally keyword-based methods.

Step 2 — Ranking. Give the final relevance score.

- **Maximise precision.**
- Re-rank on the smaller set of candidates.

Intuition — why two stages instead of one good one? Cost, and the shape of the cost. Step 1 must touch **every** chunk in the knowledge base, potentially millions, so it must be extremely cheap per item — which means precomputed embeddings and an approximate nearest-neighbour search. Step 2 touches maybe 50 candidates, so it can afford a model that is a thousand times more expensive per item. The division of labour is: cast a wide, cheap, slightly sloppy net that is unlikely to miss the right answer (recall), then apply an expensive, accurate judgement to what you caught (precision). This is the standard architecture of every search and recommendation system ever built.

Step 1, method 1: semantic search with embeddings

Encode the **query** and each **chunk** into vectors and compute a **similarity score** — typically cosine similarity or dot product.

Critically, the query encoder and the chunk encoder produce vectors in the **same** space, and each is encoded **independently**. This arrangement is called a **bi-encoder** (Sentence-BERT, Reimers et al., 2019).

Worked example. Query: "Where is Cuddly?" Candidate chunks:

- "[...] Huggy likes to work downstairs [...]"
- "[...] Where is Paris located? [...]"
- "[...] he was in a cuddly mood [...]"
- "[...] Cuddly spends most days surrounded by books [...]"

Semantic search should rank the last one highest — it is genuinely about the entity Cuddly and its location — but it is also prone to being pulled towards the second (structurally similar question, "Where is ... ?") and the third (the word "cuddly", though used as an adjective).

Intuition — why the bi-encoder architecture is what makes this scalable. Because the chunk embedding does not depend on the query, you can compute all of them **once, offline**, and store them. At query time you embed only the query and do a vector search. If the encoder saw query and chunk together you would have to run the model once per (query, chunk) pair — millions of forward passes per query, which is impossible. That is exactly the trade being made: the bi-encoder is fast because it never lets the query and the document interact, and it is less accurate for exactly the same reason. Step 2 pays for the interaction on a short list.

Step 1, method 2: keyword matching with BM25

BM25 is a classical lexical retrieval function — a refined TF-IDF. It scores a chunk by how many of the query's terms it contains, weighting rare terms more heavily, with saturation on repeated terms and a normalisation for document length.

On the same example, BM25 latches onto the literal token Cuddly, so it ranks both the "cuddly mood" chunk and the "Cuddly spends most days" chunk highly, and correctly ignores the Paris chunk (no shared rare terms).

Step 1, method 3: hybrid

Search based on a hybrid combination of semantics and BM25 — run both, merge the ranked lists (e.g. by reciprocal rank fusion), and pass the union to the reranker.

Intuition — why hybrid is the practical default, framed as complementary failure modes. Embeddings capture meaning and are robust to paraphrase, so they find "canine companion" when you searched "dog". But they are unreliable on exactly the things where precision matters most: rare proper nouns, product SKUs, error codes, function names, version numbers. A semantic encoder has no particular representation for `ERR_4471B` and will happily return `ERR_4472B`. BM25 is the opposite: it is exact on rare tokens and completely blind to synonymy. Their errors are close to uncorrelated, which is the ideal condition for combining two systems. Hybrid retrieval reliably beats either alone, which is why "just use a vector database" is incomplete advice.

Extensions that help initial retrieval

Mitigate the discrepancy in nature of embeddings — HyDE (Precise Zero-Shot Dense Retrieval without Relevance Labels, Gao et al., 2022).

The problem: a **query** and a **document** are different kinds of text. "Where is Cuddly?" is a short interrogative; the answer is a long declarative passage. Embedding them into the same space and comparing is comparing unlike things.

The fix: have the LLM write a **fake document** that answers the query — "Cuddly is in ..." — and embed **that** instead of the query. Now you are matching document-shaped text against document-shaped text.

Intuition. The hypothetical document does not need to be factually correct — it will be a hallucination, and that is fine. It only needs to be stylistically and topically correct, because its job is to land in the right neighbourhood of the embedding space. You are using the LLM's generative fluency as a query-expansion device.

Contextualise document chunks — Contextual Retrieval (Anthropic, 2024).

The problem: a chunk pulled out of a document loses the context that made it meaningful (the pronoun problem from above).

The fix: before embedding, prepend a short generated context to each chunk. The exact prompt used:

```
<document>
{WHOLE_DOCUMENT}
</document>
Here is the chunk we want to situate within the whole document:
{CHUNK_CONTENT}
Please give a short succinct context to situate this chunk within the overall
document for the purposes of improving search retrieval of the chunk. Answer
only with the succinct context and nothing else.
```

Each chunk i becomes Context i + Chunk i , and that is what gets embedded.

The cost problem, and the fix: prompt caching. This prompt includes the **whole document** once per chunk, so a 100-chunk document means passing the full document 100 times — prohibitively expensive at face value. But because the long prefix is identical across all those calls, **prompt caching** applies: the provider stores the computed KV cache for the shared prefix and charges a small fraction for cache reads. The lecture shows the pricing pages where cached input tokens cost roughly a tenth of fresh ones.

Intuition — this is where Lecture 3's PagedAttention pays off commercially. Prompt caching is only possible because KV cache blocks can be shared across requests. Contextual retrieval is a nice illustration of a technique that is obvious in principle, absurd in cost under naive accounting, and entirely practical once you understand the serving stack. Anthropic reported it cutting retrieval failure rates by around a third, and by around a half when combined with BM25 and reranking.

Step 2: ranking with a cross-encoder

The reranker encodes the **query and the chunk together** in a single forward pass and outputs a **relevance score** directly. This is a **cross-encoder** (suggested reading: "Cross-Encoders", SBERT.net).

```
Re-Ranker(user prompt, {chunk d, chunk b, chunk a, chunk c})
→ chunk a (1), chunk b (2), chunk c (3), chunk d (4)
```

Intuition — what the cross-encoder can do that the bi-encoder cannot. Because query and chunk are concatenated and passed through the same attention stack, every query token can attend directly to every chunk token. The model can therefore check term by term whether the chunk answers the specific question, including negation, qualifiers and exact entity matching — things a fixed-length vector inevitably smooths away. The price is that you must run the model once per candidate, and you cannot precompute anything, which is exactly why it is confined to the short list. Adding a reranker is typically the single highest return-on-effort improvement available to a mediocre RAG system.

Quantifying retrieval performance

Setup: evaluate whether the retrieved chunks are relevant. Some chunks are labelled relevant; a ranking is produced; you look at the top k .

Normalised Discounted Cumulative Gain at k (NDCG@ k):

$$\text{DCG}@k = \sum_{i=1}^k \frac{\text{rel}_i}{\log_2(i+1)}$$

$\text{IDCG}@k$ = the same quantity if the ranking were perfect

$$\text{NDCG}@k = \text{DCG}@k / \text{IDCG}@k$$

Reciprocal Rank at k (RR@ k):

$$\text{RR}@k = \frac{1}{\text{rank of the first relevant chunk}} \quad [0 \text{ if none in the top } k]$$

Recall at k : of all the relevant chunks that exist, what fraction appear in the top k ?

Precision at k : of the k chunks retrieved, what fraction are relevant?

Intuition — which metric to use when, because they answer different questions. Recall@ k is the metric for **step 1**: the only unforgivable failure at the candidate stage is that the right chunk is not in the list at all, since nothing downstream can recover from that. NDCG@ k is the metric for **step 2**: it is position-sensitive (the $1/\log_2(i+1)$ discount means rank 1 is worth much more than rank 10) and handles graded relevance, so it captures whether the reranker put the best material first — which matters because of the lost-in-the-middle effect. RR@ k is the right metric when there is exactly one correct answer and you only care how quickly it appears. Precision@ k matters because irrelevant chunks are not free: they cost tokens and dilute attention.

7.3 Tool calling

Motivation

RAG solved **unstructured** knowledge — documents you can chunk and embed. But a great deal of what you need is **structured**: rows in a database, live prices, a calculation, the current time.

You cannot embed a database usefully. What you can do is call a function:

```
def get_data(id, field, ...):  
    # Logic.  
    return result
```

Definition

"Tool calling [...] allows autonomous systems to complete complex tasks by dynamically accessing and [may act] upon external resources." — IBM

The lecture emphasises two phrases in that definition: **autonomous systems** — the model decides, it is not scripted — and **may act upon** — this is not only reading, it includes changing the world.

The example

Without tools:

"Find a bear near me!" → "Sorry, I don't know which bears are near you."

With tools: the model is given a function API and can call it.

The function, `find_teddy_bear.py`, and the three properties the lecture highlights about it:

```

from dataclasses import dataclass
from geopy.distance import geodesic
import requests

@dataclass
class TeddyBearInfo:
    name: str
    distance_meters: float
    mood: str
    message: str

def find_teddy_bear(location: tuple[float, float]) -> TeddyBearInfo:
    """
    Finds the nearest teddy bear to the given GPS coordinates.

    Parameters:
        location: A (latitude, longitude) pair representing the user's
            current location.

    Returns:
        TeddyBearInfo: Information about the nearest teddy bear found.
    """
    user_lat, user_lon = location
    api_url = "https://api.to.teddy.bears.com/v1/closest"
    try:
        response = requests.get(
            api_url,
            params={"latitude": user_lat, "longitude": user_lon},
            timeout=5,
        )
        response.raise_for_status()
        closest_teddy_bear = response.json()
    except requests.RequestException as e:
        raise RuntimeError(f"Failed to fetch teddy bear data from API: {e}")

    bear_lat, bear_lon = closest_teddy_bear["coords"]
    distance = geodesic((user_lat, user_lon), (bear_lat, bear_lon)).meters
    return TeddyBearInfo(
        name=closest_teddy_bear["name"],
        distance_meters=round(distance, 2),
        mood=closest_teddy_bear["mood"],
        message=f"{closest_teddy_bear['name']} is "
            f"{closest_teddy_bear['mood']} and only "
            f"{round(distance, 2)} meters away!",
    )

```

1. It **has a descriptive, well-documented API** — meaningful name, typed signature, and a docstring explaining parameters and return value.
2. It **(optionally) has some backend call** — it reaches out to a real service.
3. It **returns some info** — a structured object.

Intuition — point 1 is the whole ballgame, and it is easy to underestimate. The model never sees the function body. All it sees is the name, the type signature, and the docstring. Those three things are the entire specification from which it must decide whether to call the tool and what to pass. Which means the docstring is not documentation for humans — **it is a prompt**, and it should be written with the same care you would give any other prompt. A function called `proc_dat(x, y)` with no docstring is unusable by an LLM no matter how good the model is. This reframing — "your API surface is prompt engineering" — is the single most actionable idea in the section, and Lecture 8 shows the failure modes that follow from getting it wrong.

How it works — three steps

1. **Let the LLM find the argument for the relevant function call.** Given the available function APIs and the user's message, the model emits something equivalent to: `location = (37.42, -122.17)` with `find_teddy_bear()`.
2. **Make the function call.** Your code, not the model, executes `find_teddy_bear(location)` against the backend, which returns `{"name": "Teddy", ...}`.
3. **Let the LLM deduce a conclusion based on the results.** The JSON is fed back into the model, which writes the natural-language response.

Watch out — the model does not execute anything. This is the most commonly misunderstood point about tool calling. The LLM only emits a structured request — a function name and arguments, typically as JSON, and often produced with guided decoding (Lecture 3) to guarantee it parses. Your application decides whether to honour that request, executes it, and passes the result back. That boundary is where all of your authorisation, validation, rate-limiting and sandboxing must live, because the model's output is untrusted input.

Teaching a model to use a tool

Method 1: via training. Build SFT data of two kinds:

- **Tool prediction** — input: the function API plus the conversation history ("Find a bear near me!"); target: the correct call, `location = (37.42, -122.17)` with `find_teddy_bear()`.
- **Response generation** — input: the API, the request, the emitted call and the backend's response; target: the final natural-language answer.

The lecture shows this generalising across examples: "Find a bear in Paris!" → `location = (48.86, 2.35)` with `find_teddy_bear()`.

Method 2: via prompting. Provide `<function API> + <detailed explanation on how to use it>` in the prompt. Which raises the question: **how do you write such a description?**

One way given in the lecture: use SFT pairs as evaluation, and use a powerful reasoning model to write the description for you. That is, treat the tool description as an artifact to be optimised — have a strong model draft it, score the draft against a held-out set of (request → correct call) pairs, and iterate.

Intuition. This is prompt optimisation with a proper objective function, and it is a genuinely good pattern. Instead of a human guessing at wording, you have a measurable target (does the model pick the right tool with the right arguments on these 200 cases?) and a capable writer (a reasoning model) doing the drafting. Note that modern frontier models are already trained for tool use, so Method 2 is what you will actually do; Method 1 matters when you are adapting an open model or have unusual tools.

Common use cases

- **Information** — web/database search; weather, stocks and other trackers; codebase search.
- **Computation** — calculator; code execution, often Python.

- **Action** — send emails/messages and other in-computer actions; anything else within an assistant's domain.

Tools summary

Benefits:

- LLMs just became **way more useful**,
- they can **interact with the real world**,
- it **overcomes the "knowledge cutoff"** limitation.

Challenges (Automatic Tool Selection to Reduce Large Language Model Latency, Robert et al., 2024):

- **more tools = decreased performance**,
- **finite context length: not scalable** — every tool definition consumes prompt tokens,
- **many tools to define. Lots of work.**

Tool selection / routing

Goal: both reduce latency and improve performance.

Insert a **router** before the LLM:

```
"Find a bear near me!" → Router → list of selected tools
                                   ↓
                               <selected function APIs> + prompt → LLM → ...
```

Intuition — why more tools makes a model worse, and why routing fixes it. With fifty tool definitions in the prompt you have (a) consumed thousands of tokens before the user's question even appears, (b) created many near-duplicate options between which the model must discriminate, and (c) diluted attention across a large amount of boilerplate. Performance degrades for the same reason that a bloated RAG context degrades performance. A router — often a small classifier or an embedding search over tool descriptions — narrows fifty tools to three before the expensive model ever sees them. Note that this makes the router a new single point of failure, which is exactly the first failure mode Lecture 8 enumerates.

Standardisation: MCP

Motivation: avoid duplication of tool implementations. With m LLM applications and n tools you have $m \times n$ bespoke integrations to write and maintain.

MCP = Model Context Protocol (Anthropic, 2024). **Idea: connect tools and data to LLMs in a standard way** — turning $m \times n$ into $m + n$.

Architecture:

- **MCP host** — the application the user interacts with (e.g. Claude Desktop), containing an **MCP client**,
- **MCP server** — exposes **tools, prompts** and **resources**,
- the client and server speak the protocol.

The lecture's example. Host: Claude Desktop. User prompt: "Recommend a new poetry book to my teddy bear." A **book provider MCP server** exposes a `Find` capability backed by `find_title` over a personal collection resource, and a `Recommend` capability backed by `recommend_taste` over a top books resource. Claude Desktop discovers and calls these through the MCP client.

Intuition — this is USB for LLM tooling, and the analogy is precise. Before USB, every peripheral needed its own port and its own driver per operating system. MCP standardises three things: how a client **discovers** what a server offers, how it **invokes** a capability, and how results come back. Once a tool speaks MCP, every MCP-capable host can use it with no bespoke work — which is why adoption was rapid across the industry. Note that MCP covers more than tools: **resources** (data the model can read) and **prompts** (reusable templates the server provides) are first-class, so a server can ship an entire curated way of working with a system rather than just a function list.

7.4 Agents

Definition

"An agent is a system that autonomously pursues goals and completes tasks on a user's behalf."

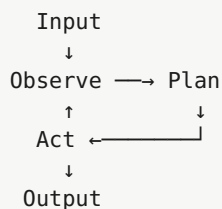
The progression

Traditional:	Question → LLM → Answer
Reasoning:	Question → LLM → Reasoning → Answer
Agent:	Question → LLM → Calls → LLM → ... → Answer

The defining feature of the third row is **the loop**: the number of LLM calls is not fixed in advance. The system decides when it is done.

ReAct = Reason + Act

Yao et al., 2022. The loop has four positions plus an exit:



Walking the lecture's worked example in full:

Input. "My teddy bear is cold. Please do something." Inputs can be **manually entered** (a user question) or come from an **external event** (a metric crossing a threshold).

Observe. "The user's teddy bear is cold, which may be due to the current temperature of the room, which is currently unknown." This step **synthesises previous actions and explicitly**

states what is currently known, including the model's own knowledge. It is a **reasoning-heavy step to figure out what is needed** — note that it identifies the gap, which is what makes the next step possible.

Plan. "Determine the temperature of the room." **Detail what tasks need to be accomplished and what tools to call.**

Act. `get_current_room_temperature()` Either **perform an action via an API** or **look for info in a database of documents** (i.e. RAG is just one kind of act).

→ **back to Observe.** "The temperature in the room is currently 65F. This is about 5F less than an average temperature. We need to increase the room temperature."

Plan. "Increase the temperature by 5F."

Act. `increase_temperature(value=5)`

→ **Observe.** "The thermostat is now set to 70F. This should be warm enough."

Output. "The thermostat is now set to 70F. Your teddy bear will soon feel warmer."

From the outside, all of that internal machinery is invisible — the user sees a request go in and a result come out. That packaging is the **"agentic" view**: a **thermostat agent**.

Intuition — why the loop must include Observe, and what actually makes this hard. The naive version of tool use is one-shot: decide a call, make it, answer. That fails the moment a task needs more than one step, or the moment a tool returns something unexpected. The Observe step is where the agent **incorporates reality** — including reality disagreeing with the plan. It is what allows recovery from a failed call, refinement after partial information, and knowing when to stop.

The hard engineering problems are all consequences of the loop. **When to terminate**, since a model that never concludes will loop forever and burn money. **Context growth**, since every observation is appended, so long tasks hit the window and suffer context rot. **Error compounding**, since a 95% reliable step run ten times gives you a 60% reliable task. Lecture 8's entire failure-mode taxonomy exists because of these.

Multi-agent systems and A2A

There can be agents for many things: a **thermostat agent**, an **occupancy agent**, an **air quality agent**, an **energy management agent**. Naturally the question arises: **how do agents communicate with each other?**

A2A = Agent2Agent (Google, 2025) — a protocol for exactly that. Its core objects:

- **AgentSkill** — one capability, with an `id`, a `description`, and `examples`. E.g. `id='maintain_comfort'`, `description='Maintain a comfortable T'`, `examples=['Set to focus mode while I'm working'];` or `id='prepare_home'`, `'Sets T to target by arrival'`, `examples=['Warm up the house to 70F when I am back'];` or `id='optimize_energy'`, `'Reduces energy consumption'`, `examples=['Save energy overnight']`.
- **AgentCard** — the agent's public advertisement: `name='thermostat agent'`, `url='http://path.to.agent'`, `version='1.0.0'`, `skills=[...]`.
- **AgentExecutor** — the implementation side, with `async def execute(...)` and `async def cancel(...)`.

Intuition — MCP and A2A are not competitors, they are different layers. MCP connects an agent **downwards** to tools and data. A2A connects agents **sideways** to each other. The AgentCard is the interesting design choice: it is a discovery document, so an agent can find out at runtime what another agent is able to do, in natural language, and delegate to it. Which means the "tool" an agent calls may itself be another agent that plans and loops. Note the presence of `cancel` — long-running autonomous tasks need to be interruptible, a requirement that does not arise for simple function calls.

Safety

Risks:

- **potential for harm in the real world,**
- **example: data exfiltration** (ToolSword, Ye et al., 2024).

Remediations:

- **training steps** (Towards Tool Use Alignment of LLMs, Chen et al., 2024),
- **inference safeguards,**
- **benchmarks,** e.g. Agent-SafetyBench.

...a very important topic! And the lecture makes the point vivid by noting that just yesterday in the news: Anthropic's report on **disrupting the first reported AI-orchestrated cyber espionage campaign** (2025).

Intuition — why agents are a categorically different security problem. A chatbot that is manipulated produces bad text. An agent that is manipulated produces bad actions — sends the email, executes the transfer, deletes the records. And the attack surface is unusual: **prompt injection** means that any content the agent reads is potential instruction. A malicious sentence hidden in a retrieved document, a web page, or an issue comment can redirect the agent, because the model has no reliable way to distinguish "data I was asked to process" from "instructions I was given". Data exfiltration is the canonical version: text in a document tells the agent to look up something private and include it in a URL it fetches. The structural mitigation is not smarter models but **least privilege** at the execution boundary — the agent should not have the ability to do the harmful thing.

Closing thoughts of the lecture

- **Hallucination is a (big) problem.**
- **Reasoning abilities are a bottleneck** — finetuning helps, but it is hard; new capabilities are very welcome.
- **Evaluation is challenging.**
- **Good to start simple, then iterate and progressively scale up.**
- **Good to start with capable models, optimise on size later.**
- **Transparency / observability helps with user trust and debuggability.**

And a personal note from the instructors: their favourite use case for AI agents in daily life is **coding**.

Intuition — the last three deserve to be read as engineering advice, because they contradict what people instinctively do. "Start simple" means: try a plain prompt before RAG, try RAG before tools, try tools before an agent loop. Each step up adds failure modes multiplicatively, and most problems do not need the top of the ladder. "Start with capable models, optimise size later" means: prove the task is achievable with the best model available, then work downwards to something cheaper — because if the strongest model cannot do it, your prompt or your task decomposition is the problem, and you will waste days debugging a weak model. "Observability" means: log every step of every loop. When an agent fails, the question is which of eight steps went wrong, and without traces that is unanswerable — which is precisely the taxonomy Lecture 8 builds.

Lecture 8 — LLM Evaluation

Video length: 1h49. Covered here: what "evaluation" means and why human rating — though the gold standard — does not scale, inter-rater agreement (Cohen's/Fleiss' kappa), rule-based metrics (BLEU, ROUGE, METEOR) and their limits, LLM-as-a-judge including its biases and best practices, enforcing output format, factuality decomposition, the full taxonomy of agentic failure modes, and benchmarks (MMLU, AIME, PIQA, SWE-bench, Harm-Bench, τ -bench) with the Pass^k metric, Pareto frontiers, data contamination and Goodhart's Law.

This lecture addresses the last of Lecture 6's four weaknesses: **hard to evaluate**.

8.1 What "evaluation" means, and why it is hard

The word covers two quite different things:

Output quality — instruction following, coherence, factuality, etc. **System performance** — latency, pricing, reliability.

Today's focus is output quality. (System performance is a normal engineering problem; output quality is the hard one.)

Context: an LLM generates free-form text, which is hard to evaluate. There is no single correct output, so there is nothing to compare against exactly.

The ideal: human rating

Human rating is closest to truth and is the "gold" standard. Generate outputs with the LLM, have humans rate them, done.

Why that does not work

Limitation 1 — the subjectivity of the evaluation task.

Consider: prompt "What birthday gift should I get?"; model response "A teddy bear is almost always a sweet gift — just pick one that feels right to you."; criterion **usefulness**. Rater 1 says it is useful (warm, actionable enough). Rater 2 says it is not (vague, no real recommendation). Both are defensible.

So you measure **agreement between raters** — but naive observed agreement is misleading, because two raters who both say "useful" 90% of the time will agree 81% of the time **purely by chance**.

The metric: "How much better is our agreement than what we'd expect just by chance, given how the raters actually use the categories?"

$$\kappa = \frac{p_{\text{observed}} - p_{\text{expected}}}{1 - p_{\text{expected}}}$$

Variants: Cohen's kappa (two raters), **Fleiss' kappa** (many raters), **Krippendorff's alpha** (arbitrary numbers of raters, missing data, and ordinal or interval scales).

(Cohen, 1960; Fleiss, 1971.)

Limitation 2 — slow. Limitation 3 — expensive.

Intuition — why kappa is the number to demand, and how to read it. The numerator is the agreement you achieved above chance; the denominator is the agreement that was available above chance. So $\kappa = 0$ means you did no better than coin-flipping and $\kappa = 1$ means perfect. Rough conventions: below 0.4 is poor, 0.4–0.6 moderate, 0.6–0.8 substantial, above 0.8 excellent.

Here is the practically vital consequence. If your **humans** cannot achieve a decent kappa on a criterion, then that criterion is **not well enough defined to evaluate anything with** — and no automated judge can rescue it, because there is no ground truth to approximate. Low kappa is not a rater problem, it is a rubric problem: the fix is to sharpen the definition, decompose the criterion, or move to a binary scale. This is why kappa appears here, before any automated method: it is the tool that tells you whether your evaluation question is even answerable.

8.2 Rule-based metrics

Idea: write the labels once, and use them as references. Have humans produce reference outputs, then compare each new model output against those references with a deterministic function.

Common rule-based metrics:

- **METEOR** — Metric for Evaluation of Translation with Explicit ORdering (Banerjee et al., 2005). Unigram matching with stemming and synonym matching, plus an explicit penalty for word-order fragmentation.
- **BLEU** — BiLingual Evaluation Understudy (Papineni et al., 2002). Precision-flavoured n-gram overlap with a brevity penalty.
- **ROUGE** — Recall-Oriented Understudy for Gisting Evaluation (Lin et al., 2004). Recall-flavoured n-gram overlap. **Many variants** — ROUGE-N (n-gram overlap), ROUGE-L (longest common subsequence), and others.

Limitations

1. Does not take stylistic variation into account. The lecture's demonstration — three sentences that mean essentially the same thing:

- "A plush teddy bear can comfort a child during bedtime."
- "Soft stuffed bears often help kids feel safe as they fall asleep."
- "Many youngsters rest more easily at night when they cuddle a gentle toy companion."

Judged by n-gram overlap, sentences 1 and 3 share almost nothing, so a model producing 3 against reference 1 would be scored as a failure. It is a perfectly good answer.

2. Correlation with human rating is not that great.

3. Still requires human ratings! You need reference outputs, so you have not escaped the cost — only moved it from "rate every model output" to "write a reference for every input".

Intuition — the structural flaw. These metrics measure **surface form overlap** while we care about **meaning**. That was tolerable for machine translation in 2002, where outputs were short, constrained, and multiple references could be collected. It falls apart entirely for open-ended generation, where the space of good answers is effectively unbounded and n-gram overlap with any particular reference is close to meaningless. Note that they are not worthless: they are cheap, deterministic and reproducible, which makes them reasonable for regression testing (did today's build get worse than yesterday's?) even when they are useless as absolute quality measures.

8.3 LLM-as-a-Judge

LaaJ = LLM-as-a-Judge (Zheng et al., 2023, Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena).

Idea: use an LLM to rate the quality of a response.

The inputs and outputs:

Prompt	Model response	Criteria
"Why are teddy bears comforting?"	"They are comforting because they are soft and loyal companions."	Relevance
	↓ LLM-as-a-Judge	
Rationale "Direct explanation, is on topic"		Score PASS

An example judge prompt:

```
Evaluate how relevant the model's answer is to the user's prompt.

Prompt: {prompt}
Model Response: {model_response}

Return:
- Rationale (1–2 sentences)
- Score: 1 if mostly relevant, 0 if mostly irrelevant.
```

Enforcing the output format

A judge whose output cannot be parsed is useless, so this is where **guided decoding** from Lecture 3 slide 65 comes back — and the lecture shows that OpenAI, Gemini and Anthropic all document structured-output support.

In practice, two steps:

Step 1. Define the desired output structure.

```
class Response:
    rationale: str
    score: Literal[0, 1]
```

Step 2. Pass it as part of the model call.

```
response = client.responses.parse(
    model=model,
    input=input,
    text_format=Response,
)
```

Benefits compared to what came before

- **no need for a reference / label** — this is the big one; you have escaped the human-written-reference bottleneck entirely,
- **interpretability via rationales** — e.g. "The response does not contain any grammatical mistakes. It also is clear, easy to understand and is helpful with respect to the task."

The two main variations

Pointwise. "Evaluate the quality of: Response" → "Very good".

Pairwise. "Which one is better: Response A or Response B?" → "Response A".

Intuition — same trade-off as preference data in Lecture 5, and the same resolution.

Pairwise is more reliable, because relative judgements are easier and need no absolute scale. Pointwise is what you need for monitoring, because you cannot pair every production response against something. In practice: pairwise for model selection and A/B comparisons, pointwise with a crisp pass/fail rubric for continuous evaluation.

The three biases — each with a symptom and a remedy

Position bias. Symptom: ask "Which is better: A or B?" → the judge says **A**. Ask the same question with the order swapped, "Which is better: B or A?" → the judge says **B**. It is preferring the position, not the content. Remedy: **"take the average"** — run both orderings and combine (a genuine preference should survive the swap; disagreement means a tie) — or **tweak the position embeddings**.

Verbosity bias. Symptom: Response A is "short and correct"; Response B "goes into details, a lot of which are not needed and is overall not very helpful". The judge picks **B**, because it is longer. Remedy: **explicit guidelines**, **few-shot** examples demonstrating that concise-and-correct beats long-and-padded, and/or a **penalty on output length**.

Self-enhancement bias. Symptom: Response A is a human-curated answer that perfectly answers the question; Response B was generated by the same model that is now acting as judge. The judge picks **B** — its own output. Remedy: **don't use the same model as judge and as generator**.

Intuition — where these biases come from, which tells you how much to trust the fixes.

They are not bugs in the judge model; they are consequences of how it was trained. **Position bias** comes from autoregressive structure and from whatever ordering regularities existed in its preference training data. **Verbosity bias** is inherited straight from RLHF: human raters systematically prefer longer, more detailed answers (they read as more effortful and authoritative), so the reward model learned it and the policy learned it, and now the judge exhibits it. **Self-enhancement bias** arises because a model assigns higher likelihood to text in its own style, and likelihood correlates with its notion of quality.

Which means the remedies are mitigations, not cures. Averaging over positions genuinely fixes position bias, because it is symmetric. Prompt instructions reduce verbosity bias but do not eliminate it, since you are fighting a learned prior. Using a different model for judging is the only real fix for self-enhancement — and note it also implies you should be suspicious of any evaluation where a lab judges its own model with its own model.

Best practices

The consolidated list:

- **crisp guidelines,**
- **binary scale over more granular ones,**
- **write the rationale before outputting the score,**
- **mitigate biases** (position, verbosity, self-enhancement, etc.),
- **calibrate with human judgements,**
- **low temperature for reproducibility.**

Intuition — why each of these, since they are not arbitrary.

Binary over granular: exactly the argument from section 8.1. If humans cannot reliably distinguish a 6 from a 7, the judge's choice between them is noise dressed up as measurement. Collapsing to pass/fail throws away nothing real and makes the metric stable. If you need nuance, use several binary criteria rather than one fine-grained scale.

Rationale before score: this is chain of thought from Lecture 3, applied to judging. If the score comes first, the rationale is a post-hoc justification of an already-committed answer and cannot influence it. If the rationale comes first, the reasoning is in the context when the score is produced, and it genuinely conditions the outcome. The ordering in the prompt is doing real work.

Calibrate with human judgements: this closes the loop. You keep a human-labelled set, measure the judge's agreement with it (kappa again), and only trust the judge on criteria where that agreement is high. The judge is a cheap approximation of humans, and you must periodically check the approximation.

Low temperature: an evaluation metric that returns different numbers on identical inputs is not a metric. Note the Lecture 3 caveat that even $T = 0$ is not perfectly deterministic in a batched serving stack.

The revised workflow

```

LLM → LLM-as-a-Judge (fast, cheap, broad coverage)
    ↳ Human ratings (slow, expensive, used to calibrate the judge)
  
```

You do not replace humans; you use them sparingly, to validate an automated judge that then runs at volume.

Typical dimensions to evaluate

Task performance: usefulness, factuality, relevance. **Alignment:** tone, style, safety.

Focus on factuality

Objective: quantify the factuality of an output. Take:

"Teddy bears, first created in the 1920s, were named after President Theodore Roosevelt after he proudly wanted to shoot a captured bear on a hunting trip."

Open question: how should we quantify the nuance? This passage is not simply true or false. Part is wrong (teddy bears date from 1902, not the 1920s), part is right (they are named after Roosevelt), and part inverts the actual story (Roosevelt refused to shoot the captured bear — that refusal is the whole origin of the name).

The method (Long-form factuality in large language models, Wei et al., 2024): **decompose into atomic facts**, verify each, and aggregate with weights.

Atomic fact	Weight w_i
Teddy bears were first created in the 1920s.	0.3
Teddy bears were named after President Theodore Roosevelt.	0.4
Theodore Roosevelt was on a hunting trip where a bear was captured.	0.2
Theodore Roosevelt proudly wanted to shoot the captured bear.	0.1

where w_i is the **importance of the fact**, and

$$\text{score} = \sum_i w_i \cdot \mathbb{1}[\text{fact } i \text{ is correct}]$$

Here that yields **score = 0.60**.

Intuition — why decomposition is the right move, and it generalises. Asking "is this paragraph factual?" is unanswerable when it is partly factual, and any single score you assign is arbitrary. Splitting into atomic claims turns one impossible judgement into several tractable ones — each atomic claim is checkable, against a search engine or a knowledge base — and makes the result interpretable, since you can point at exactly which claim failed. The importance weights matter because not all errors are equal: getting the decade wrong is a detail, while inverting Roosevelt's refusal into eagerness reverses the meaning of the story. Note the general lesson, which applies well beyond factuality: **when a judgement is too coarse to make reliably, decompose it into judgements that are not.** This is the same instinct as "binary over granular scales" and "several criteria instead of one".

8.4 Evaluating agents: the failure-mode taxonomy

Back to the **ReAct** loop from Lecture 7 — with the observation that **a typical agentic call sees multiple loops** through Observe → Plan → Act.

And back to the three tool steps:

1. Let the LLM find the argument for the relevant function call.
2. Make the function call.
3. Let the LLM deduce a conclusion based on the results.

Each of the three can fail, in characteristic ways. This taxonomy is the most directly practical content in the lecture.

Failure class 1: tool prediction errors (step 1)

(a) Does not use the tool. Symptom: the LLM directly issues a response — "Sorry, I don't know where I can find one." — despite having the tool available. Potential causes: **tool router error** (the router never surfaced the tool); **the model doesn't know how to use the tool**. Remedies: **retrain the tool router; SFT-train the model, or better adjust the prompt associated with the target API**.

(b) Hallucinates a tool. Symptom: the LLM calls a tool that doesn't exist — `location = (37.42, -122.17)` with `find_bear()` when the real function is `find_teddy_bear()`. Potential causes: **the model is too weak; API naming is not logical; instructions are unclear**. Remedies: **upgrade the model; revamp the API; iterate on the top-level instructions**.

(c) Uses the wrong tool. Symptom: the LLM picks another available tool — e.g. `send_message(recipient=LocalBusiness, message="may I buy a teddy bear?")` instead of searching. Potential causes: **tool router error; the model chose the wrong tool**. Remedies: **retrain the tool router; SFT-train the model or better adjust the prompt associated with the target API**.

(d) Infers the wrong argument. Symptom: the right tool, the wrong input — `location = (0, -0)`. Potential causes: **the argument cannot be inferred** (the agent was never told the user's location); **the model doesn't know how to use the tool**. Remedies: **introduce a helper tool and/or ensure the context carries the right information; SFT-train the model or better adjust the prompt**.

Intuition — (b) and (d) are the two most instructive. Hallucinating `find_bear()` instead of `find_teddy_bear()` is what happens when your API naming is not the obvious name for the concept: the model generates the name it would expect, and if that is not the name you chose, you get a phantom call. Rename the function to what a competent person would guess. And (d)'s first cause — "the argument cannot be inferred" — is not a model failure at all. If the agent has no way to know the user's coordinates, no amount of prompting fixes it; you need a `get_user_location()` tool, or the location in context. Diagnosing "the model is dumb" when the real answer is "the information was not available" is the most common wasted debugging session in agent development.

Failure class 2: tool call errors (step 2)

(a) Wrong response. Symptom: the tool returns a wrong value or an error —

`find_teddy_bear(location) → ValueError: could not...` Causes: the tool returns an incorrect value; the tool errors instead of returning a value. Remedy: **fix the tool implementation!** (The lecture notes that sometimes errors could be legitimate, though that is not standard practice.)

(b) No response. Symptom: the tool returns nothing at all. **Often seen with: the final response is hallucinated.** Cause: either a bug, or a "feature" of the tool implementation. Remedies: **return something, even if it's an empty JSON; emit meaningful tool outputs as a general rule.**

Intuition — (b) deserves special attention because of its symptom. When a tool returns nothing, the model is left with a gap in its context where an observation should be. And a language model's default response to a gap is to **fill it plausibly** — so you get a confident, entirely fabricated answer, and nothing in the trace looks like an error. This is the worst class of bug: silent, invisible, and downstream of a place nobody is looking. Returning `{}` with a message like `{"status": "no results found"}` converts an invisible hallucination into an explicit observation the model can reason about honestly.

Failure class 3: response generation errors (step 3)

Wrong response. Symptom: the final response doesn't convey the tool's response — the tool returned `{"name": "Teddy", ...}` and the model says "Didn't find any bear!" Potential causes: **the model lacks grounding capabilities; the tool response spams the context window; the tool response does not convey information meaningfully.** Remedies: **upgrade the LLM in charge of synthesising the response; trim the information returned by the backend; make the tool output format descriptive.**

Summary of the taxonomy

```

step 1: <function APIs> + prompt → LLM → call
        failures: does not use / hallucinates / uses the wrong tool,
                  infers wrong argument

step 2: call → Backend → result
        failures: wrong response, no response

step 3: result → LLM → response
        failure:  wrong response

```

Common issues: takeaway

Modeling:

- weak reasoning or grounding capabilities,
- **too much going on in the context window,**
- tool modelling isn't right.

Tool:

- the tool itself has a problem,

- the output of the tool isn't interpretable.

...special care and patience is needed to debug and fix issues!

Intuition — the pattern across the entire taxonomy. Read the remedies again and notice how few of them are "use a better model". Most are: **fix your API names, fix your docstrings, fix your tool's return format, put the needed information in context, return something instead of nothing.** Agent reliability is overwhelmingly an interface design problem, not a model problem. The corollary is that debugging requires **traces**: you must be able to see which of the three steps failed, because the remedies are completely different. This is why Lecture 7 closed on observability.

8.5 Benchmarks

The lecture organises common benchmarks into four categories, then adds agents.

	Knowledge	Reasoning	Coding	Safety
what it tests	accurately state facts about the world; more "breadth" than "depth"; reflects pre-training quality	solve multi-step problems; includes "maths" and "common sense" sub-fields; internal reasoning	generate syntactically correct code; tests programming proficiency; proxy for tool-use abilities	prevent harmful, toxic, inappropriate behaviour; surface vulnerabilities before deployment; alignment with custom preferences
ex-ample	MMLU	AIME, PIQA	SWE-bench	HarmBench

MMLU — Massive Multitask Language Understanding

Hendrycks et al., 2020. **4 possible choices per question, across 57 tasks** spanning elementary mathematics, US history, computer science, law, and more. **Evaluation criteria: find the right choice among A/B/C/D — a hardcoded match.**

AIME — American Invitational Mathematics Examination

~30 maths problems, spanning geometry, algebra and analysis, requiring thorough reasoning. **Evaluation criteria: give the right 3 digits — hardcoded match.** (AIME answers are always integers from 000 to 999, which is exactly why it is convenient as a benchmark.)

PIQA — Physical Interaction: Question Answering

Bisk et al., 2019. **2 possible choices per question** on everyday situations anchored in physics; **approximately 20,000 examples.** **Evaluation criteria: find the right choice among Sol1/Sol2 — hardcoded match.**

SWE-bench — SoftWare Engineering benchmark

Jimenez et al., 2023. **2,294 software engineering problems from real GitHub issues across 12 popular Python repositories.** Each problem provides **a base commit and an**

already-merged PR with tests. Evaluation criteria: the generated PR passes all test cases — hardcoded match.

Intuition — why SWE-bench became the benchmark everyone cites. It has the three properties you want and almost never get together. It is **real** — actual issues from actual repositories, not synthetic puzzles. It is **automatically verifiable** — the maintainers' own tests decide, with no judge and no ambiguity. And it is **agentic** — solving an issue requires navigating a large codebase, reading files, understanding context and editing multiple places, which is why the lecture calls coding a "proxy for tool-use abilities". It is the closest thing the field has to an end-to-end measure of useful autonomous work.

HarmBench — Harmful Behavior Benchmark

Mazeika et al., 2024. **510 unique harmful behaviours** (400 text-based, 110 multimodal), split into "**Standard**", "**Copyright**", "**Contextual**" and "**Multimodal**", with the criterion that they violate laws and widely-held norms. **Evaluation criteria: attack success rate (ASR)**, computed by a **classifier** (not a hardcoded match, because judging whether a response is harmful requires a model).

τ -bench — Tool-Agent-User Interaction Benchmark

Yao et al., 2024. A given set of **database schema, APIs and policies** across two domains:

- **Airline agent** — 500 users, 300 flights, 2000 reservations; ~10 tools and 50 tasks.
- **Retail agent** — 500 users, 50 products, 1000 orders; ~10 tools and 115 tasks.

Evaluation criteria: maximise reward and pass^k .

Pass^k — **a metric for consistency/reliability**

"Probability that ALL k attempts succeed."

Pass^k = probability that *every one* of k independent attempts succeeds

Watch out — Pass^k (caret) is not $\text{Pass}@k$ (at). They are near-opposites. $\text{Pass}@k$ = **at least one** of k succeeds — it measures **capability**, and it increases with k . Pass^k = **all k succeed** — it measures **consistency**, and it decreases with k .

Intuition — why agents need the second one. For a coding assistant with a test suite, $\text{Pass}@k$ is the honest metric: you can retry, and one success is a win. For an agent operating on the real world — booking flights, issuing refunds — retrying is not free and inconsistency is itself the failure. An agent that books the correct flight 80% of the time and a wrong one 20% of the time is unusable regardless of how good its best case is. Pass^k exposes exactly this: an agent with 80% per-attempt success has $\text{Pass}^5 \approx 0.33$, which is a fair description of how it will feel to depend on. τ -bench chose this metric deliberately, and reported numbers on it were sobering.

Profile: what benchmarks actually tell you

Role of benchmarks:

- **a projection of performance along a given axis,**
- **different models may be good at different things.**

The lecture illustrates with practitioners' empirical "facts": Claude's Sonnet models — coding; Gemini's Flash models — cheap; and so on. And with Gemini 3's launch (three days before the lecture), whose announcement is broken down along **reasoning, coding, tool use, knowledge**.

The Pareto frontier

Definition: a Pareto curve is the set of solutions that "optimises" a trade-off — the points where you cannot improve one axis without giving up on another.

Possible trade-offs:

- quality vs. cost/latency,
- quality vs. safety,
- quality vs. context length.

Intuition — this is the shape in which model choice should actually be made. "Which model is best?" is not a well-posed question. "Which models are on the Pareto frontier for quality versus cost?" is, and the answer is usually a handful — everything else is dominated (something is both better and cheaper, so there is never a reason to pick it). Then you choose a point on the frontier according to your own budget. Note the quality-versus-safety axis: a model that refuses less is more useful and more dangerous, so where you sit is a product decision, not a technical one.

Data contamination

Problem: a benchmark's clues may be contained in the training set. The benchmark questions and answers are on the public internet, which is where pretraining data comes from — so a high score may reflect memorisation rather than capability.

Precautions:

- **use an identifier such as a hash** — Google's BIG-bench embeds a canary string in its files so that scrapers can be detected and dataset authors can ask for exclusion,
- **for tools, use a blocklist** — an agent with web access will otherwise simply look the answer up (Gemini 3's evaluation methodology documents doing this),
- **evaluate on newer test versions** — use, say, AIME 2025 for a model whose training cutoff precedes it.

Intuition. Contamination is not usually deliberate cheating; it is nearly unavoidable when you train on the whole web. The consequence is that a benchmark **degrades over time**: the day it is published it measures capability, and a few years later it partly measures memorisation. This is why the field keeps having to build new benchmarks, and why evaluation on freshly minted problems (this year's competition, this month's GitHub issues) is so much more informative than a strong score on a five-year-old suite. Note the interaction with Lecture 4's "train on the test task" recommendation: the goal in both cases is to make sure you are measuring the ability rather than the exposure.

Limitations, and the final word

"When a measure becomes a target, it ceases to be a good measure." — **Goodhart's Law** (Strathern, 1997)

Lessons:

- **should not over-index on benchmarks,**
- **need ~organic perspectives to complete the picture: Chatbot Arena,**
- **...just try a few models out yourself!**

Intuition — Goodhart's Law is the connective tissue of this whole course, and it is worth collecting the instances. Optimise against a reward model and you get reward hacking (Lecture 5). Optimise against a length-normalised GRPO loss and you get rambling (Lecture 6). Optimise against MMLU and you get models tuned for four-option multiple choice. Optimise against human preference votes and you get verbosity and sycophancy (Lectures 4 and 8). It is the same failure every time: the metric was a proxy for what you wanted, and sufficiently hard optimisation finds the gap between proxy and goal.

The practical defences are the ones the lecture lands on. Use **many** uncorrelated metrics, so that gaming all of them simultaneously is harder than actually improving. Keep **held-out, freshly-generated** evaluations that no training loop has ever seen. Keep **humans in the loop** at low volume as a calibration anchor. And **try the models yourself** on your own real tasks — which sounds unscientific but is the only measurement guaranteed not to have been optimised against.

Lecture 9 — Current Trends

Video length: 1h52. Covered here: a full recap of the quarter, Transformers applied beyond text (Vision Transformer, vision-language models, diffusion transformers), diffusion LLMs as an alternative to autoregressive decoding, cross-pollination between modalities, open foundational questions, hardware directions, current and expected applications, ongoing challenges, and how to stay current.

9.1 Recap of the quarter

The lecture walks back through all eight previous lectures. Consolidated:

Lecture 1 — Transformers. Tokenising `A cute teddy bear is reading.`, embeddings, the query/key/value attention mechanism, and the full encoder-decoder architecture from Attention Is All You Need.

Lecture 2 — Transformer-based models & tricks. RoPE (rotating queries and keys so that attention sees only relative distance), GQA (Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, Ainslie et al., 2023), and the encoder-only / decoder-only / encoder-decoder taxonomy.

Lecture 3 — Large Language Models. Mixture of experts replacing the FFNN with $\text{FFNN}_1 \dots \text{FFNN}_n$ plus a gate G , routed per token; the Mixtral analysis of what experts specialise in; sampling, temperature and decoding strategies.

Lecture 4 — LLM training. Kaplan scaling laws; the Chinchilla compute-optimal law; FlashAttention exploiting the HBM/SRAM hierarchy (big and slow versus small and fast); and the lifecycle Pretraining → Finetuning → Preference tuning.

Lecture 5 — LLM tuning. The RL mapping (LLM = agent, tokens = environment, input so far = state, next token = action, next-token probability = policy, human preference = reward); Bradley-Terry; the RLHF loop with a frozen reward model and a trained policy; the PPO objective balancing maximise rewards against don't deviate too much from the old/base model.

Lecture 6 — LLM reasoning. Question → LLM → reasoning chain → answer; chain of thought; GRPO versus PPO; the DeepSeek-R1 recipe.

Lecture 7 — Agentic LLMs. RAG; tool calling (prompt + function API → LLM → arguments to run with the API); agents and the ReAct loop.

Lecture 8 — LLM evaluation. LLM-as-a-Judge (prompt + model response + criteria → rationale + score); the four benchmark categories with MMLU, AIME/PIQA, SWE-bench and HarmBench.

9.2 Transformers beyond text

Why they generalise at all

Context. Introduced in 2017 for machine translation; relies on self-attention; built on the concepts of query, key and value.

Benefits. **Weaker inductive biases** and therefore **more generalisability**.

Intuition — "weaker inductive biases" is the whole answer, and it is a double-edged one.

A convolutional network has strong built-in assumptions: locality (nearby pixels are related) and translation equivariance (a cat is a cat wherever it appears). Those assumptions are correct for images, which is why CNNs learn efficiently from modest datasets. An RNN assumes sequential locality. A Transformer assumes almost nothing — every element can interact with every other, and the relationships are learned from data. On small datasets this is a liability: the model must learn from scratch what a CNN was given for free. On **large** datasets it becomes a decisive advantage, because the model can learn the actual structure of the data rather than being constrained by a hand-designed approximation of it. This is why Transformers took over everywhere at roughly the same time — the datasets crossed the threshold where flexibility beats built-in assumptions.

Attention for images

The attention mechanism does not care what its inputs mean. In text, tokens are word pieces.

For images, the tokens can be **patches**. Everything else — the query/key/value machinery, the softmax, the value averaging — is unchanged.

Vision Transformer (ViT)

Dosovitskiy et al., 2020, An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale.

The end-to-end example from the lecture:

1. Take the image and cut it into fixed-size **patches** of $P \times P$ pixels (16×16 in the paper).
2. **Flatten** each patch into a vector of length $P \cdot P \cdot C$ (C = colour channels) and pass it through a **linear projection** to dimension D . Each patch is now a token.
3. **Prepend a [CLS] token**, exactly as in BERT.
4. Add the patch **embedding** and the **position embedding** → position-aware embeddings.
5. Feed the sequence into the **encoder** (the ViT architecture is an encoder-only Transformer).
6. Read the **encoded embedding at the [CLS] position**, pass it through a **feed-forward network**, and get the class: teddy bear.

Intuition — notice that this is BERT with a different tokeniser. That is not a glib comparison; it is exactly what ViT is. [CLS] for pooling, learned position embeddings, a bidirectional encoder stack, a classification head. The only genuinely new component is the patch projection that turns a grid of pixels into a sequence of vectors. Patching is also what makes it affordable: a 224×224 image has 50,176 pixels, and attention over that many tokens is impossible at $O(n^2)$, but 16×16 patches reduce it to 196 tokens, which is an ordinary sequence length. The paper's central finding was that ViT underperforms CNNs on ImageNet-scale data and outperforms them once pretrained on much larger datasets — the inductive-bias trade-off, measured.

Vision Language Models (VLM)

Goal: "How cute is this teddy bear?" + an image $\rightarrow$ "Very cute!"

Two methods:

Method 1: recycle the decoder-only architecture (Visual Instruction Tuning — LLaVA, Liu et al., 2023). Encode the image into a sequence of vectors (with a vision encoder such as ViT/CLIP), project them into the LLM's embedding space, and **concatenate them with the text tokens**. The LLM then treats image patches as if they were just more tokens in its context — a "typical" LLM, unchanged, fed a mixed sequence.

Method 2: leverage a cross-attention layer (as in The Llama 3 Herd of Models, 2024). Keep the text decoder as it is and insert **cross-attention** layers that attend from the text stream into the image representation, exactly as the original Transformer's decoder attended into its encoder.

Intuition — the trade-off between the two. Method 1 is simpler and cheaper to build: no architectural change, so you can bolt vision onto an existing LLM with a small projection layer and some instruction tuning. Its cost is that image tokens **consume context window** — hundreds or thousands of tokens per image — and they participate in self-attention with everything else, which is expensive. Method 2 keeps image representations outside the main sequence, so the text context is untouched and cost scales better with many or high-resolution images; the price is new parameters and a modified architecture, meaning you cannot simply upgrade a text model in place. Method 1 dominated the open ecosystem for exactly that reason.

Where else Transformers appear

- **Text generation** — CME 295's subject.
- **Vision understanding** — Vision Transformer (ViT).
- **Image generation** — Diffusion Transformer (DiT), MultiModal-DiT (MM-DiT).
- **And many others** — recommendation, speech.

Transformers are actually used in a lot of different places.

9.3 Diffusion LLMs

The limitation being attacked

The current approach is **AutoRegressive Modeling (ARM)**: predict one token, append it, predict the next.

Problem: inference-time generation is not parallelisable — although training is.

Intuition — this is the same complaint that killed RNNs, resurfacing. Lecture 1's argument against recurrence was "you cannot parallelise along the sequence". Transformers fixed that **for training**, where all positions are processed at once with a causal mask. But at inference an autoregressive model is irreducibly sequential: token 100 cannot be computed before token 99 exists. Every trick in Lecture 3 — KV caching, speculative decoding, multi-token prediction — is a way of softening this constraint without removing it. Diffusion LLMs attack the constraint itself.

Alternative in the news

The lecture notes the commercial momentum: an Inception announcement (TechCrunch, November 2025), a diffusion-based LLM showcased by **Google at I/O** (May 2025), and a **ByteDance** announcement (July 2025). This is no longer purely academic.

Background: how image diffusion works

Intuition given in the lecture:

- noise is easy to sample,
- the transformation is learned,
- it is mathematically well-defined.

And the analogy — building a sculpture:

"The sculpture is already complete within the marble block, before I start my work. It is already there, I just have to chisel away the superfluous material." — Michelangelo

High-level goal: learn some transformation that goes from noise to the desired data distribution.

The mechanism (Denoising Diffusion Probabilistic Models, Ho et al., 2020):

1. **Forward process** — add noise to an image, progressively, until it is pure noise. This requires no learning; it is a fixed schedule.
2. **Reverse process** — **learn to denoise** it. Train a network to take a noisy image and predict a slightly less noisy one. At generation time, start from pure noise and run the learned reverse process repeatedly.

From images to text

The translation is one substitution: where images use **noise**, text uses **MASK**.

1. Forward process — mask tokens with some probability:

A teddy bear is reading → A MASK bear is MASK

2. Reverse process — learn to unmask tokens:

A MASK bear is MASK → A teddy bear is reading

MDM = Masked Diffusion Model. At generation time you start from an all-`MASK` sequence of the desired length and iteratively unmask:

MASK MASK MASK MASK MASK —Diffusion LLM→ A teddy bear is reading

Decoding is done in fewer forward passes, because each pass can fill in **many** positions at once, in any order.

(Suggested readings: Lou et al., 2023, on discrete diffusion by estimating data-distribution ratios; Sahoo et al., 2024, Simple and Effective Masked Diffusion Language Models; Nie et al., 2025, Large Language Diffusion Models.)

Discussion

Advantages:

- **better suited for some tasks,**
- **~10× output tokens per second compared to ARM.**

Challenges and current work:

- **adapting ARM-based techniques** — the entire toolbox from the previous eight lectures (KV caching, RLHF, GRPO, tool calling) assumes autoregressive generation and must be reinvented,
- **performance** — quality does not yet match the best autoregressive models.

Intuition — where the speedup comes from, and what it costs. An ARM needs n forward passes to produce n tokens. A diffusion LM needs however many denoising steps you choose — typically far fewer than n — and each step fills in many positions. That is the 10× figure.

There is a second, more conceptual advantage. Autoregressive generation commits irrevocably: once token 5 is emitted it can never be revised, even if token 40 reveals it was a mistake. Diffusion generation is **iterative refinement over the whole sequence**, so it can revisit and change earlier positions. That should be a natural fit for tasks where global structure matters more than left-to-right flow — code editing, infilling, constrained generation, planning.

The cost is that independence assumptions inside each denoising step make it hard to model token dependencies as sharply as a chain rule factorisation does, which is where the current quality gap comes from. Note also how deeply the ecosystem assumes autoregression: KV caching has no direct analogue when every position may change at every step, and even "streaming the response to the user" is a different proposition when the text materialises all at once rather than left to right.

9.4 Cross-pollination between modalities

Trend: modalities trade ideas about concepts that work. The lecture's examples are deliberately symmetric:

Architecture:

- **"Diffusion" training for text output** — an image technique moving to text (Large Language Diffusion Models, Nie et al., 2025).
- **Transformers to deal with images** — a text architecture moving to images (Scalable Diffusion Models with Transformers — DiT, Peebles et al., 2022).

Input representation: DeepSeek-OCR (Contexts Optical Compression, Wei et al., 2025) — render text **as an image** and feed it to a vision encoder, because a picture of a paragraph can be represented in fewer tokens than the paragraph itself. Text becoming an image in order to be cheaper as text.

Tricks: RoPE variants — e.g. Multimodal Scalable RoPE for images (Qwen-Image, Wu et al., 2025), adapting the rotary position idea from 1-D sequences to 2-D grids.

Intuition. Notice the pattern: the architecture went from text to images (DiT), the training objective went from images to text (diffusion LMs), the input representation went from text to images and back (OCR compression), and the positional trick generalised from 1-D to 2-D. The boundary between modalities is dissolving — increasingly there is one toolbox, and modality is just a choice of how you tokenise. DeepSeek-OCR is the most delightfully counterintuitive item: it says that for long documents, an image of the text can be a more efficient representation than the text, which inverts everything you would assume about which modality is compact.

9.5 Back to the basics: foundational research is still open

Foundational research is still very much ongoing.

Microscopic level — design choices still vary widely across papers:

- optimisers: **AdamW vs. MuonClip** vs. others (Kimi K2: Open Agentic Intelligence, Kimi Team, 2025),
- normalisation — where, and which kind, or whether at all,
- MHA/MQA/GQA design choices,
- activation functions,
- MoE or not,
- number of layers.

"Fuel" — future streams of high-quality data? With the reference to The Curse of Recursion: Training on Generated Data Makes Models Forget (Shumailov et al., 2023).

Taking a step back — is the Transformer the best architecture?

Intuition — two things worth taking seriously here. First, the fact that labs still disagree about the optimiser and the normalisation scheme in 2025 tells you that the recipe is empirical, not derived. There is no theory saying AdamW is right; it is what worked. That should make you appropriately sceptical of confident claims about why any of this works.

Second, the data question may be the real ceiling. Scaling laws say loss keeps falling with more data — but high-quality human text is finite and largely already consumed. The obvious work-around, training on model-generated text, has a documented failure mode: **model collapse**, where each generation trained on the previous one loses distributional tails and drifts towards the mean, forgetting the rare and the specific. It is a photocopy of a photocopy. Whether synthetic data can be made safe (by filtering, by verification, by mixing with real data) is one of the genuinely open questions in the field, and it interacts with Lecture 6's rejection sampling, which is a form of verified synthetic data generation and does work.

9.6 New frontiers and hardware

From best performance to best trade-off

Until now: the focus was on **best performance**. **More and more:** the focus is on the **best quality/cost trade-off** — i.e. moving along the **Pareto frontier** from Lecture 8 rather than only pushing its upper-right corner.

Hardware optimisation: analog in-memory computing

Leroux et al., 2025, "Analog in-memory computing attention mechanism for fast and energy-efficient large language models".

Observation: current GPUs optimise for matrix-vector and matrix-matrix operations.

Challenges — Transformer blocks have special needs:

- **attention requires frequent KV reads/writes,**
- **memory movement dominates cost on GPUs.**

Idea: "natively" support attention operations in hardware:

- **store the KV cache in dedicated cells,**
- **use analog signals to model the computations.**

Results: up to $\sim 100\times$ latency and $\sim 70,000\times$ energy savings with respect to NVIDIA's H100.

Intuition — this is FlashAttention's lesson taken to its logical conclusion. Lecture 4 established that attention is bound by memory movement, not arithmetic, and FlashAttention's response was to restructure the algorithm to move less data. Analog in-memory computing's response is more radical: **don't move the data at all — compute where it is stored.** A crossbar of analog memory cells performs a matrix-vector multiply as a physical consequence of applying voltages, in one step, with no fetch. Store the KV cache in those cells and the dot-product-and-weighted-sum that is attention happens in place.

The 70,000× energy figure should be read as an upper bound from a research prototype on a specific operation, not a product claim. Analog computing has real obstacles — limited precision, device drift, difficulty of writing values — which is why it has repeatedly failed to displace digital. But the argument for why attention specifically is a good fit is sound, and energy is becoming the binding constraint on the whole field, which is why this appears in a lecture on trends.

9.7 Applications: today and tomorrow

Daily life has already dramatically changed. Key applications today:

- **coding** ("true" coding, and text-to-query),
- **general conversational assistant** (common facts, web search),
- **creativity**,
- **learning**.

Tomorrow: democratisation of existing agents — e.g. the **Google Workspace Studio** launch (December 2025), "Automate everyday work with AI agents".

Near term: the **browser** — e.g. OpenAI's **Atlas** launch (October 2025). Looking further: an **OS-level LLM**?

Long term: "truly" autonomous agents with vast responsibilities?

Impossible? Actually useful customer service (finally?) — the instructors' joke, and a fair one.

Ongoing challenges

Top of mind:

- **fixed weights, not updated** ($\neq$ continuous learning),
- **"hallucinations"**,
- **personalisation**,
- **interpretability**,
- **safety**.

Intuition — why "fixed weights" is listed first, and why it is the deepest item on the list.

Everything else in this course works around the fact that a model's parameters are frozen at the end of training. RAG works around it by injecting knowledge at inference. Tool calling works around it by fetching live information. Long contexts work around it by holding more in working memory. But none of these is learning: nothing the model encounters in a conversation changes what it knows tomorrow. A human colleague who forgot everything at the end of each day, no matter how capable, would be limited in exactly this way. Genuine continuous learning would collapse the distinction between the pretraining/finetuning/inference stages that this entire course is organised around — which is why it is both the most consequential open problem and the hardest, since updating weights on the fly risks catastrophic forgetting and opens a large safety surface.

9.8 How to stay up to date

Papers:

- **arXiv > Computer Science > Computation and Language** (cs.CL),
- general ML venues (**NeurIPS**, **ICML**, **ICLR**) and NLP venues (**ACL**, **EMNLP**).

Code:

- authors' **GitHub** repositories, linked from their papers,
- **Hugging Face "trending papers"** — for browsing state-of-the-art datasets and methods.

Miscellaneous:

- **Twitter/X** — academics and industry leaders,
- **YouTube** — theoretical (Two Minute Papers, Yannic Kilcher) and practical (Google Developers, HuggingFace, Andrej Karpathy),
- **company/academia technical papers and blogs** — Amazon Science, Anthropic, Apple ML, Google DeepMind, Meta AI, Microsoft AI, OpenAI, Stanford NLP, and others.

Course materials worth keeping

- **VIP Cheatsheet** — github.com/afshinea/stanford-cme-295-transformers-large-language-models, translated into 11 languages (Čeština, English, Español, فارسی, Français, Italiano, 日本語, 한국어, Inu, Türkçe, 中文).
- **Super Study Guide: Transformers & Large Language Models** — the course textbook, superstudy.guide, which is the source of many of the figures used throughout the lectures.

Closing synthesis

If you want a single thread through the nine lectures, it is this. The Transformer replaced recurrence with attention because attention parallelises (L1), and everything since has been the consequences of that choice: engineering around its $O(n^2)$ cost and its position-blindness (L2), scaling it into a language model and making inference affordable (L3), training it at a scale where memory and precision dominate the design (L4), then discovering that a model

which merely predicts text is not a model that behaves well — so aligning it to preferences (L5) and to verifiable correctness (L6) — then discovering that even an aligned model is trapped inside its own weights, so connecting it to knowledge and to actions (L7) — and then discovering that none of it can be improved without a way to measure it (L8). Lecture 9's point is that each of these decisions is still open, including the very first one.